



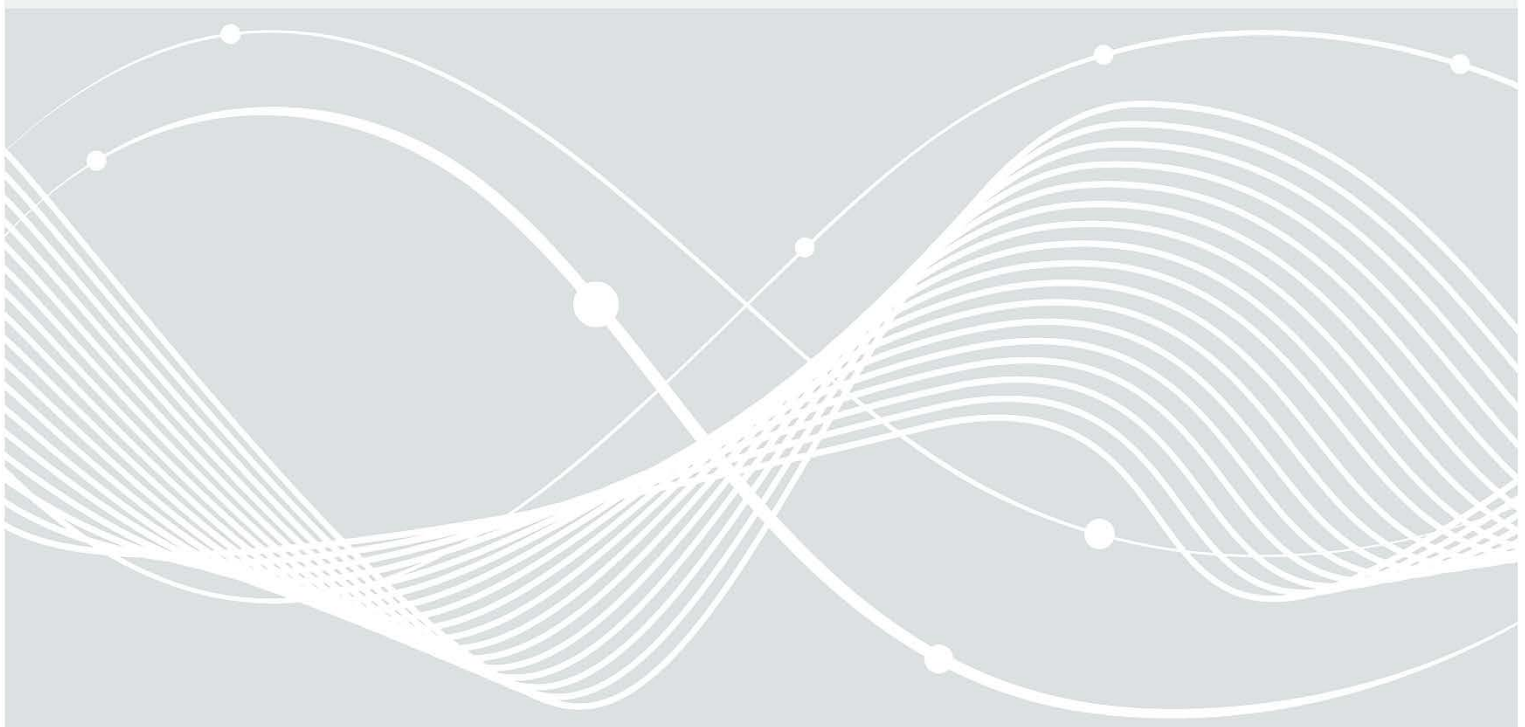
Bundesamt
für Sicherheit in der
Informationstechnik

Deutschland
Digital•Sicher•BSI•

BSI Technische Richtlinie 03125 Beweiswerterhaltung kryptographisch signierter Dokumente

Anlage TR-ESOR-ENC: Profil für die Aufbewahrung von verschlüsselten Inhalten

Bezeichnung	Profil für die Aufbewahrung von verschlüsselten Inhalten
Kürzel	BSI TR-ESOR-ENC
Version	1.3
Datum	29.04.2025



Bundesamt für Sicherheit in der Informationstechnik
Postfach 20 03 63
53133 Bonn
Tel.: +49 22899 9582-0
E-Mail: tresor@bsi.bund.de
Internet: <https://www.bsi.bund.de>
Copyright © Bundesamt für Sicherheit in der Informationstechnik 2025

Inhalt

1	Einführung.....	7
2	Übersicht.....	10
3	Profilierung.....	13
3.1	Profilspezifische Anforderungen	13
3.2	TR-ESOR-ENC-Architektur.....	14
3.3	Zugriff auf die verschlüsselten Dokumente.....	16
3.4	Angepasste Prozesse gem. [TR-ESOR-ENC]	17
3.4.1	Ablage elektronischer Unterlagen	19
3.4.2	Abfrage der bewahrten Daten.....	26
3.4.3	Ändern der bewahrten Daten.....	28
3.4.4	Rückgabe der technischen Beweisdaten	35
3.4.5	Prüfen der beweisrelevanten Daten und technischen Beweisdaten	36
3.4.6	Vernichten von Archivinformationspaketen, inkl. Daten.....	41
3.4.7	Erneuerung des Hashbaums.....	44
3.4.8	Erneuerung des Schlüsselmaterials.....	48
3.4.9	Änderung der Zugriffsberechtigung inkl. Entfernung der alten Zugriffsberechtigung.....	61
3.5	Profilierung der verwendeten Schnittstellen [TR-ESOR-E]	67
3.5.1	Die Schnittstelle TR-S.3'	68
3.5.2	Schnittstelle des lokalen Up-Download-Moduls.....	75
3.5.3	Schnittstelle TR-S.5'	77
3.6	Profilierung der notwendigen Datenformate [TR-ESOR-F]	77
3.6.1	Rollenreferenzen für Zugriffssteuerung	78
3.6.2	Referenz auf ein Z-AIP.....	79
3.6.3	Aufbau eines Z-AIP	80
3.6.4	Aufbau von D-AIP	81
3.6.5	Referenz auf das Klartextdokument (DLRef)	82
4	Abkürzungsverzeichnis.....	83
5	Anhang – Umgang mit VS-NfD eingestufte Information.....	84
5.1	Einführung.....	84
5.2	Architektur (VS-NfD reduziert).....	85
5.3	Prozesse (VS-NfD reduziert)	87
5.3.1	Ablage elektronischer Unterlagen (VS-NfD reduziert).....	87
5.3.2	Abfrage der bewahrten Daten (VS-NfD reduziert)	89
5.3.3	Ändern der bewahrten Daten (VS-NfD reduziert).....	89
5.3.4	Rückgabe der technischen Beweisdaten (VS-NfD reduziert).....	90
5.3.5	Prüfen der beweisrelevanten Daten und technischen Beweisdaten (VS-NfD reduziert)	90

5.3.6	Vernichten von Archivinformationspaketen, inkl. Daten (VS-NfD reduziert).....	91
5.3.7	Erneuerung des Hashbaums (VS-NfD reduziert)	92
5.3.8	Erneuerung des Schlüsselmaterials (VS-NfD reduziert)	93
5.3.9	Änderung der Zugriffsberechtigung inkl. Entfernung der alten Zugriffsberechtigung (VS-NfD reduziert).....	95
5.4	Schnittstellen (VS-NfD reduziert)	95
5.4.1	Die Schnittstelle TR-S.1' (VS-NfD reduziert)	95
5.4.2	Die Schnittstelle TR-S.3' (VS-NfD reduziert)	96
5.5	Datenformate (VS-NfD reduziert).....	97
5.5.1	Aufbau von D-AIP (VS-NfD reduziert).....	97
5.5.2	Referenzen auf Klartextdokumente (VS-NfD reduziert).....	98
6	Anhang – Beispielhafter Anwendungsfall BNotK [informativ].....	99
6.1	Allgemeines	99
6.2	Aufbau.....	99
6.3	Anwendungsfälle	100
6.3.1	Initiale Ablage	100
6.3.2	Abruf verschlüsselter Dokumente	101
6.3.3	Umschlüsselung verschlüsselter Dokumente	101
6.4	Datenfluss.....	101
7	Anhang – XML-Schema-Definition	103

Abbildungen

Abbildung 1: Schematische Darstellung der IT-Referenzarchitektur mit TR-S.4 .	8
Abbildung 2: Schematische Darstellung der IT-Referenzarchitektur mit TR-S.512 .	8
Abbildung 3: Basisarchitektur [TR-ESOR-ENC]	14
Abbildung 4: Grobe Architektur der lokalen TR-ESOR-Middleware.	15
Abbildung 5: Schematische Darstellung der IT-Referenzarchitektur mit TR-S.4 gem. [TR-ESOR-ENC].	15
Abbildung 6: Schematische Darstellung der IT-Referenzarchitektur mit TR-S.512 gem. [TR-ESOR-ENC].	16
Abbildung 7: Steuerung des Zugriffs auf einzelnen verschlüsselten Dokumenten – Beispiel.	17
Abbildung 8: Ablage gem. [TR-ESOR-ENC].	20
Abbildung 9: Abfrage inkl. Entschlüsselung gem. [TR-ESOR-ENC].	26
Abbildung 10: Änderung gem. [TR-ESOR-ENC].	29
Abbildung 11: Verifikation der Beweisdaten gem. [TR-ESOR-ENC].	36
Abbildung 12: Vernichten von AIPs inkl. zugehörigen Daten gem. [TR-ESOR-ENC].	41
Abbildung 13: Hashwertberechnung bei Hashbaumerneuerung gem. [TR-ESOR-ENC].	45
Abbildung 14: Migration eines D-AIP im Zuge der Erneuerung der symmetrischen Schlüssel - Beispiel.	49
Abbildung 15: Erneuerung der symmetrischen Schlüssel gem. [TR-ESOR-ENC].	51
Abbildung 16: Erneuerung der asymmetrischen Schlüssel gem. [TR-ESOR-ENC].	59
Abbildung 17: Erweiterung der Zugriffsberechtigung gem. [TR-ESOR-ENC].	62
Abbildung 18: Einschränkung der Zugriffsberechtigung gem. [TR-ESOR-ENC].	65
Abbildung 19: Umsetzung der IT-Referenzarchitektur von [TR-ESOR-ENC] auf Basis von TR-S.4 .	67
Abbildung 20: Umsetzung der IT-Referenzarchitektur von [TR-ESOR-ENC] auf Basis von TR-S.512 .	68
Abbildung 21: Definition der Rollenreferenzen als xaip:RecipientInfos-XML-Struktur.	78
Abbildung 22: Definition des AccessControlAIPReference-Elements.	79
Abbildung 23: Struktur eines Z-AIP – Beispiel (vereinfachte Darstellung).	80
Abbildung 24: Struktur eines D-AIP – Beispiel.	81
Abbildung 25: Beispielhafter Anwendungsfall: Speicherung VS-NfD-Information in einem nicht VS-TR-ESOR.	84
Abbildung 26: Speicherung VS-NfD-Information in einem nicht VS-TR-ESOR gem. [TR-ESOR-ENC] - Standardansatz.	84
Abbildung 27: Speicherung VS-NfD-Information in einem nicht VS-TR-ESOR gem. [TR-ESOR-ENC], reduzierter Ansatz.	85
Abbildung 28: Schematische Darstellung der IT-Referenzarchitektur mit TR-S.4 gem. [TR-ESOR-ENC], reduziert.	86
Abbildung 29: Schematische Darstellung der IT-Referenzarchitektur mit TR-S.512 gem. [TR-ESOR-ENC], reduziert.	86
Abbildung 30: Ablage gem. [TR-ESOR-ENC] (VS-NfD reduziert).	87
Abbildung 31: Änderung gem. [TR-ESOR-ENC] (VS-NfD reduziert).	89
Abbildung 32: Verifikation der Beweisdaten gem. [TR-ESOR-ENC] (VS-NfD reduziert).	90
Abbildung 33: Vernichten von AIPs inkl. zugehörigen Daten gem. [TR-ESOR-ENC] (VS-NfD reduziert).	91
Abbildung 34: Hashwertberechnung bei Hashbaumerneuerung gem. [TR-ESOR-ENC] (VS-NfD reduziert).	92
Abbildung 35: Erneuerung der symmetrischen Schlüssel gem. [TR-ESOR-ENC] (VS-NfD reduziert).	93
Abbildung 36: Erneuerung der asymmetrischen Schlüssel gem. [TR-ESOR-ENC] (VS-NfD reduziert).	94
Abbildung 37: Einschränkung der Zugriffsberechtigung gem. [TR-ESOR-ENC] (VS-NfD reduziert).	95
Abbildung 38: Beispiel eines D-AIP mit zwei enc:PlainTextProxy-Elementen (VS-NfD reduziert).	98
Abbildung 39: Struktur des enc:PlainTextProxy-Element gem. [TR-ESOR-ENC] (VS-NfD reduziert).	98
Abbildung 40: Vereinfachte Darstellung der Architektur des Urkundenarchivs der Notare.	100

Tabellen

Tabelle 1: Erklärung zu Artefakten in Ablaufdiagrammen.	19
Tabelle 2: Ablage elektronischer Unterlagen – Beschreibung zur Abbildung 8.	25
Tabelle 3: Abfrage inkl. Entschlüsselung – Beschreibung zur Abbildung 9.	28
Tabelle 4: Änderung der bewahrten Daten – Beschreibung zur Abbildung 10	35
Tabelle 5: Verifikation der Beweisdaten – Beschreibung zur Abbildung 11.	41
Tabelle 6: Vernichten der aufbewahrten Daten – Beschreibung zur Abbildung 12.	44
Tabelle 7: Hashwertberechnung bei der Hashbaumerneuerung – Beschreibung zur Abbildung 13.....	48
Tabelle 8: Erneuerung der symmetrischen Schlüssel - Beschreibung zur Abbildung 15.	58
Tabelle 9: Erneuerung der asymmetrischen Schlüssel - Beschreibung zur Abbildung 15	61
Tabelle 10: Erweiterung der Zugriffsberechtigung - Beschreibung zur Abbildung 17.	64
Tabelle 11: Einschränkung der Zugriffsberechtigung – Beschreibung zur Abbildung 18.	67
Tabelle 12: Änderungen in der Abbildung 30 bezogen auf die Abbildung 8.	88
Tabelle 13: Änderungen in der Abbildung 31 bezogen auf die Abbildung 10.	90
Tabelle 14: Änderungen in Abbildung 32 bezogen auf Abbildung 11.	91
Tabelle 15: Änderungen in der Abbildung 33 bezogen auf Abbildung 12.....	91
Tabelle 16: Änderungen in der Abbildung 34 bezogen auf die Abbildung 13.	92
Tabelle 17: Änderungen in der Abbildung 35 bezogen auf die Abbildung 15.	94
Tabelle 18: Änderungen in der Abbildung 36 bezogen auf die Abbildung 16.	94
Tabelle 19: Änderung in der Abbildung 37 bezogen auf die Abbildung 17.....	95

1 Einführung

Ziel der Technischen Richtlinie „Beweiswerterhaltung kryptographisch signierter Dokumente“ ist die Spezifikation sicherheitstechnischer Anforderungen für den langfristigen Beweiswerterhalt von kryptographisch signierten elektronischen Dokumenten und Daten nebst zugehörigen elektronischen Verwaltungsdaten (Metadaten).

Eine für diese Zwecke definierte Middleware (TR-ESOR-Middleware) im Sinn dieser Richtlinie umfasst alle diejenigen Module (M) und Schnittstellen (S), die zur Sicherung und zum Erhalt der Authentizität und zum Nachweis der Integrität der aufbewahrten Dokumente und Daten eingesetzt werden.

Die im Hauptdokument dieser Technischen Richtlinie (vgl. [TR-ESOR]) vorgestellte Referenzarchitektur besteht aus den nachfolgend beschriebenen Schnittstellen, Funktionen und logischen Einheiten:

- der **TR-S.4**- oder ETSI TS 119 512-Schnittstelle **TR-S.512** in der Profilierung [TR-ESOR-TRANS] der TR-ESOR-Middleware, die dazu dient, die TR-ESOR-Middleware in die bestehende IT- und Infrastrukturlandschaft einzubetten;
- dem „ArchiSafe-Modul“ (vgl. [TR-ESOR-M.1]), welches den Informationsfluss in der Middleware regelt, die Sicherheitsanforderungen an die Schnittstellen zu den IT-Anwendungen umsetzt und für eine Entkopplung von Anwendungssystemen und ECM-/Langzeitspeicher sorgt;
- dem „Krypto“-Modul (vgl. [TR-ESOR-M.2]) nebst den zugehörigen Schnittstellen **TR-S.1** und **TR-S.3**, das alle erforderlichen Funktionen zur Berechnung von Hashwerten, zur Prüfung elektronischer Signaturen bzw. Siegel bzw. Zeitstempel, zur Nachprüfung elektronischer Zertifikate und zum Einholen qualifizierter Zeitstempel sowie (optional) elektronischer Signaturen bzw. Siegel für die Middleware zur Verfügung stellt. Darüber hinaus kann es Funktionen zur Ver- und Entschlüsselung von Daten und Dokumenten zur Verfügung stellen;
- dem „ArchiSig-Modul“ (vgl. [TR-ESOR-M.3]) mit der Schnittstelle **TR-S.6**, das die erforderlichen Funktionen für die Beweiswerterhaltung der digital signierten Unterlagen bereitstellt;
- einem ECM-/Langzeitspeicher mit den Schnittstellen **TR-S.2** und **TR-S.5**, der die physische Archivierung/Aufbewahrung und auch das Speichern der beweiswerterhaltenden Zusatzdaten übernimmt.

Dieser ECM-/Langzeitspeicher ist nicht mehr direkt Teil der Technischen Richtlinie, gleichwohl werden über die beiden Schnittstellen, die noch Teil der TR-ESOR-Middleware sind, Anforderungen daran gestellt.

Ebenso wenig ist die Applikationsschicht, die auch einen XML-Adapter enthalten kann, direkter Teil der Technischen Richtlinie, auch wenn dieser XML-Adapter als Teil einer Middleware implementiert werden kann.

Die empfohlene IT-Referenzarchitektur ist in Abbildung 1 und Abbildung 2 dargestellt und besteht im Wesentlichen aus den in [TR-ESOR], Kap. 7 grob beschriebenen logischen Komponenten und Schnittstellen. Diese werden in Anhängen zur TR weiter detailliert.

Die Grafik zeigt zudem die externen Komponenten und Systeme an, die das Bild vervollständigen. Grundsätzlich wird als obere Schnittstelle der TR-ESOR-Middleware entweder die **TR-S.4**-Schnittstelle gemäß [TR-ESOR-E], die in Abbildung 1 dargestellt ist, oder die **TR-S.512**-Schnittstelle gemäß [ETSI TS 119 512] in der Profilierung [TR-ESOR-TRANS], die in Abbildung 2 gezeigt wird, unterstützt.

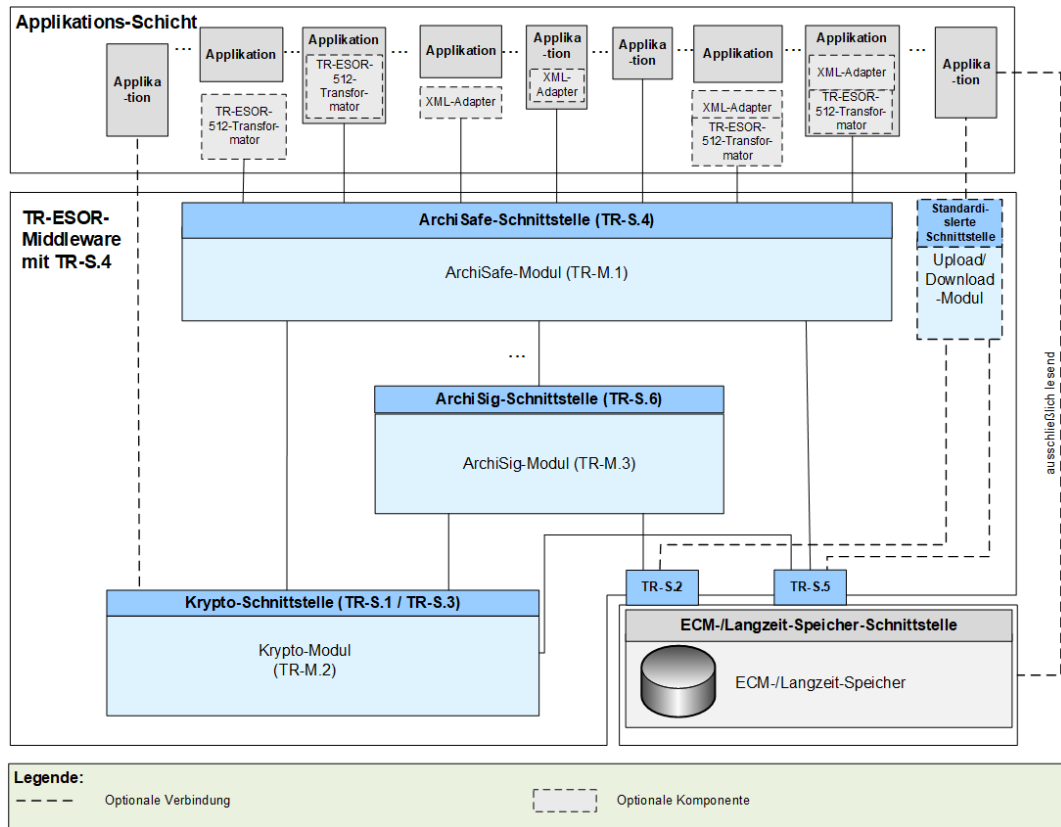


Abbildung 1: Schematische Darstellung der IT-Referenzarchitektur mit **TR-S.4**.

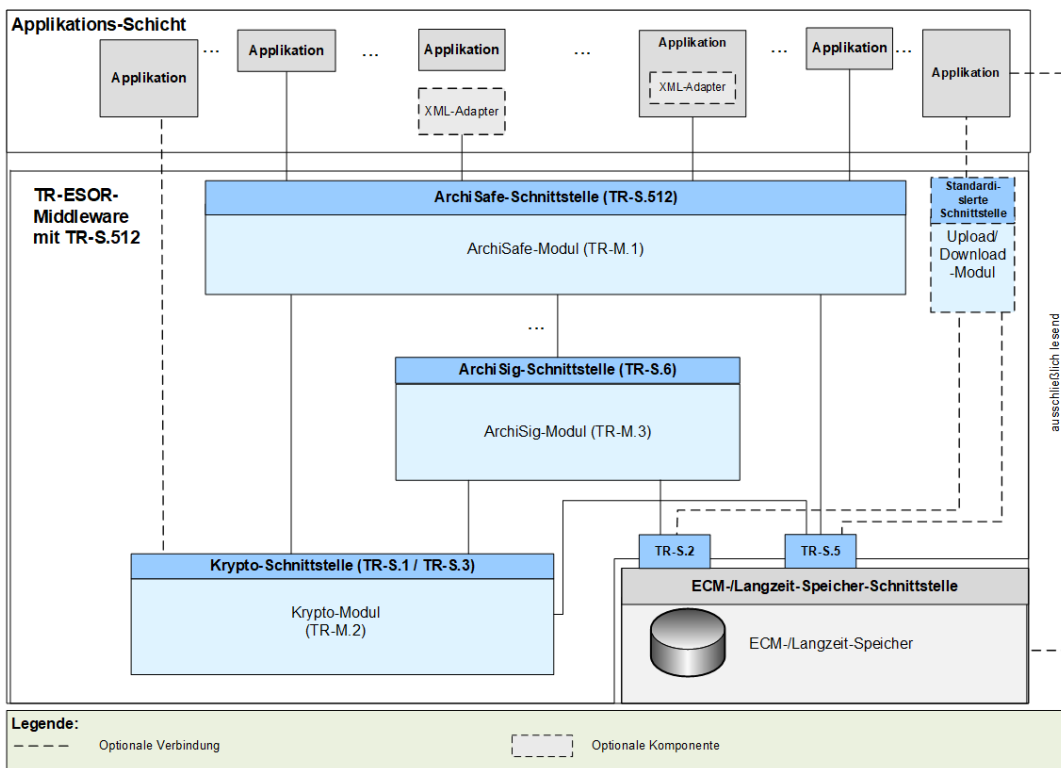


Abbildung 2: Schematische Darstellung der IT-Referenzarchitektur mit **TR-S.512**.

Die in Abbildung 1 bzw. Abbildung 2 dargestellte IT-Referenzarchitektur orientiert sich an der ArchiSafe Referenzarchitektur und soll die logische (funktionale) Interoperabilität künftiger Produkte mit den Zielen und Anforderungen der Technischen Richtlinie ermöglichen und unterstützen.

Sofern der optionale XML-Adapter und/oder der optionale TR-ESOR-512-Transformator¹ vorhanden sind, können beide in folgenden Ausprägungen vorliegen:

- Jeweils eigenständige Komponente mit Schnittstellen zur Applikation sowie zum ArchiSafe-Modul
- Jeweils eigenständige Komponente, jedoch Teil der Applikation mit Schnittstelle zum ArchiSafe-Modul
- XML-Adapter und TR-ESOR-512-Transformator als eine gemeinsame Komponente, die beide Teile enthält mit Schnittstellen zur Applikation sowie zum ArchiSafe-Modul
- XML-Adapter und TR-ESOR-512-Transformator als eine gemeinsame Komponente, die beide Teile enthält und Teil der Applikation ist, mit Schnittstelle zum ArchiSafe-Modul.

Der “ETSI TS119512 TR-ESOR Transformator” ermöglicht Bewahrungsdiensten gemäß [eIDAS-VO], empfangene ETSI TS119512 (V1.1.2) Nachrichten² in TR-S4 Nachrichten zu transformieren. Diese Nachrichten können dann an ein angeschlossenes TR-ESOR-System³ geschickt werden, ohne irgendwelche Änderungen dieses TR-ESOR-Systems.

Der Einsatz des TR-ESOR-512-Transformators wird EMPFOHLEN, sofern das TR-ESOR-Produkt mit einer **TR-S.4**-Schnittstelle in Europa zum Einsatz kommt und Interoperabilität mit europäischen (qualifizierten) Bewahrungsdiensten und Bewahrungsprodukten hergestellt werden soll.

Diese Technische Richtlinie ist modular aufgebaut und spezifiziert in einzelnen Anlagen zum Hauptdokument die funktionalen und sicherheitstechnischen Anforderungen an die erforderlichen IT-Komponenten und Schnittstellen der TR-ESOR-Middleware. Die Spezifikationen sind strikt plattform-, produkt-, und herstellerunabhängig.

Das vorliegende Dokument trägt die Bezeichnung „Anlage TR-ESOR-ENC“ und erweitert sowie konkretisiert die Anforderungen, Datenformate, Protokolle und Architektur für die beweiswerterhaltende Aufbewahrung kryptografisch signierter Daten und Dokumente speziell für die Anwendungsfälle, in denen die Daten bzw. Dokument zusätzlich verschlüsselt aufbewahrt werden müssen. Insbesondere der Anwendungsfall, in dem das zentrale Aufbewahrungssystem die aufzubewahrenden Inhalte im Klartext nicht verarbeiten darf, stellt die Grundlage für dieses Profil dar. Die entsprechenden profilspezifischen Anforderungen und Annahmen werden hierzu im Kap. 3.1 beschrieben. Als zwei möglichen Anwendungsfälle wurden die Ablage von VS-NfD-eingestufte Information in einem nicht VS-NfD-zugelassenen TR-ESOR-System (vgl. Kap. 5), sowie der Umgang mit Notarurkunden (vgl. Kap. 6) vorgestellt. Das Notarurkunden Szenario wurde bereits umgesetzt und ist bei der Bundesnotarkammer im Einsatz.

¹ Siehe „[Freier ETSI TS 119512 TR-ESOR Transformator unter einer Open Source Lizenz](#)“.

² In der Profilierung von [TR-ESOR-TRANS

³ Siehe <https://www.bsi.bund.de/EN/tr-esor> oder <https://www.bsi.bund.de/DE/tr-esor>.

2 Übersicht

Nicht nur die öffentliche Verwaltung, auch Unternehmen stehen zunehmend vor der Herausforderung, für immer mehr elektronisch erzeugte, verarbeitete und gespeicherte Dokumente und Daten zunehmend neben der Verfügbarkeit, der Lesbarkeit sowie Integrität und Authentizität auch die Vertraulichkeit langfristig zu gewährleisten⁴.

In einigen Anwendungsfällen kann es daher dazu kommen, dass in einem [TR-ESOR]-System die aufzubewahrenden Daten nicht im Klartext sondern nur verschlüsselt verarbeitet werden dürfen, z. B. als verschlüsselte Ablage von Informationen der Einstufung VS-NfD, ohne dass für das TR-ESOR-System eine VS-NfD konforme Infrastruktur aufgebaut worden ist (Stichwort: VS-NfD-eingestufte Daten, vgl. hierzu Kap. 5), oder falls diese Daten generell nur durch bestimmte Personen einsehbar sind (Beispiel: Notarurkunden).

Um die geforderte Vertraulichkeit der zu schützenden Daten zu gewährleisten, werden diese im Vorfeld der Ablage im [TR-ESOR]-System durch den Datenerzeuger im Rahmen der Applikationsschicht bzw. im XML-Adapter entsprechend verschlüsselt.

Der hier spezifizierte Lösungsansatz geht davon aus, dass die verschlüsselten Archivdatenobjekte oder die Archivdatenobjekte der Einstufung VS-NfD seitens des Datenerzeugers bzw. **der anbietenden Stellen** (VSA-konform), im Rahmen Ihrer Applikationsschicht bzw. im XML-Adapter⁵ zu verschlüsseln sind, dann als verschlüsselte Daten an den beauftragten IT-Dienstleister für den Betrieb des TR-ESOR-Systems zu übermitteln sind und dort entsprechend der technischen Vorgaben an das TR-ESOR-System übergeben werden und von TR-ESOR hinsichtlich der Beweiswerterhaltung bearbeitet werden und verschlüsselt im ECM-/Langzeitspeicher abgelegt werden.

So hat der Datenerzeuger bzw. die anbietende Stelle die ausschließliche Kontrolle über seine/ihre Daten.

Der Datenerzeuger bzw. die anbietende Stelle kann die Kontrolle über seine/ihre Daten auch einem Nachfolger oder zusätzlich Berechtigten übergeben.

Der Datenerzeuger bzw. die anbietende Stelle wird im folgenden Text auch Zugriffsberechtigter genannt.

Die Verschlüsselung hat aber zur Folge, dass das [TR-ESOR]-System bestimmte Anforderungen (z. B. Signaturprüfung etc.) nicht erfüllen kann und der schützende Bewahrungsmechanismus sich ausschließlich auf die verschlüsselten Daten und nicht die Klartextdaten beziehen würde (es können die Hashwerte ausschließlich über die verschlüsselten Daten berechnet werden, die Klartextdaten liegen dem [TR-ESOR]-System nicht vor).

Nachfolgende Abschnitte dieses Dokuments beschreiben diesen vorgenannten Ansatz und zeigen auf, welche Teile der [TR-ESOR]-Spezifikation in welchem Umfang abgeändert bzw. ergänzt werden müssen, um den o.g. Anforderungen gerecht zu werden.

HINWEIS 1

In der vorliegenden TR-ESOR-Version 1.3 werden die zwei Begriffe „(beweiswerterhaltende) Langzeitspeicherung“ und „(beweiswerterhaltende) Bewahrung“ synonym verwendet. Ebenso werden die zwei Begriffe „Archivinformationspaket (AIP)“ und „Archivdatenobjekt“ sowie die Begriffe „aufbewahren“ und „archivieren“ synonym verwendet.

⁴ REGULATION OF THE EUROPEAN PARLIAMENT AND OF THE COUNCIL amending Regulation (EU) No 237 910/2014 as regards establishing the European Digital Identity Framework, Brussels, 11 April 2024, 238 European Union, PE-CONS 68/1/23 REV 1, Article 3 (48).

⁵ Siehe Abbildung 1 bzw. 2.

HINWEIS 2

TR-ESOR spezifiziert ein Bewahrungsprodukt (engl. Preservation Product) gemäß [ETSI SR 019 510], [ETSI TS 119 511] und [ETSI TS 119 512] und [eIDAS-VO].

Die TR 03125 TR-ESOR ist in [ETSI SR 019 510] in den Kapiteln 4.7.3, 5.2 und B3.2 beschrieben.

Die in TR-ESOR erforderlichen grundlegenden Bewahrungstechniken, z. B. das Bewahrungsprotokoll, das Beweisdaten-Format Evidence Record, die Archivdaten-Format (L)XAIP und ASiC-AIP sind in [ETSI TS 119 512] als **normative Elemente** enthalten.

HINWEIS 3

Die obere TR-ESOR-Eingangsschnittstelle **TR-S.4** oder die TS119512-Eingangsschnittstelle **TR-S.512** gemäß der „Preservation-API“ in [ETSI TS 119 512] in der Profilierung von [TR-ESOR-TRANS], die logisch äquivalent zur Eingangsschnittstelle **TR-S.4** gem. [TR-ESOR-E] ist, wie in der Tabelle 3 in [TR-ESOR-E], Kapitel 4.1 dargestellt, muss benutzt werden. Eine andere Eingangsschnittstelle anstelle von **TR-S.4** bzw. **TR-S.512** ist nicht erlaubt (vgl. A7.1-1 in [TR-ESOR]).

HINWEIS 4

In der vorliegenden TR-ESOR-Version 1.3 umfasst der Begriff „Archivinformationspaket“ (AIP) in allen TR-ESOR-Anhängen:

- a) das Archivdatenobjekt „XAIP“ gemäß [TR-ESOR-F], Kap. 3.1 als auch
- b) das logische XAIP „LXAIP“ gemäß [TR-ESOR-F], Kap. 3.2 und
- c) das „ASiC-AIP“ gemäß [TR-ESOR-F], Kap. 3.3 auf Basis von [ETSI EN 319162-1].

In TR-ESOR Version V1.3 wird zwischen XAIP, LXAIP und ASiC-AIP differenziert.

Mit (L)XAIP wird XAIP oder LXAIP bezeichnet.

HINWEIS 5

In dieser TR-ESOR Version 1.3 ist „BIN“ beschränkt auf die folgenden Bewahrungsobjekt-Formate (engl. preservation object formats):

- CAdES gemäß [ETSI TS 119 512], Annex A.1.1 (<http://uri.etsi.org/ades/CAdES>). Sofern kein MIME Type gesetzt ist, wird als Default application/cms verwendet;
- XAdES gemäß [ETSI TS 119 512], Annex A.1.2 (<http://uri.etsi.org/ades/XAdES>). Sofern kein MIME Type gesetzt ist, wird als Default application/xml verwendet;
- PAdES gemäß [ETSI TS 119 512], Annex A.1.3 (<http://uri.etsi.org/ades/PAdES>). Sofern kein MIME Type gesetzt ist, wird als Default application/pdf verwendet;
- ASiC-E gemäß [ETSI TS 119 512], Annex A.1.4 (<http://uri.etsi.org/ades/ASiC/type/ASiC-E>). Sofern kein MIME Type gesetzt ist, wird als Default [application/vnd.etsi.asic-e+zip](http://uri.etsi.org/ades/ASiC/type/ASiC-E) verwendet;
- ASiC-S gemäß [ETSI EN 319 512] (<http://uri.etsi.org/ades/ASiC/type/ASiC-S>). Sofern kein MIME Type gesetzt ist, wird als Default application/vnd.etsi.asic-s+zip verwendet.
- DigestList gemäß [ETSI TS 119 512], Annex A.1.6 (<http://uri.etsi.org/19512/format/DigestList>). Sofern kein MIME Type gesetzt ist, wird als Default application/xml verwendet;
- ASiC-ERS (in TR-ESOR v1.3 mit ASiC-AIP bezeichnet) gemäß [BSI TR-ESOR-F] Kap. 3.3 und gemäß [ETSI TS 119 512], Annex A.3.1 (<http://uri.etsi.org/ades/ASiC/type/ASiC-ERS>).

Im Falle Upload/Download-Funktion ist zusätzlich nachfolgendes Format erlaubt:

- Binärdaten (BIN) als „Octet Stream“, die ausschließlich in den ECM-/Langzeitspeicher mit „Upload-Request“ gespeichert werden, – aber nur, sofern:
 - a) verbunden mit einem korrespondierenden LXAIP und dort referenziert gem. [TR-ESOR-F], Kap. 3.2,
 - b) ggf. mit „Download-Request“ ausgelesen werden – verbunden mit einem korrespondierenden LXAIP, das mit der ArchiveRetrieval-Funktion ausgelesen wurde, oder eingebettet in einem XAIP und ausgelesen mit der „ArchiveRetrieval“-Funktion.
 - c) Der Upload von XAIP, oder LXAIP, oder ASiC-AIP ist nicht zugelassen.

HINWEIS 6

Im folgenden Text umfasst der Begriff „**Digitale Signatur**“:

- „fortgeschrittene elektronische Signaturen“ gemäß [eIDAS-VO], Artikel 3 Nr. 11,
- „qualifizierte elektronische Signaturen“ gemäß [eIDAS-VO], Artikel 3 Nr. 12,

- „fortgeschrittenen elektronische Siegel“ gemäß [eIDAS-VO], Artikel 3 Nr. 26 und
- „qualifizierte elektronische Siegel“ gemäß [eIDAS-VO], Artikel 3 Nr. 27.

Insofern umfasst der Begriff „digital signierte Dokumente“ sowohl solche, die fortgeschrittene elektronische Signaturen oder Siegel bzw. qualifizierte elektronische Signaturen oder Siegel tragen.

Mit dem Begriff der „**kryptographisch signierten Dokumente**“ sind in dieser TR neben:

- den gemäß [eIDAS-VO], Artikel 3 Nr. 12 qualifiziert signierten,
- den gemäß [eIDAS-VO], Artikel 3 Nr. 27 qualifiziert gesiegelten oder
- den gemäß [eIDAS-VO], Artikel 3 Nr. 34 qualifiziert zeitgestempelten Dokumenten (im Sinne der eIDAS-Verordnung)

auch Dokumente:

- mit einer fortgeschrittenen Signatur gemäß [eIDAS-VO], Artikel 3 Nr. 11 oder
- mit einem fortgeschrittenen Siegel gemäß [eIDAS-VO], Artikel 3 Nr. 26 oder
- mit einem elektronischen Zeitstempel gemäß [eIDAS-VO], Artikel 3 Nr. 33

erfasst, wie sie oft in der internen Kommunikation von Behörden entstehen.

Nicht gemeint sind hier Dokumente mit einfachen Signaturen oder Siegeln basierend auf anderen (z. B. nicht-kryptographischen) Verfahren.

3 Profilierung

Nachfolgend werden die notwendigen Anpassungen und Erweiterungen an TR-ESOR-Architektur, -Prozessen, -Anforderungen, dargestellt.

Hinweis!

Gem. [TR-ESOR] werden zwei unterschiedlichen Eingangsschnittstellen **TR-S.4** und **TR-S.512** unterstützt. Nachfolgend wird für die Beschreibung dieses Profil (ohne Einschränkung der Allgemeinheit) ausschließlich die **TR-S.4**-Schnittstelle verwendet. Die verwendeten Funktionen der **TR-S.4**-Schnittstelle können im Falle des beabsichtigten Einsatzes der **TR-S.512**-Schnittstelle mit Hilfe der im [TR-ESOR-E], Kap. 4 definierten Abbildung entsprechend ersetzt werden.

3.1 Profilspezifische Anforderungen

Folgende wesentliche fachliche Anforderungen müssen erfüllt werden:

- A.1** Die Berechtigung zum Zugriff auf die zu schützenden Dokumente⁶ muss kryptographisch abgebildet werden. Der Einsatz von hybriden Verschlüsselungsverfahren und entsprechender Zertifikate wird hierzu empfohlen (vgl. [RFC5652], Kap. 6). Die Granularität der Berechtigungen muss dem Anwendungsfall angepasst werden (vgl. hierzu Kap 4). Die Berechtigung kann sich im Laufe der Bewahrung ändern. Es können neue Berechtigungen erteilt oder bestehende zurückgezogen werden.
- A.2** Der Zugriff auf die zu schützende Information im Klartext darf ausschließlich unter voller Kontrolle eines Zugriffsberechtigten erfolgen. Insbesondere die zentrale TR-ESOR-Middleware darf zu keinem Zeitpunkt der Bewahrung einen Zugriff auf die zu schützende Klartextinformation erhalten⁷. Es dürfen ausschließlich verschlüsselte zu schützenden Informationen in der zentralen TR-ESOR-Middleware abgelegt werden.
- A.3** Für die Berechnung der für den Aufbau eines Hashbaums benötigten Hashwerte muss die Klartextinformation herangezogen werden.
- A.4** Die im Klartext lokal vorliegende Signaturobjekte müssen vor der Ablage einer erfolgreichen Prüfung unterzogen werden. Die Prüfdaten müssen zusammen mit den Signaturobjekten zur Aufbewahrung abgelegt werden.

Folgende weitere abgeleitete Anforderungen müssen erfüllt werden:

- A.5** Die für die Zugriffsregelung mittels Verschlüsselung eingesetzte asymmetrische Kryptographie (vgl. **A.1**) wird nicht durch die Middleware überwacht. Die entsprechende Überwachung muss hierfür durch die im Anwendungsfall führende Fachanwendung (bzw. eine Drittanwendung) zugesichert werden. Die dafür notwendige Unterstützung innerhalb der Middleware ist im Kap. 3.4.8.2 dargestellt.
- A.6** Die für die Zugriffsregelung eingesetzte symmetrische Kryptographie (vgl. **A.1**) wird nicht durch die Middleware überwacht. Die entsprechende Überwachung muss hierfür durch die im Anwendungsfall führende Fachanwendung (bzw. eine Drittanwendung) zugesichert werden. Die dafür notwendige Unterstützung innerhalb der Middleware ist im Kap. 3.4.8.1 beschrieben.

⁶ Grundsätzlich ist es nicht limitiert, dass nur die Datenobjekte verschlüsselt abgelegt werden. Es ist durchaus möglich, dass auch Metadaten und/oder Credentials bei Bedarf gleichwohl verschlüsselt abgelegt werden. Ein AIP darf sowohl verschlüsselte als auch unverschlüsselte Objekte gleichermaßen beinhalten.

⁷ Sollte die zentrale TR-ESOR-Middleware die verschlüsselten Objekte zentral entschlüsseln dürfen, so ist der Anwendungsfall bereits mit aktuellen Mitteln unter der Verwendung der korrespondierenden Transformation-Elemente zu bewältigen (vgl. hierzu [TR-ESOR-F], Kap. 3.1.4).

Hinweis!

Die Erneuerung der kryptographischen Schlüssel muss stets vor dem Ablauf ihrer Sicherheitseignung erfolgen.

- A.7** Die eingesetzte TR-ESOR-Middleware muss das AIP-Format LXAIP vollumfänglich unterstützen. Einige Daten, insbesondere die Referenzen auf die ausschließlich lokal gehaltenen Dokumente, können nur auf diesem Weg abgebildet werden.

3.2 TR-ESOR-ENC-Architektur

Um den Anforderungen aus dem Kap. 3.1 gerecht zu werden, wurde unabhängig von der Art der Eingangsschnittstelle, hier **TR-S.4** oder **TR-S.512**, eine einheitliche Basisarchitektur für **[TR-ESOR-ENC]** erarbeitet (siehe Abbildung 3). Diese erweitert die im **[TR-ESOR]**, Kap. 3.3, Abbildung 1 dargestellte allgemeine Basisarchitektur TR-ESOR.

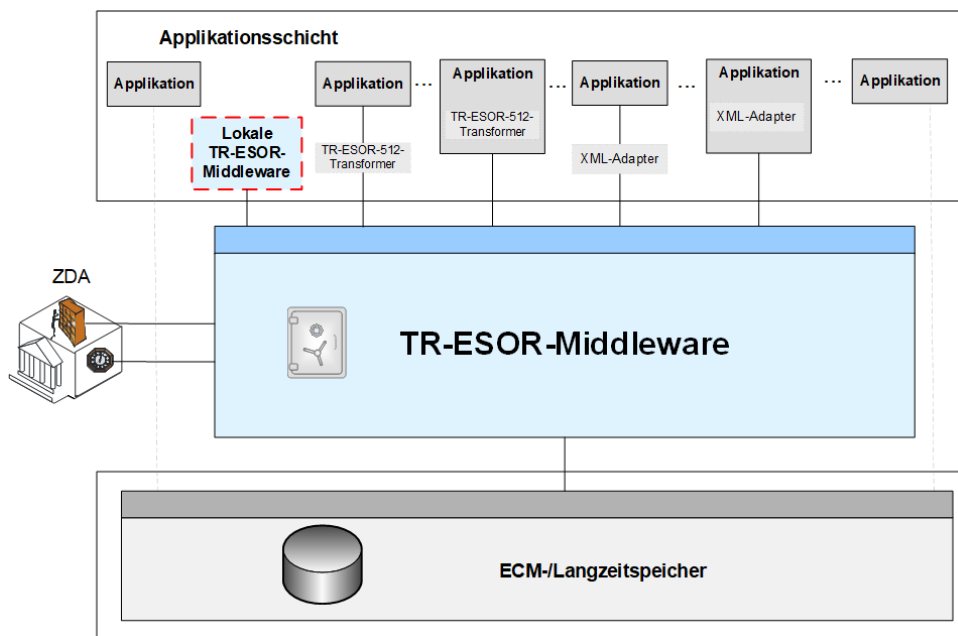
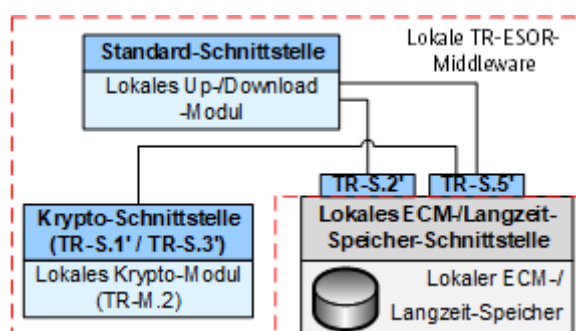


Abbildung 3: Basisarchitektur **[TR-ESOR-ENC]**

Zusätzlich zu der zentral gelegenen Komponente TR-ESOR-Middleware kommt **eine lokale TR-ESOR-Middleware Komponente** hinzu. Die beiden Komponenten kommunizieren miteinander. Der grobe Aufbau der lokalen TR-ESOR-Middleware ist in Abbildung 4 dargestellt. Diese Komponente besteht aus drei Modulen:

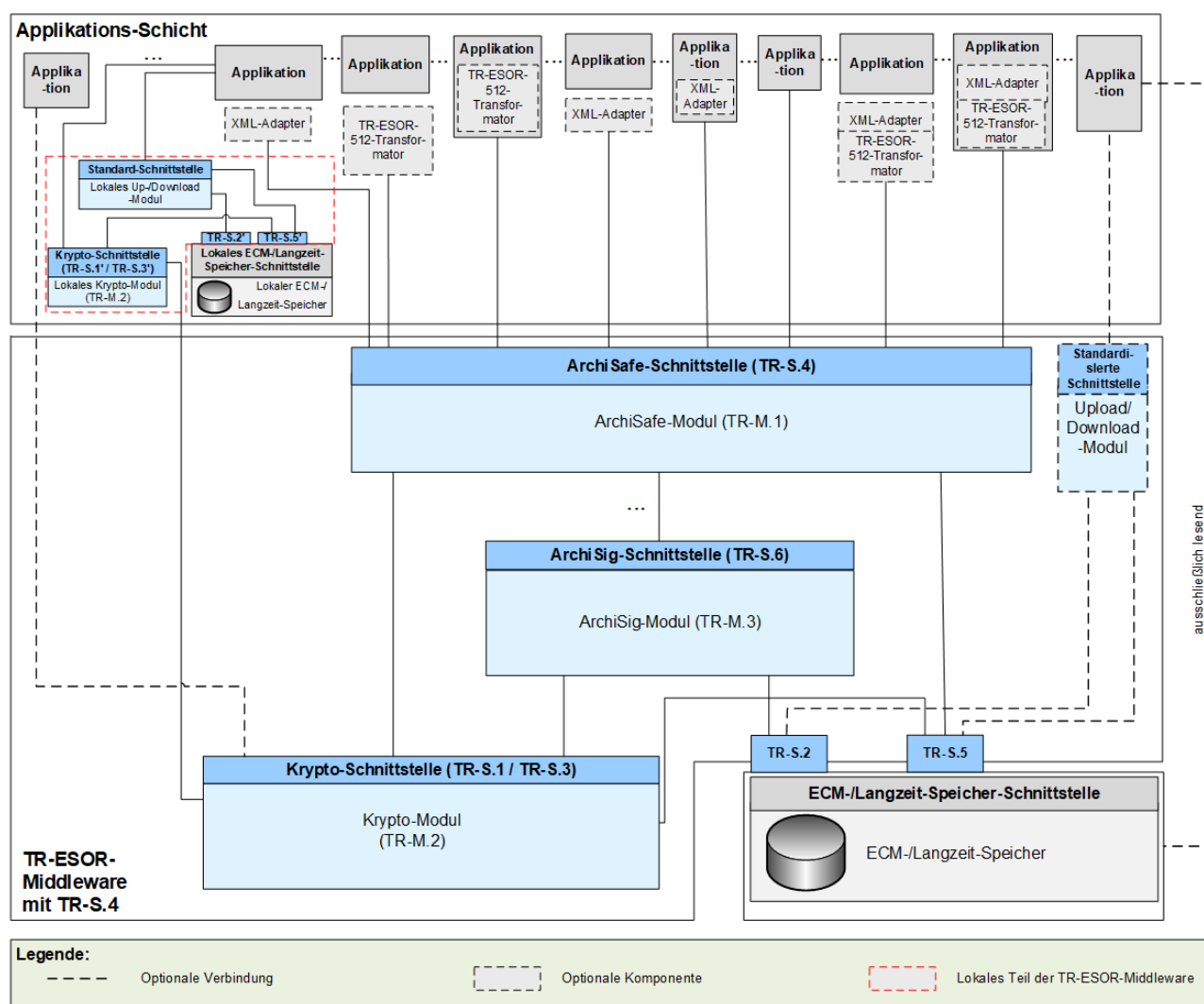
- **Lokales Krypto-Modul** – gem. **[TR-ESOR-M.2]** – stellt eine lokale Instanz von **[TR-ESOR-M.2]**-konformer Implementierung dar, die lokal die Berechnung der Hashwerte und Prüfung der elektronischen Signaturen/Zeitstempel und Siegel (auch lokal oder via einem dedizierten Prüfservice) übernimmt. Dieses Modul bietet entsprechende Schnittstellen **TR-S.1'** und **TR-S.3'**, die das gleiche Verhalten wie die Schnittstellen **TR-S.1** und **TR-S.3** in der zentralen TR-ESOR-Middleware aufweisen (vgl. hierzu **[TR-ESOR-E]**, Kap. 5.1 und 5.3).
- **Lokales Up-/Download-Modul** – übernimmt die lokale Ablage der Daten und Dokumente im Klartext.
- **Lokaler ECM-/Langzeitspeicher** – bietet die beiden Schnittstellen **TR-S.2'** und **TR-S.5'**, welche das Verhalten der entsprechenden Schnittstellen **TR-S.2** und **TR-S.5** implementieren (vgl. hierzu **[TR-ESOR-E]**, Kap. 5.2 und 5.4).


 Abbildung 4: Grobe Architektur der lokalen TR-ESOR-Middleware⁸

Um die Anforderung **A.1** zu erfüllen, benötigt das lokale Krypto-Modul zusätzlich die Fähigkeiten zum Ver- und Entschlüsseln der aufzubewahrenden Information.

- A.8** Das lokale Krypto-Modul muss die Funktionen zur Verschlüsselung und Entschlüsselung und somit zur Steuerung des Zugriffs auf die Information beinhalten. Die Funktionen müssen gem. der Vorgaben aus dem Kap. 3.5 umgesetzt werden

In der Abbildung 5 wird eine von der Abbildung 1 abgeleitete Architektur für **[TR-ESOR-ENC]** (inkl. die lokale Anteile dargestellt in der Abbildung 4) mit der Eingangsschnittstelle **TR-S.4** dargestellt.


 Abbildung 5: Schematische Darstellung der IT-Referenzarchitektur mit **TR-S.4** gem. **[TR-ESOR-ENC]**.

⁸ Eine angepasste Architektur für den Anwendungsfall „Umgang mit VS-NfD-eingestufte Information“ ist dem Anhang im Kapitel 5 zu entnehmen.

In der Abbildung 6 ist eine von der Abbildung 2 abgeleitete Architektur für [TR-ESOR-ENC] (inkl. der lokalen Anteile, dargestellt in der Abbildung 4) mit der Eingangsschnittstelle TR-S.512 dargestellt.

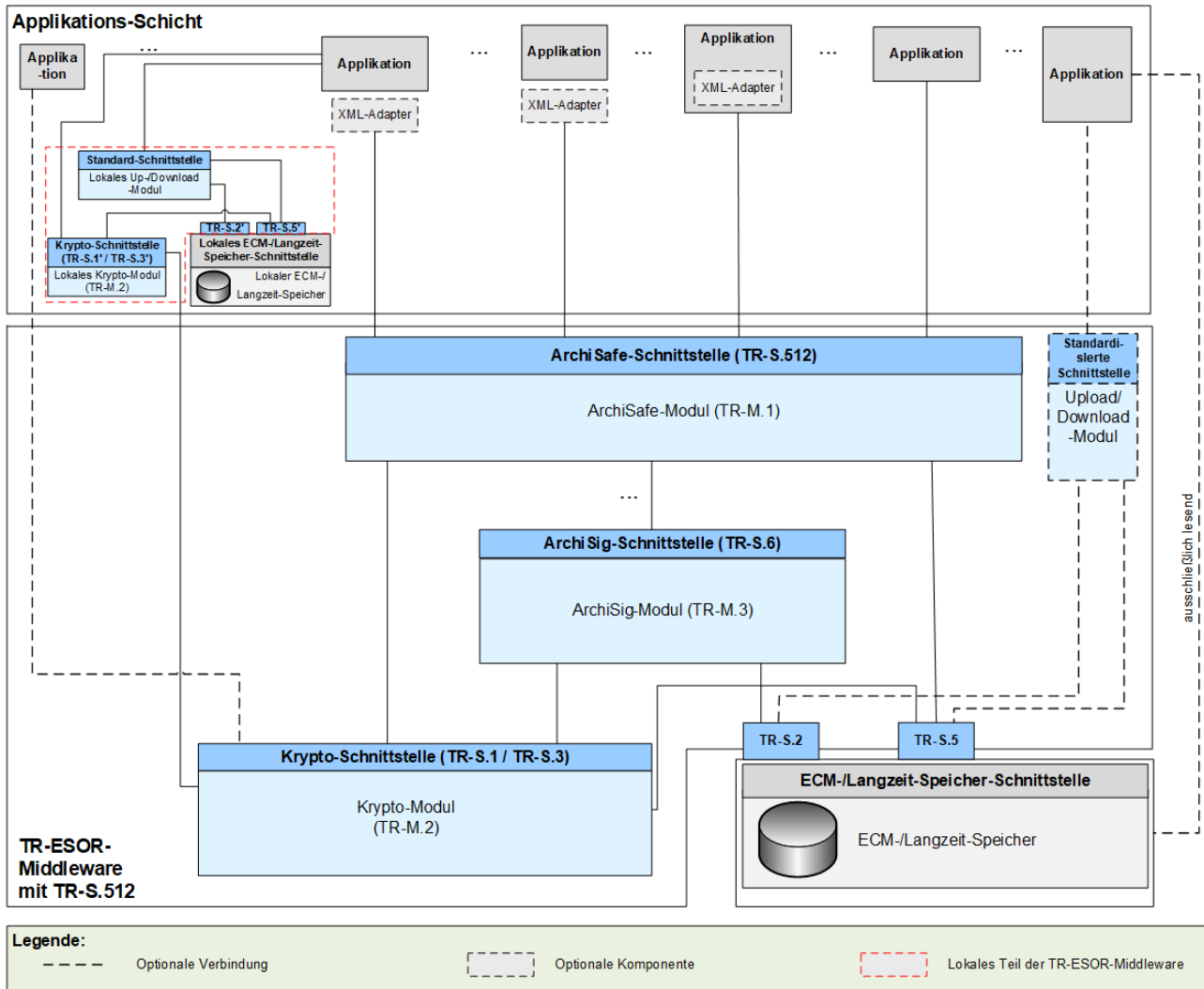


Abbildung 6: Schematische Darstellung der IT-Referenzarchitektur mit TR-S.512 gem. [TR-ESOR-ENC].

Die Kommunikation zwischen dem Krypto-Modul und dem lokalen Krypto-Modul muss folgende Anforderungen erfüllen:

- A.9** Die Authentizität der beiden Kommunikationsenden muss hinreichend geprüft werden. Es muss die beiderseitige Authentifizierung der Kommunikation-Partner erfolgen⁹.
- A.10** Die Vertraulichkeit der Kommunikation muss hinreichend zugesichert werden.

3.3 Zugriff auf die verschlüsselten Dokumente

Grundsätzlich kann eine beliebige Anzahl von verschlüsselten Dokumenten im Archivinformationspaket (AIP) abgelegt werden. Auch die Zuordnung der für Zugriff berechtigten Rollen kann je nach verschlüsseltem Dokument unterschiedlich sein. In der Abbildung 7 ist eine beispielhafte Darstellung einer Zuordnung zwischen den verschlüsselten Dokumenten aus einem Datenobjekte-AIP (D-AIP)¹⁰ und den zugriffsberechtigten Rollen, die in korrespondierenden Zugriffssteuerung-AIPs (Z-AIP)¹⁰ abgelegt werden. Es müssen dabei folgende Anforderungen erfüllt werden:

⁹ Vgl. hierzu auch [TR-ESOR], Kap. 7.2, A7.2-2.

¹⁰ Es handelt sich dabei um ein (L)XAIP, das einen bestimmten in diesem Kapitel definierter Aufbau aufweist.

- A.11** Jede zugriffsberechtigte Rolle muss mindestens über ein asymmetrisches Schlüsselpaar (Kpub und Kpriv) alleine verfügen. Unter der Verwendung dieses Schlüsselpaares wird die Zugriffssteuerung umgesetzt.
- A.12** Jedem einzelnen verschlüsselten Dokument im Datenobjekt-AIP muss genau ein Zugriffssteuerung-AIP zugewiesen werden.
- A.13** Ein Zugriffssteuerung-AIP kann mehreren verschlüsselten Dokumenten eines Datenobjekt-AIP zugewiesen werden

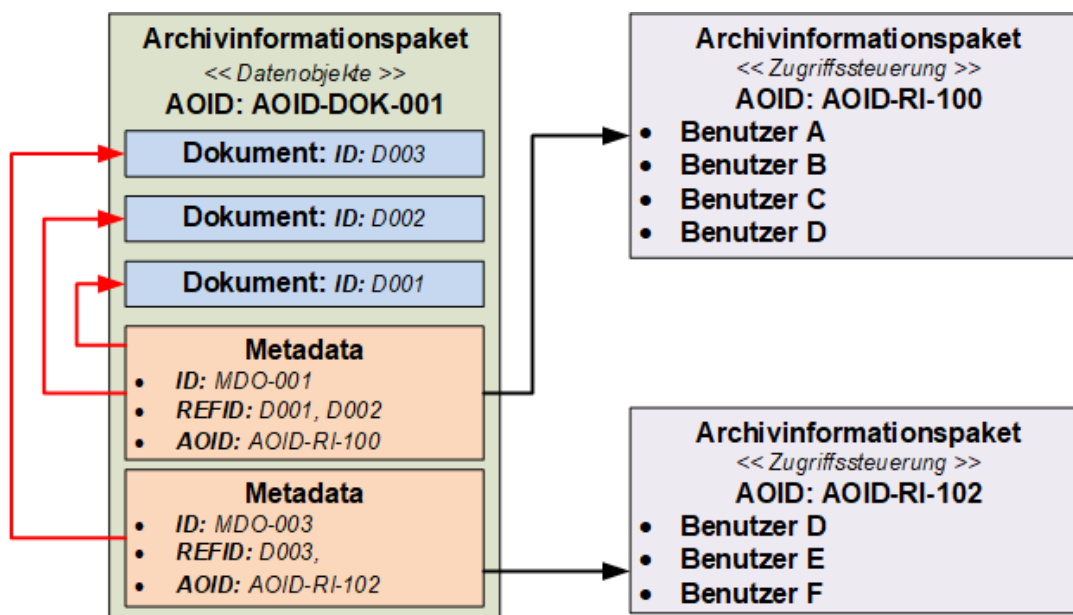


Abbildung 7: Steuerung des Zugriffs auf einzelnen verschlüsselten Dokumenten – Beispiel.

Die Verknüpfung zwischen einzelnen verschlüsselten Dokumenten und den korrespondierenden Zugriffssteuerung-AIPs werden mit Hilfe von innerhalb Metadaten abgelegten Datenstrukturen hergestellt. So ordnet beispielsweise (vgl. Abbildung 7) das Metadatenobjekt mit der ID MDO-001 den beiden verschlüsselten Dokumenten mit IDs D001 und D002 das Z-AIP mit der AOID AOID-RI-100 zu. Auf diese Weise wird ausgedrückt, dass die innerhalb von diesem Z-AIP abgelegten Referenzen auf Rollen Benutzer A, Benutzer B, Benutzer C und Benutzer D die verschlüsselte Dokumente D001 und D002 entschlüsseln können. Das verschlüsselte Dokument D003 kann dagegen durch die Rollen Benutzer D, Benutzer E und Benutzer F aus dem Z-AIP mit der AOID AOID-RI-102 entschlüsselt werden. Somit kann die Rolle Benutzer D alle drei Dokumente aus dem D-AIP mit der AOID AOID-DOK-001 entschlüsseln und darf auf die Information zugreifen.

3.4 Angepasste Prozesse gem. [TR-ESOR-ENC]

Folgende [TR-ESOR]-Prozesse müssen gem. [TR-ESOR-ENC] angepasst werden:

- Ablage elektronischer Unterlagen (vgl. Kap. 3.4.1),
- Ändern der bewahrten Unterlagen (vgl. Kap. 3.4.2),
- Prüfen der beweisrelevanten Daten und technischen Beweisdaten (vgl. Kap. 3.4.5),
- Erneuerung des Hashbaums (vgl. Kap. 3.4.7).

Die nachfolgende Tabelle 1 erklärt die in nachfolgenden Ablaufdiagrammen verwendeten Artefakte und korrespondierende Namensgebung.

Artefakt	Erklärung
D	Ein beliebiges Dokument im Klartext (nicht verschlüsselt).
Denc	Ein mit einem symmetrischen Schlüssel verschlüsseltes Klartextdokument D.

Artefakt	Erklärung
Kenc	Ein für die Dokumentenverschlüsselung verwendeter symmetrischer Verschlüsselungsschlüssel.
Kpub	Der öffentliche Schlüssel des für die Zugriffssteuerung verwendeten asymmetrischen Schlüsselpaars (z.B. Kpub1 und Kpriv1).
Kpriv	Der private Schlüssel des für die Zugriffssteuerung verwendeten asymmetrischen Schlüsselpaars (z.B. Kpub1 und Kpriv1).
RecipientInfo	Eine Datenstruktur in der eine Zugriffsrolle abgebildet worden ist (Ablage des mit dem Kpub verschlüsselten Kenc), z.B. RecipientInfo1.
RecipientInfos	Eine Sammlung von RecipientInfo-Strukturen, z.B. RecipientInfo1 und RecipientInfo2.
ReXAIP	Ein XAIP (Z-AIP), das eine RecipientInfo-Struktur beinhaltet und somit Zugriff auf korrespondierenden verschlüsselten Dokumente (Denc) steuert.
ReAOID	Eine eindeutige AOID zu einem ReXAIP.
ReXAIP+	Ein ReXAIP mit dem eingetragenen ReAOID (Erweiterung im Zuge der Ablage).
K(ReXAIP+)	Eine kanonische Version von ReXAIP+.
DLRef	Eine lokale Referenz auf das lokal abgelegte Klartextdokument D.
LXAIP	Ein LXAIP (D-AIP), das die zu schützende Dokumente (u.a. Denc) beinhaltet.
VR(D)	Ein Verifikationsprotokoll zu den elektronischen Signaturen, enthalten im Klartextdokument D.
LXAIP+	Ein um das VR(D) angereichertes LXAIP (Erweiterung im Zuge der Ablage).
LXAIP++	Ein um die zugehörige AOID angereichertes LXAIP+ (Erweiterung im Zuge der Ablage).
H(Denc)	Hashwert über ein Dokument, hier über das verschlüsselte Klartextdokument Denc, berechnet mit Hashalgorithmus H.
D'	Eine neue Version des Klartextdokuments D.
Kenc'	Ein neu generierter symmetrischer Schlüssel für die Verschlüsselung von D'.
Denc'	Das symmetrisch verschlüsselte (Kenc') Klartextdokument D'.
RecipientInfo'	Eine RecipientInfo-Struktur, die einen einzelnen Zugriff auf Denc' steuert, z.B. RecipientInfo1'.
RecipientInfos'	Eine Sammlung von neu erzeugten RecipientInfo-Strukturen, z.B. RecipientInfo1' und RecipientInfo2'.
ReXAIP'	ReXAIP mit der RecipientInfos'-Struktur.
ReAOID'	Eine AOID vom ReXAIP'.
ReXAIP+'	Das ReXAIP' mit eingetragenen ReAOID'.
DLRef'	Eine lokale Referenz auf das lokal abgelegte Klartextdokument D'.
DLXAIP'	Eine Delta-LXAIP (D-AIP), die das Aktualisieren von LXAIP++ beschreibt.
VR(D')	Ein Verifikationsprotokoll zu den elektronischen Signaturen, enthalten im Klartextdokument D'.
LXAIP++'	Ein um die Daten aus dem DLXAIP' angereichertes LXAIP++ (Erweiterung im Zuge der Aktualisierung: Versionen V001 und V002 sind enthalten).
V002	Die zweite Version im LXAIP++'
ER-V1-ReXAIP++	Evidence Record für die erste Version (V001) von ReXAIP++.
ER-V1-LXAIP++	Evidence Record für die erste Version (V001) von LXAIP++.
H'(D)	Hashwert über das Klartextdokument D berechnet mit einem neuen Hashalgorithmus H'.

Artefakt	Erklärung
Kenc* oder Kenc'*	Ein im Zuge der Neuverschlüsselung erzeugter symmetrischer Schlüssel für das Dokument D oder D'.
Denc* oder Denc'*	Ein mit dem Kenc* oder Kenc'* neuverschlüsseltes Dokument D oder D'.
RecipientInfo* bzw. RecipientInfo'*	RecipientInfo bzw. RecipientInfo' nach der erfolgten Neuverschlüsselung der geschützten Dokumente, z. B. D.
RecipientInfos* bzw. RecipientInfos'*	RecipientInfos bzw. RecipientInfos' nach der erfolgten Neuverschlüsselung der geschützten Dokumente, z. B. D.
ReXAIP* bzw. ReXAIP+*'	ReXAIP bzw. ReXAIP+' nach der erfolgten Neuverschlüsselung der geschützten Dokumente, z. B. D.
LXAIP* bzw. LXAIP++*'	LXAIP bzw. LXAIP++' nach der erfolgten Neuverschlüsselung der geschützten Dokumente, z. B. D.

Tabelle 1: Erklärung zu Artefakten in Ablaufdiagrammen.

3.4.1 Ablage elektronischer Unterlagen

In der Abbildung 8 ist in einem Sequenzdiagramm ein Ablauf der Ablage eines Klartextdokuments $\mathcal{D}$ gem. [TR-ESOR-ENC] dargestellt (vgl. auch [TR-ESOR], Kap. 7.5.1). Dabei ist zu beachten, dass im Falle der Ablage von mehreren solchen Klartextdokumenten die entsprechenden Schritte für jedes Dokument wiederholt werden müssen.

Hinweis zur der nachfolgenden Abbildung 8:

Es wird davon ausgegangen, dass die Applikation die entsprechende Zuteilung der Zugriffsberechtigung auf die verschlüsselten Daten steuert.

Die Ablage wird in drei Einzelphasen verteilt:

- Lokale Initialisierung – (Schritte 1 bis 13) die in der lokalen Middleware stattzufindenden Vorbereitungen,
- Ablage der Zugriffsrechte – (Schritte 14 bis 27) die zentrale Ablage der Zugriffsrechte,
- Ablage der Daten – (Schritte 28 bis 63) die zentrale Ablage der zu schützenden Daten.

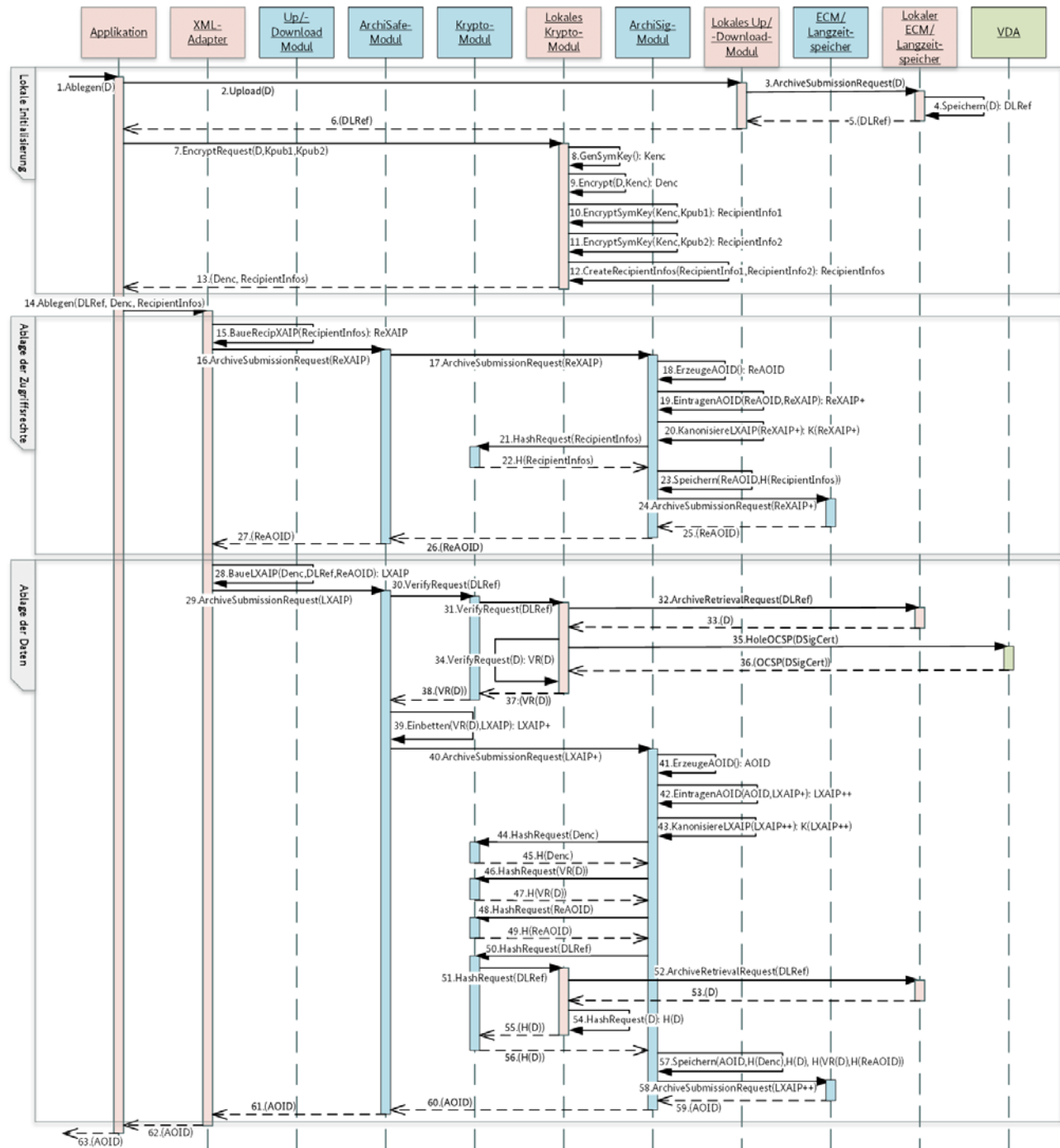


Abbildung 8: Ablage gem. [TR-ESOR-ENC].

Die nachfolgende Tabelle 2 beschreibt die einzelnen Schritte, dargestellt in der Abbildung 8.

Nr.	Aufruf	Beschreibung
1.	Ablegen(D)	Der Nutzer initiiert die Ablage eines neuen signierten Dokuments. Es wird das Dokument D übergeben.
2.	Upload(D)	Die Applikation initiiert einen Upload vom Klartextdokument D an der Schnittstelle zum lokalen Up-/Download-Modul.

Nr.	Aufruf	Beschreibung
3.	ArchiveSubmissionRequest(D)	Das lokale Up-/Download-Modul ruft die ArchiveSubmission -Funktion an der TR-S.2 -Schnittstelle vom lokalen ECM/Langzeitspeicher auf und übergibt das Klartextdokument D .
4.	Speichern(D) → DLRef	Der lokale ECM/Langzeitspeicher erzeugt eine Referenz DLRef für das lokal gespeicherte Klartextdokument D und legt dieses unter DLRef Referenz ab.
5.	Return(DLRef)	Die erzeugte Referenz (DLRef) auf das lokal abgelegte Klartextdokument D wurde an das lokale Up-/Download-Modul zurückgegeben.
6.	Return(DLRef)	Das lokale Up-/Download-Modul gibt die Referenz auf das lokal abgelegt Klartextdokument D an die Applikation zurück.
7.	EncryptRequest(D, Kpub1, Kpub2)	Die Applikation steuert das lokale Krypto-Modul an, um die hybride Verschlüsselung des Dokuments via die TR-S.3' Schnittstelle zu initiieren. Es wird das Klartextdokument D und zwei öffentliche Schlüssel (X509-Zertifikate) der zwei berechtigten Benutzer Kpub1 und Kpub2 übergeben.
8.	GenSymKey() → Kenc	Das lokale Krypto-Modul leitet einen symmetrischen Schlüssel für die Verschlüsselung ab. Ausgabe Kenc – symmetrische Verschlüsselungsschlüssel.
9.	Encrypt(D, Kenc) → Denc	Das lokale Krypto-Modul verschlüsselt D mit dem generierten Schlüssel Kenc . Ergebnis ist ein verschlüsseltes Dokument Denc .
10.	EncryptSymKey(Kenc, Kpub1) → RecipientInfo1	Das lokale Krypto-Modul verschlüsselt den generierten symmetrischen Verschlüsselungsschlüssel Kenc mit dem asymmetrischen öffentlichen Schlüssel des Nutzers Kpub1 . Das Ergebnis ist eine Struktur RecipientInfo1 .
11.	EncryptSymKey(Kenc, Kpub2) → RecipientInfo2	Das lokale Krypto-Modul verschlüsselt den generierten symmetrischen Verschlüsselungsschlüssel Kenc mit dem asymmetrischen öffentlichen Schlüssel des Nutzers Kpub2 . Das Ergebnis ist eine Struktur RecipientInfo2 .
12.	CreateRecipientInfos(RecipientInfo1, RecipientInfo2) → RecipientInfos	Das lokale Krypto-Modul legt die beiden erzeugten RecipientInfo1 und RecipientInfo2 in eine gemeinsame Struktur RecipientInfos , inkl. weiterer Daten, wie z. B. Schlüsselverschlüsselungsalgorithmus.
13.	Return(Denc, RecipientInfos)	Antwort auf den Schritt 2., das lokale Krypto-Modul liefert das verschlüsselte Dokument D als Denc und die korrespondierende(n) RecipientInfo -Struktur(en) als Zugriffsberechtigung an die Applikation zurück.

Nr.	Aufruf	Beschreibung
14.	Ablegen (DLRef, Denc, RecipientInfos)	Die Applikation initiiert die Ablage der Informationen am XML-Adapter und übergibt die Referenz auf das lokal gespeicherte Klartextdokument <code>D</code> (<code>DLRef</code>), das verschlüsselte Dokument <code>Denc</code> und die <code>RecipientInfo</code> -Struktur.
15.	BaueRecipXAIP (RecipientInfos) → ReXAIP	Der XML-Adapter erzeugt ein XAIP (<code>ReXAIP</code>) und legt darin die erhaltene <code>RecipientInfo</code> -Struktur ab.
16.	ArchiveSubmissionRequest (ReXAIP)	Der XML-Adapter ruft die Funktion <code>ArchiveSubmission</code> -Funktion an der TR-S.4 -Schnittstelle (ArchiSafe-Modul) der zentralen Middleware auf und übergibt das erzeugte <code>ReXAIP</code> .
17.	ArchiveSubmissionRequest (ReXAIP)	Das ArchiSafe-Modul ruft die <code>ArchiveSubmission</code> -Funktion an der TR-S.6 -Schnittstelle auf und übergibt das <code>ReXAIP</code> .
18.	ErzeugeAOID () → ReAOID	Das ArchiSig-Modul erzeugt eine <code>ReAOID</code> für <code>ReXAIP</code> .
19.	EintragenAOID (ReAOID, ReXAIP) → ReXAIP+	Das ArchiSig-Modul trägt die erzeugte <code>ReAOID</code> in das <code>ReXAIP</code> i ein. Es entsteht ein <code>ReXAIP+</code> .
20.	KanonisiereLXAIP (ReXAIP+) → K(ReXAIP+)	Das ArchiSig-Modul kanonisiert das <code>ReXAIP+</code> , es entsteht ein <code>K(ReXAIP+)</code> .
21.	HashRequest (RecipientInfos)	Das ArchiSig-Modul ruft die <code>Hash</code> -Funktion an der TR-S.3 -Schnittstelle (ArchiSig-Modul) auf, um den Hashwert über die aus dem <code>K(ReXAIP+)</code> entnommene <code>RecipientInfos</code> -Struktur zu berechnen.
22.	Return (H(RecipientInfos))	Es wird der Hashwert <code>H(RecipientInfos)</code> zurückgeliefert.
23.	Speichern (ReAOID, H(RecipientInfos))	Das ArchiSig-Modul speichert die <code>ReAOID</code> und <code>H(RecipientInfos)</code> in internen Datenstrukturen und aktualisiert den Hashbaum.
24.	ArchiveSubmissionRequest (ReXAIP+)	Das ArchiSig-Modul ruf die <code>ArchiveSubmission</code> -Funktion an der TR-S.2 -Schnittstelle (ECM/Langzeitspeicher) auf, um die <code>ReXAIP+</code> im Speicher persistent abzulegen.
25.	Return (ReAOID)	Das ECM/Langzeitspeicher-Modul liefert die <code>ReAOID</code> als Bestätigung der erfolgreichen Speicherung des korrespondierenden <code>ReXAIP+</code> an das ArchiSig-Modul zurück.
26.	Return (ReAOID)	Das ArchiSig-Modul liefert die <code>ReAOID</code> als Bestätigung der erfolgreichen Verarbeitung des korrespondierenden <code>ReXAIP+</code> an das ArchiSafe-Modul zurück.
27.	Return (ReAOID)	Das ArchiSafe-Modul liefert die <code>ReAOID</code> an die Applikation zurück, als Bestätigung der erfolgreichen Ablage vom <code>ReXAIP+</code> im die zentrale Middleware.

Nr.	Aufruf	Beschreibung
28.	BaueLXAIP (Denc, DLRef, ReAOID) → LXAIP	Der XML-Adapter baut ein LXAIP zusammen, welches das verschlüsselte Klartextdokument Denc, die Referenz auf das lokal gespeicherte Klartextdokument D (DLRef), und die Referenz auf die zuvor abgelegte XAIP mit den Zugriffsberechtigungen (ReXAIP), nämlich ReAOID beinhaltet.
29.	ArchiveSubmissionRequest (LXAIP)	Der XML-Adapter ruft an der TR-S.4 -Schnittstelle der zentralen Middleware (das ArchiSafe-Modul) die ArchiveSubmission-Funktion auf und übergibt das erzeugte LXAIP.
30.	VerifyRequest (DLRef)	Das ArchiSafe-Modul ermittelt, dass es sich u.a. um eine lokal auflösbare Referenz (DLRef) handelt und ruft die Verify-Funktion an der TR-S.1 -Schnittstelle des zentralen Krypto Moduls auf und übergibt DLRef.
31.	VerifyRequest (DLRef)	Das zentrale Krypto-Modul ruft die Verify-Funktion an der TR-S.1' -Schnittstelle des lokalen Krypto-Moduls auf und übergibt die Referenz auf das lokal gespeicherte Klartextdokument (DLRef).
32.	ArchiveRetrievalRequest (DLRef)	Das lokale Krypto-Modul ruft die ArchiveRetrieval-Funktion an der TR-S.5' -Schnittstelle des lokalen ECM/Langzeitspeicher-Moduls auf und übergibt die lokale Referenz auf das Klartextdokument, DLRef.
33.	Return (D)	Das lokale ECM/Langzeitspeicher-Modul ermittelt das zu der lokalen Referenz DLRef zugehörige Klartextdokument D und liefert dieses an das lokale Krypto-Modul zurück.
34.	VerifyRequest (D) → VR(D)	Das lokale Krypto-Modul verifiziert die Signaturen/Siegl/Zeitstempel, die auf dem Klartextdokument D eingebracht worden sind und liefert das Verifikationsprotokoll VR(D).
35.	HoleOCSP (DSigCert)	Das lokale Krypto-Modul ermittelt während der Verifikation der kryptographischen Artefakte, die an das Dokument D angebracht worden sind, die Sperrstatus der relevanten Zertifikate, indem die korrespondierende Schnittstelle (OCSP) des VDA angesprochen wird.
36.	Return (OCSP(DSigCert))	Der VDA liefert entsprechend Status zu angefragten Zertifikaten zurück (OCSP(DSigCert) etc.).
37.	Return (VR(D))	Das lokale Krypto-Modul liefert an das zentrale Krypto-Modul das Verifikationsprotokoll über die Validierung des Klartextdokument inkl. der darin enthaltenen kryptographischen Artefakte zurück.
38.	Return (VR(D))	Das zentrale Krypto-Modul liefert das Verifikationsprotokoll VR(D) an das zentrale ArchiSafe-Modul zurück.

Nr.	Aufruf	Beschreibung
39.	Einbetten (VR(D),LXAIP) → LXAIP+	Das zentrale ArchiSafe-Modul bettet das erhaltene Verifikationsprotokoll in das vorhandenen LXAIP ein. Das Ergebnis ist LXAIP+.
40.	ArchiveSubmissionRequest (LXAIP+)	Das zentrale ArchiSafe-Modul ruft die ArchiveSubmission-Funktion an der TR-S.6 -Schnittstelle des zentralen ArchiSig-Moduls auf und übergibt das LXAIP+.
41.	ErzeugeAOID () → AOID	Das zentrale ArchiSig-Modul erzeugt eine neue AOID.
42.	EintragenAOID (AOID,LXAIP) → LXAIP++	Das zentrale ArchiSig-Modul trägt die neu generierte AOID in das LXAIP in das VersionManifest-Element ein. Es entsteht eine LXAIP++.
43.	KanonisiereLXAIP (LXAIP++) → K(LXAIP++)	Das zentrale ArchiSig-Modul kanonisiert das LXAIP++ und es entsteht ein K(LXAIP++).
44.	HashRequest (Denc)	Das zentrale ArchiSig-Modul ruft die Hash-Funktion an der TR-S.3 -Schnittstelle des zentralen Krypto-Moduls auf und übergibt das verschlüsselte Dokument Denc.
45.	Return (H(Denc))	Das zentrale Krypto-Modul liefert den Hashwert H(Denc) über das verschlüsselte Element Denc zurück.
46.	HashRequest (VR(D))	Das zentrale ArchiSig-Modul ruft die Hash-Funktion an der TR-S.3 -Schnittstelle des zentralen Krypto-Moduls auf und übergibt das Verifikationsprotokoll über das Klardokument D.
47.	Return (H(VR(D)))	Das zentrale Krypto-Modul liefert den Hashwert über das Verifikationsprotokoll vom Klartextdokument D, H(VR(D)), zurück.
48.	HashRequest (ReAOID)	Das zentrale ArchiSig-Modul ruft die Hash-Funktion an der TR-S.3 -Schnittstelle des Krypto-Moduls auf und übergibt die ReAOID, vom zuvor abgelegten ReXAIP, das die Zugriffsberechtigung auf die verschlüsselten Inhalte hält.
49.	Return (H(ReAOID))	Das zentrale Krypto-Modul liefert den Hashwert H(ReAOID) über die ReAOID, vom zuvor abgelegten ReXAIP, welches die Zugriffsberechtigung auf die verschlüsselten Inhalte hält.
50.	HashRequest (DLRef)	Das zentrale ArchiSig-Modul ruft die Hash-Funktion an der TR-S.3 -Schnittstelle des zentralen Krypto-Moduls auf und übergibt die Referenz DLRef auf das zuvor lokal abgelegte Klartextdokument D.
51.	HashRequest (DLRef)	Das zentrale Krypto-Modul ruft die Hash-Funktion an der TR-S.3' Schnittstelle des lokalen Krypto-Moduls auf und übergibt die Referenz DLRef auf das zuvor lokal abgelegte Klartextdokument D.

Nr.	Aufruf	Beschreibung
52.	ArchiveRetrievalRequest (DLRef)	Das lokale Krypto-Modul ruft die <code>ArchiveRetrieval</code> -Funktion an der TR-S.5' -Schnittstelle des lokalen ECM/Langzeitspeichers auf und übergibt die Referenz <code>DLRef</code> auf das zuvor lokal abgelegte Klartextdokument <code>D</code> .
53.	Return (D)	Der lokale ECM/Langzeitspeicher liefert das Klartextdokument <code>D</code> an das lokale Krypto-Modul zurück.
54.	HashRequest (D) → H(D)	Das lokale Krypto-Modul ruft die eigene <code>Hash</code> -Funktion auf und berechnet den Hashwert <code>H(D)</code> über das Klartextdokument <code>D</code> .
55.	Return (H(D))	Das lokale Krypto-Modul liefert den Hashwert <code>H(D)</code> an das zentrale Krypto-Modul der zentralen Middleware zurück.
56.	Return (H(D))	Das zentrale Krypto-Modul liefert den Hashwert <code>H(D)</code> an das zentrale ArchiSig-Modul zurück.
57.	Speichern (AOID, H(Denc), H(D), H(VR(D), H(ReAOID)))	Das zentrale ArchiSig-Modul speichert die <code>AOID</code> , <code>H(Denc)</code> , <code>H(D)</code> , <code>H(VR(D))</code> , <code>H(ReAOID)</code> in internen Datenstrukturen und aktualisiert den Hashbaum.
58.	ArchiveSubmissionRequest (LXAIP++)	Das zentrale ArchiSig-Modul ruft die <code>ArchiveSubmission</code> -Funktion an der TR-S.2 -Schnittstelle des zentralen ECM/Langzeitspeichers auf, um die <code>LXAIP++</code> im Speicher persistent abzulegen.
59.	Return (AOID)	Der zentrale ECM/Langzeitspeicher liefert die <code>AOID</code> als Bestätigung der erfolgreichen Speicherung des korrespondierenden <code>LXAIP++</code> an das zentrale ArchiSig-Modul zurück.
60.	Return (AOID)	Das zentrale ArchiSig-Modul liefert die <code>AOID</code> als Bestätigung der erfolgreichen Verarbeitung des korrespondierenden <code>LXAIP++</code> an das zentrale ArchiSafe-Modul zurück.
61.	Return (AOID)	Das zentrale ArchiSafe-Modul liefert die <code>AOID</code> an den XML-Adapter zurück als Bestätigung der erfolgreichen Ablage vom <code>LXAIP++</code> im die zentrale Middleware.
62.	Return (AOID)	Der XML-Adapter liefert die <code>AOID</code> an die Applikation zurück.
63.	Return (AOID)	Die Applikation liefert die <code>AOID</code> an den Nutzer zurück.

Tabelle 2: Ablage elektronischer Unterlagen – Beschreibung zur Abbildung 8.

Hinweis!

Sollten weitere Datenobjekte schutzbedürftige Information beinhalten, so müssen diese auch (analog zum Klartextdokument `D`) verschlüsselt abgelegt werden. Das könnte z. B. auf das im Schritt 37 ermittelte Verifikationsprotokoll `VR(D)` zutreffen, das in einem solchen Fall vor der Rückgabe an das zentrale Krypto-Modul verschlüsselt werden müsste. Die nachgelagerte Hashwertberechnung über `VR(D)` müsste analog zum Muster im Schritten 50 bis 56 erfolgen.

3.4.2 Abfrage der bewahrten Daten

Abhängig von der Art der abzurufenden Daten muss die dazugehörige Vorgehensweise unterschieden werden:

- Die Klartextdokumente werden lokal via die lokale Download-Schnittstelle unter der Verwendung der beim Upload erhaltenen lokalen Referenz z. B. `DLRef` ermittelt.
- Das komplette LXAIP (D-AIP) kann mit Hilfe der dazugehörigen `AOID` via die **TR-S.4**- oder **TR-S.512**-Schnittstelle aus der zentralen Middleware ermittelt werden. Dieses beinhaltet die verschlüsselten Dokumente sowie lokale Referenzen auf die Klartextdokumente und das entsprechende `RecipientInfos`-Element (gespeichert in korrespondierenden `ReXAIP` (Z-AIP)). Hierzu gilt:
 - die Klartextdokumente werden wie im 1. Punkt beschrieben ermittelt,
 - anderenfalls werden die korrespondierenden berechtigten Rollen ermittelt (`RecipientInfos`-Element aus dem zugehörigen `ReXAIP`), das passende `RecipientInfo`-Element gefunden, mit Hilfe des vorhandenen und vorgelegten privaten Schlüsselmaterials (z. B. `Kpriv1` auf einer Smartcard) der aus dem `RecipientInfo`-Element kommende symmetrische Schlüssel entschlüsselt und in einem nachkommenden Schritt die im LXAIP vorliegenden verschlüsselten Daten entschlüsselt (vgl. Abbildung 9).

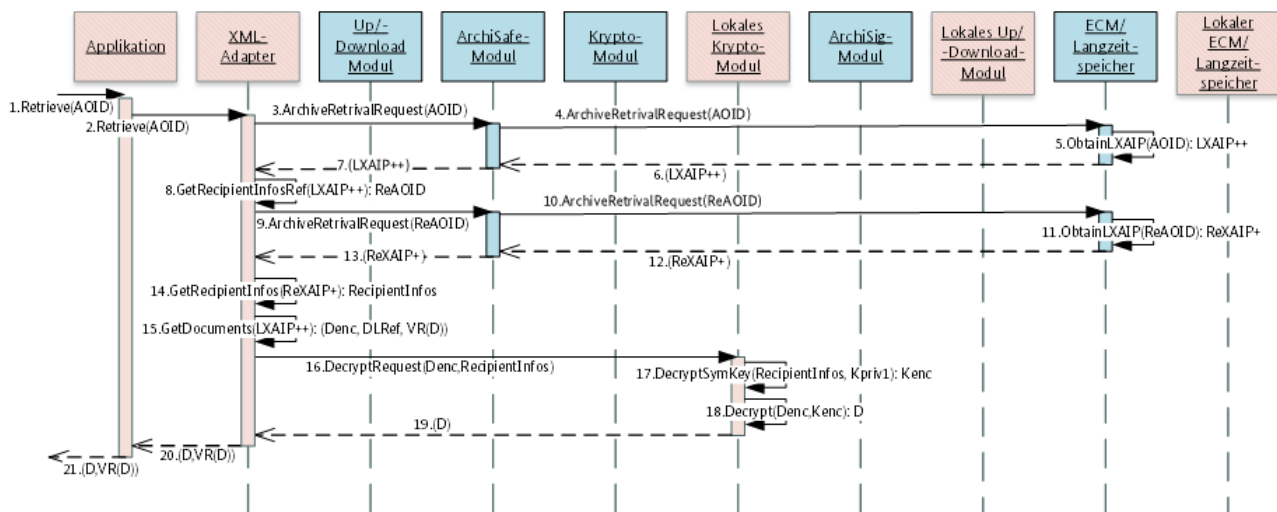


Abbildung 9: Abfrage inkl. Entschlüsselung gem. [TR-ESOR-ENC].

Die nachfolgende Tabelle 3 beschreibt die einzelnen Schritte, dargestellt in der Abbildung 9.

Nr.	Aufruf	Beschreibung
1.	Retrieve(AOID)	Der Nutzer initiiert die Abfrage der Dokumente aus einem LXAIP (D-AIP) mit einem <code>AOID</code> <code>AOID</code> . Zusätzlich könnte auch eine Version des LXAIP übergeben werden.
2.	Retrieve(AOID)	Die Applikation delegiert die Abfrage an den XML-Adapter.
3.	ArchiveRetrievalRequest(AOID)	Der XML-Adapter ruft die <code>ArchiveRetrieval</code> -Funktion an der TR-S.4 -Schnittstelle des zentralen <code>ArchiSafe</code> -Moduls auf und übergibt die <code>AOID</code> , um die korrespondierende LXAIP (D-AIP) abzurufen.

Nr.	Aufruf	Beschreibung
4.	RetrievalRequest (AOID)	Das zentrale ArchiSafe-Modul ruft die <code>ArchiveRetrieval</code> -Funktion an der TR-S.5 -Schnittstelle vom zentralen ECM/Langzeitspeicher auf und übergibt die AOID auf.
5.	ObtainLXAIP (AOID) → LXAIP++	Der zentrale ECM/Langzeitspeicher ermittelt intern das zu der übergebenen AOID AOID zugehörige LXAIP++.
6.	Return (LXAIP++)	Der zentrale ECM/Langzeitspeicher liefert das ermittelte LXAIP++ an das zentrale ArchiSafe-Modul zurück.
7.	Return (LXAIP++)	Das zentrale ArchiSafe-Modul liefert das LXAIP++ an den XML-Adapter zurück.
8.	GetRecipientInfosRef (LXAIP++) → ReAOID	Der XML-Adapter extrahiert die Referenz (AOID) auf das XAIP (Z-AIP) mit den zugehörigen <code>RecipientInfos</code> , nämlich ReAOID.
9.	ArchiveRetrivalRequest (ReAOID)	Der XML-Adapter ruft die <code>ArchiveRetrival</code> -Funktion an der TR-S.4 -Schnittstelle des zentralen ArchiSafe-Moduls auf und übergibt ReAOID, um das XAIP mit dem korrespondierenden <code>RecipientInfos</code> -Element zu ermitteln.
10.	ArchiveRetrivalRequest (ReAOID)	Das zentrale ArchiSafe-Modul ruft die <code>ArchiveRetrieval</code> -Funktion an der TR-S.5 -Schnittstelle vom zentralen ECM/Langzeitspeicher auf und übergibt ReAOID.
11.	ObtainLXAIP (ReAOID) → ReXAIP+	Der zentrale ECM/Langzeitspeicher ermittelt intern das zu der übergebenen ReAOID zugehörige ReXAIP.
12.	Return (ReXAIP+)	Der zentrale ECM/Langzeitspeicher liefert das ermittelte ReXAIP an das zentrale ArchiSafe-Modul zurück.
13.	Return (ReXAIP+)	Das zentrale ArchiSafe-Modul liefert das ReXAIP an den XML-Adapter zurück.
14.	GetRecipientInfos (ReXAIP+) → RecipientInfos	Der XML-Adapter extrahiert die <code>RecipientInfos</code> aus dem erhaltenen ReXAIP.
15.	GetDocuments (LXAIP++) → (Denc, DLRef, VR(D))	Der XML-Adapter extrahiert die relevanten Daten aus dem zuvor erhaltenen LXAIP++: • Denc – verschlüsseltes Klartextdokument, • DLRef – lokale Referenz auf das Klartextdokument D, • VR(D) – Prüfbericht zu im Klartextdokument D enthaltenen Signaturen.
16.	DecryptRequest (Denc, RecipientInfos)	Der XML-Adapter ruft die <code>Decrypt</code> -Funktion an der TR-S.3' -Schnittstelle des lokalen Krypto-Moduls auf und übergibt das verschlüsselte Dokument Denc sowie die ermittelten <code>RecipientInfos</code> , um das Klartextdokument D zu erhalten.

Nr.	Aufruf	Beschreibung
17.	DecryptSymKey (RecipientInfos, Kpriv1) → Kenc	Das lokale Krypto-Modul identifiziert die passende RecipientInfo-Struktur und ermittelt den zugehörigen, benötigten privaten Schlüssel Kpriv1 ¹¹ . Mit Hilfe von Kpriv1 wird der in der RecipientInfo-Struktur mitenthaltene symmetrische Schlüssel entschlüsselt → Kenc.
18.	Decrypt (Denc, Kenc) → D	Das lokale Krypto-Modul entschlüsselt mit dem im Schritt 17 ermittelten symmetrischen Schlüssel Kenc, das verschlüsselte Dokument Denc und erhält somit das Klartextdokument D.
19.	Return (D)	Das lokale Krypto-Modul liefert das entschlüsselte Klartextdokument D an den XML-Adapter zurück.
20.	Return (D, VR(D))	Das lokale Krypto-Modul liefert das Klartextdokument D und (optional) den zugehörigen Signaturprüfbericht VR (D) an die Applikation zurück.
21.	Return (D, VR(D))	Der XML-Adapter liefert das Klartextdokument D und (optional) den zugehörigen Signaturprüfbericht VR (D) an den Nutzer zurück.

Tabelle 3: Abfrage inkl. Entschlüsselung – Beschreibung zur Abbildung 9.

3.4.3 Ändern der bewahrten Daten

Mit Hilfe der ArchiveUpdate-Funktion wird eine neue Version der bereits abgelegten Objekte erzeugt. Hierbei werden die bereits abgelegten Daten nicht verändert, sondern es werden lediglich die veränderten Daten in einer neuen Version hinzugefügt.

Abhängig davon, welche Daten geändert werden, müssen unterschiedliche Abläufe angewandt werden:

- Die verschlüsselten Daten, z. B. das verschlüsselte Dokument Denc. In dem Fall muss der in der Abbildung 10 definierte Ablauf benutzt werden. Insbesondere muss folgende Anforderung berücksichtigt werden:
- A.14** Es muss die Auswirkung der Änderung-Operation auf die Einhaltung der Regeln der Zugriffssteuerung auf die verschlüsselten Anteile bewertet werden. Die ArchiveUpdate¹²- bzw. UpdatePOC¹³-Funktion gem. [TR-ESOR] kann die bereits vorhandenen Daten eines LXAIP nicht ändern und auch im Falle einer Nichtberücksichtigung in der neuen Version bleibt das logisch gelöschte Dokument in der vorherigen Version des LXAIP bestehen und kann grundsätzlich abgerufen werden. Das Risiko eines nicht (mehr) befugten Zugriffs muss daher bewertet werden und es müssen entsprechende Maßnahmen implementiert werden. Es wird die Durchführung einer Risikoanalyse empfohlen.
- Sonstige Daten – hierzu muss der bereits in [TR-ESOR], Kap. 7.5.2 definierte Ablauf angewandt werden.

¹¹ Der korrespondierende private Schlüssel, kann beispielweise auf einer Smartcard liegen und anhand der in der RecipientInfo-Struktur abgelegten Daten identifiziert werden. Die Nutzung vom Kpriv1 muss entsprechend z. B. durch die Eingabe der PIN durch den Inhaber und dessen erfolgten Authentifizierung im Nachgang autorisiert werden.

¹² Siehe TR-S.4-Schnittstelle gemäß [TR-ESOR-E], Kap. 3.2

¹³ Siehe TR-S.512-Schnittstelle gemäß [TR-ESOR-E], Kap. 4.1

Hinweis zur der nachfolgenden Abbildung 10:

Es wird davon ausgegangen, dass die führende Applikation die entsprechende Zuteilung der Zugriffsberechtigung auf die verschlüsselten Daten steuert.

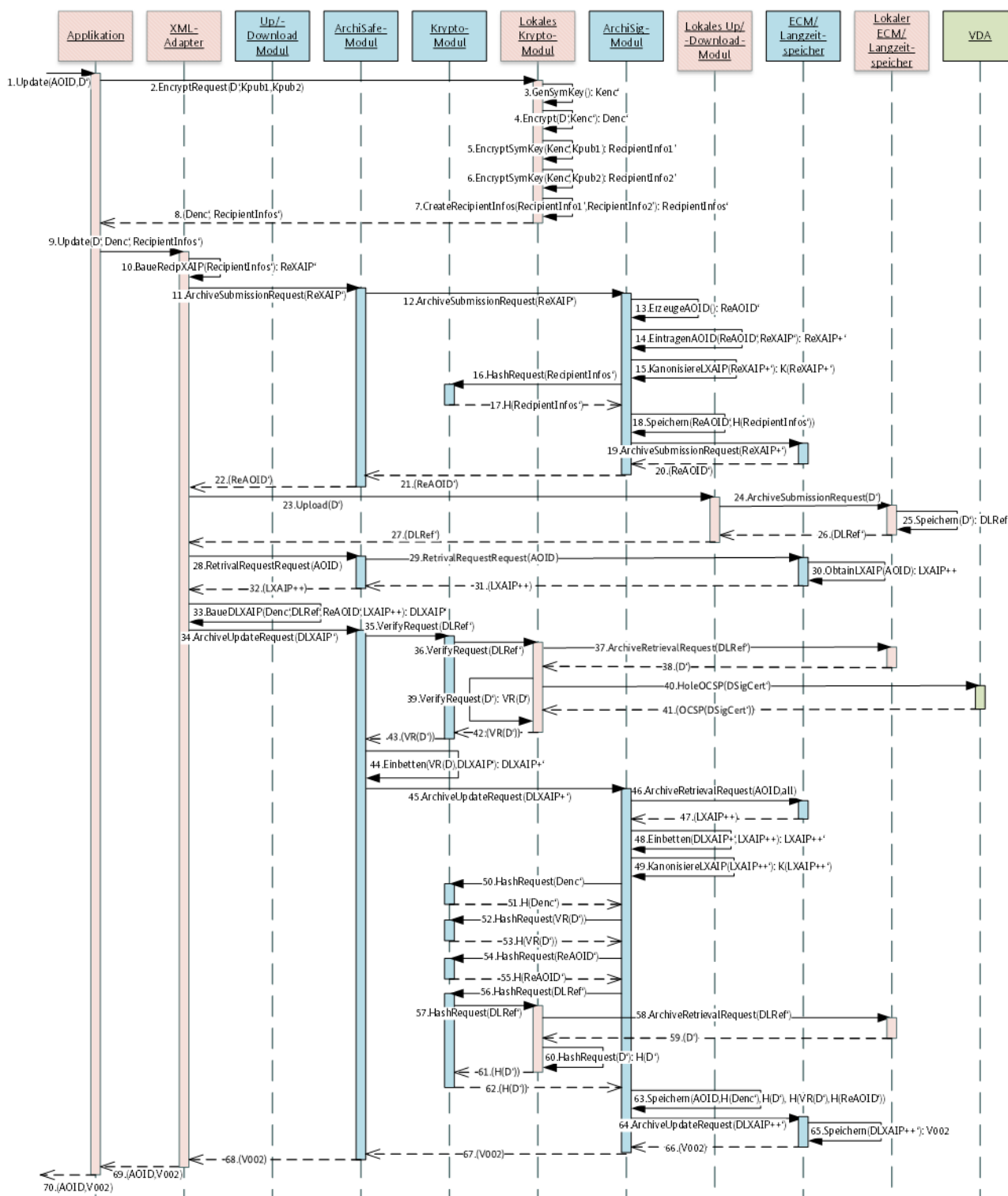


Abbildung 10: Änderung gem. [TR-ESOR-ENC].

Die nachfolgende Tabelle 4 beschreibt die einzelnen Schritte, dargestellt in der Abbildung 10.

Nr.	Aufruf	Beschreibung
1.	Update (AOID,D')	Der Nutzer initiiert die Aktualisierung des Klartextdokuments D mit einem Klartextdokument D' in der Applikation.
2.	EncryptRequest (D',Kpub1,Kpub2)	Die Applikation steuert das lokale Krypto-Modul an, um die Verschlüsselung des Dokuments via die TR-S.3 -Schnittstelle zu initiieren. Es werden das Klartextdokument D' und zwei öffentliche Schlüssel (X509-Zertifikate) der berechtigten Nutzer K_{pub1} und K_{pub2} übergeben.
3.	.GenSymKey () → K_{enc}'	Das lokale Krypto-Modul leitet einen symmetrischen Schlüssel für die Verschlüsselung ab. Ausgabe K_{enc}' – symmetrische Verschlüsselungsschlüssel.
4.	Encrypt (D', K_{enc}') → D_{enc}'	Das lokale Krypto-Modul verschlüsselt D' mit dem generierten Schlüssel K_{enc}' . Ergebnis ist ein verschlüsseltes Dokument D_{enc}' .
5.	EncryptSymKey (K_{enc}' , K_{pub1}) → $RecipientInfo1'$	Das lokale Krypto-Modul verschlüsselt den generierten symmetrischen Verschlüsselungsschlüssel K_{enc}' mit dem asymmetrischen öffentlichen Schlüssel des Nutzers K_{pub1}' . Das Ergebnis ist eine Struktur $RecipientInfo1'$.
6.	EncryptSymKey (K_{enc}' , K_{pub2}) → $RecipientInfo2'$	Das lokale Krypto-Modul verschlüsselt den generierten symmetrischen Verschlüsselungsschlüssel K_{enc}' mit dem asymmetrischen öffentlichen Schlüssel des Nutzers K_{pub2}' . Das Ergebnis ist eine Struktur $RecipientInfo2'$.
7.	CreateRecipientInfos ($RecipientInfo1'$, $RecipientInfo2'$) → $RecipientInfos'$	Das lokale Krypto-Modul legt die beiden erzeugten $RecipientInfo1'$ und $RecipientInfo2'$ in eine gemeinsame Struktur $RecipientInfos'$, inkl. weiterer Daten, wie z. B. Schlüsselverschlüsselungsalgorithmus.
8.	Return (D_{enc}' , $RecipientInfos'$)	Antwort auf den Schritt 2., das lokale Krypto-Modul liefert das verschlüsselte Dokument D' als D_{enc}' und die korrespondierende $RecipientInfo'$ -Struktur als Zugriffsberechtigung an die Applikation zurück.
9.	Update (D', D_{enc}' , $RecipientInfos'$)	Die Applikation ruft die Update -Funktion des XML-Adapters auf und übergibt die zu aktualisierenden Daten: D' – angepasste Klartextdokument D, D_{enc}' – verschlüsseltes Klartextdokument D', $RecipientInfos'$ – die Struktur mit der Zugriffsberechtigungen.
10.	BaueRecipXAIP ($RecipientInfos'$) → $ReXAIP'$	Der XML-Adapter erzeugt ein XAIP ($ReXAIP'$) und legt darin die erhaltene $RecipientInfo'$ -Struktur ab.

Nr.	Aufruf	Beschreibung
11.	ArchiveSubmissionRequest (ReXAIP')	Der XML-Adapter ruft die <code>ArchiveSubmission</code> -Funktion an der TR-S.4 Schnittstelle des zentralen ArchiSafe-Moduls auf und übergibt das erzeugte ReXAIP'.
12.	ArchiveSubmissionRequest (ReXAIP')	Das zentrale ArchiSafe-Modul ruft die <code>ArchiveSubmission</code> -Funktion an der TR-S.6 -Schnittstelle auf und übergibt das ReXAIP'.
13.	ErzeugeAOID () → ReAOID'	Das zentrale ArchiSafe-Modul erzeugt eine ReAOID' für ReXAIP'.
14.	EintragenAOID (ReAOID', ReXAIP') → ReXAIP+'	Das zentrale ArchiSafe-Modul trägt die erzeugte ReAOID' in das ReXAIP' in das VersionManifest-Element ein. Es entsteht ein ReXAIP+ '.
15.	KanonisiereLXAIP (ReXAIP+') → K(ReXAIP+')	Das zentrale ArchiSafe-Modul kanonisiert das ReXAIP+ ', es entsteht ein K(ReXAIP+ ').
16.	HashRequest (RecipientInfos')	Das zentrale ArchiSafe-Modul ruft die <code>Hash</code> -Funktion an der TR-S.3 -Schnittstelle (ArchiSig-Modul), um den Hashwert über die aus dem K(ReXAIP+ ') entnommene RecipientInfos'-Struktur zu berechnen.
17.	Return (H(RecipientInfos'))	Es wird der Hashwert H(RecipientInfos') zurückgeliefert.
18.	Speichern (ReAOID', H(RecipientInfos'))	Das zentrale ArchiSig-Modul speichert die ReAOID' und H(RecipientInfos') in internen Datenstrukturen und aktualisiert den Hashbaum.
19.	ArchiveSubmissionRequest (ReXAIP+')	Das zentrale ArchiSig-Modul ruft die <code>ArchiveSubmission</code> -Funktion an der TR-S.2 -Schnittstelle des zentralen ECM/Langzeitspeichers auf, um die ReXAIP+ ' im Speicher persistent abzulegen.
20.	Return (ReAOID')	Das zentrale ECM/Langzeitspeicher-Modul liefert die ReAOID' als Bestätigung der erfolgreichen Speicherung des korrespondierenden ReXAIP+ ' an das zentrale ArchiSig-Modul zurück.
21.	Return (ReAOID')	Das zentrale ArchiSig-Modul liefert die ReAOID' als Bestätigung der erfolgreichen Verarbeitung des korrespondierenden ReXAIP+ ' an das zentrale ArchiSafe-Modul zurück.
22.	Return (ReAOID')	Das zentrale ArchiSafe-Modul liefert die ReAOID' an die Applikation zurück, als Bestätigung der erfolgreichen Ablage vom ReXAIP+ ' in die zentrale Middleware.
23.	Upload (D')	Die Applikation initiiert einen Upload vom Klartextdokument D' an der Schnittstelle zum lokalen Up-/Download-Modul.

Nr.	Aufruf	Beschreibung
24.	ArchiveSubmissionRequest(D')	Das lokale Up-/Download-Modul ruft die <code>ArchiveSubmission</code> -Funktion an der TR-S.2 Schnittstelle vom lokalen ECM/Langzeitspeicher auf und übergibt das Klartextdokument D' .
25.	Speichern(D') → DLRef'	Der lokale ECM/Langzeitspeicher erzeugt eine Referenz $DLRef'$ für das lokal gespeicherte Klartextdokument D' und legt dieses unter $DLRef'$ Referenz ab.
26.	Return(DLRef')	Die erzeugte Referenz ($DLRef'$) auf das lokal abgelegte Klartextdokument D' wurde an das lokale Up-/Download-Modul zurückgegeben.
27.	Return(DLRef')	Das lokale Up-/Download-Modul gibt die Referenz $DLRef'$ auf das lokal abgelegt Klartextdokument D' an die Applikation zurück.
28.	ArchiveRetrivalRequest(AOID)	Der XML-Adapter ruf die <code>ArchiveRetrieval</code> -Funktion an der TR-S.4 -Schnittstelle des zentralen ArchiSafe-Moduls auf und übergibt die <code>AOID</code> .
29.	ArchiveRetrivalRequest(AOID)	Das zentrale ArchiSafe-Modul ruft die <code>ArchiveRetrieval</code> -Funktion an der TR-S.5 -Schnittstelle des zentralen ECM/Langzeitspeichers auf und übergibt die <code>AOID</code> .
30.	ObtainLXAIP(AOID) → LXAIP++	Der zentrale ECM/Langzeitspeicher ermittelt das zur <code>AOID</code> zugehörige D-AIP $LXAIP++$ in der Version <code>V001</code> .
31.	Return(LXAIP++)	Der zentrale ECM/Langzeitspeicher liefert das im 30. Schritt ermittelte $LXAIP++$ an das zentrale ArchiSafe-Modul zurück.
32.	Return(LXAIP++)	Das zentrale ArchiSafe-Modul liefert das $LXAIP++$ an den XML-Adapter zurück.
33.	BaueDLXAIP(Denc', DLRef', ReAOID', LXAIP++) → DLXAIP'	-Der XML-Adapter erstellt basierend auf $LXAIP++$ ein $DLXAIP'$ für die Version <code>V002</code> und legt darin folgende Objekte ab: • $Denc'$ – verschlüsseltes Klartextdokument D', • $DLRef'$ – lokale Referenz auf Klartextdokument D', • $ReAOID'$ – Referenz auf die zugehörige Z-AIP.
34.	ArchiveUpdateRequest(DLXAIP')	Der XML-Adapter ruft die <code>ArchiveUpdate</code> -Funktion an der TR-S.4 -Schnittstelle des zentralen ArchiSafe-Moduls auf und übergibt das im Schritt 33 erstellte $DLXAIP'$.
35.	VerifyRequest(DLRef')	Das zentrale ArchiSafe-Modul ermittelt, dass es sich u.a. um eine lokal auflösbare Referenz ($DLRef'$) handelt, ruft die <code>Verify</code> -Funktion an der TR-S.1 Schnittstelle des zentralen Krypto Moduls auf und übergibt $DLRef'$.

Nr.	Aufruf	Beschreibung
36.	VerifyRequest (DLRef')	Das zentrale Krypto-Modul ruft die <code>Verify</code> -Funktion an der TR-S.1' -Schnittstelle des lokalen Krypto-Moduls auf und übergibt die Referenz auf das lokal gespeicherte Klartextdokument (DLRef').
37.	ArchiveRetrievalRequest (DLRef')	Das lokale Krypto-Modul ruft die <code>ArchiveRetrieval</code> -Funktion an der TR-S.5' -Schnittstelle des lokalen ECM/Langzeitspeicher-Moduls auf und übergibt die lokale Referenz auf das Klartextdokument, DLRef'.
38.	Return (D')	Das lokale ECM/Langzeitspeicher-Modul ermittelt das zu der lokalen Referenz DLRef' zugehörige Klartextdokument D' und liefert dieses an das lokale Krypto-Modul zurück.
39.	VerifyRequest (D') → VR(D')	Das lokale Krypto-Modul verifiziert die Signaturen/Siegl/Zeitstempel, die auf dem Klartextdokument D' eingebracht worden sind und liefert das Verifikationsprotokoll VR(D').
40.	HoleOCSP (DSigCert')	Das lokale Krypto-Modul ermittelt während der Verifikation der kryptographischen Artefakte, die an das Dokument D' angebracht worden sind, die Sperrstatus der relevanten Zertifikate, indem die korrespondierende Schnittstelle (OCSP) des VDA angesprochen wird.
41.	Return (OCSP(DSigCert'))	Der VDA liefert die entsprechend Status zu den angefragten Zertifikaten zurück (OCSP(DSigCert') etc.).
42.	Return (VR(D'))	Das lokale Krypto-Modul liefert an das zentrale Krypto-Modul das Verifikationsprotokoll VR(D') über die Validierung des Klartextdokument D' inkl. der darin enthaltenen kryptographischen Artefakte zurück.
43.	Return (VR(D'))	Das zentrale Krypto-Modul liefert das Verifikationsprotokoll VR(D') an das zentrale ArchiSafe-Modul zurück.
44.	Einbetten (VR(D'),DLXAIP') → DLXAIP+'	Das zentrale ArchiSafe-Modul bettet das erhaltene Verifikationsprotokoll in das vorhandenen LXAIP' ein. Das Ergebnis ist LXAIP+'.
45.	ArchiveUpdateRequest (DLXAIP+')	Das zentrale ArchiSafe-Modul ruft die <code>ArchiveUpdate</code> -Funktion an der TR-S.6 -Schnittstelle des zentralen ArchiSig-Moduls auf und übergibt das im Schritt 44 erzeugte LXAIP+'.
46.	ArchiveRetrievalRequest (AOID, all)	Das zentrale ArchiSig-Modul ruft <code>ArchiveRetrieval</code> -Funktion an der TR-S.2 -Schnittstelle des zentralen ECM/Langzeitspeichers auf und übergibt die AOID, sowie Flag „all“, um alle Versionen des D-AIP zu ermitteln.

Nr.	Aufruf	Beschreibung
47.	Return (LXAIP++)	Der zentrale ECM/Langzeitspeicher ermittelt und liefert das LXAIP++ an das zentrale ArchiSig-Modul zurück.
48.	Einbetten (DLXAIP+',LXAIP++) → LXAIP++'	Das zentrale ArchiSig-Modul integriert die im DLXAIP+' vorhandene Version v002 in das vorliegende LXAIP++. Es entsteht LXAIP++'.
49.	KanonisiereLXAIP (LXAIP++'): K(LXAIP++')	Das zentrale ArchiSig-Modul kanonisiert das LXAIP++'. Es entsteht ein K(LXAIP++').
50.	HashRequest (Denc')	Das zentrale ArchiSig-Modul ruft die Hash-Funktion an der TR-S.3-Schnittstelle des zentralen Krypto-Moduls auf und übergibt das verschlüsselte Dokument Denc'.
51.	Return (H(Denc'))	Das zentrale Krypto-Modul liefert den Hashwert H(Denc') über das verschlüsselte Element Denc' zurück.
52.	HashRequest (VR(D'))	Das zentrale ArchiSig-Modul ruft die Hash-Funktion an der TR-S.3-Schnittstelle des zentralen Krypto-Moduls auf und übergibt das Verifikationsprotokoll VR(D') über das Klardokument D'.
53.	Return (H(VR(D')))	Das zentrale Krypto-Modul liefert den Hashwert über das Verifikationsprotokoll vom Klartextdokument D', H(R(D')), zurück.
54.	HashRequest (ReAOID')	Das zentrale ArchiSig-Modul ruft die Hash-Funktion an der TR-S.3-Schnittstelle des zentralen Krypto-Moduls auf und übergibt die ReAOID', vom zuvor abgelegten ReXAIP', das die Zugriffsberechtigung auf die verschlüsselten Inhalte hält.
55.	Return (H(ReAOID'))	Das zentrale Krypto-Modul liefert den Hashwert H(ReAOID') über die ReAOID', vom zuvor abgelegten ReXAIP', welches die Zugriffsberechtigung auf die verschlüsselten Inhalte hält.
56.	HashRequest (DLRef')	Das zentrale ArchiSig-Modul ruft die Hash-Funktion an der TR-S.3-Schnittstelle des zentralen Krypto-Moduls auf und übergibt die Referenz, DLRef', auf das zuvor lokal abgelegte Klartextdokument D'.
57.	HashRequest (DLRef')	Das zentrale Krypto-Modul ruft die Hash-Funktion an der TR-S.3'-Schnittstelle des lokalen Krypto-Moduls auf und übergibt die Referenz, DLRef', auf das zuvor lokal abgelegte Klartextdokument D'.
58.	ArchiveRetrievalRequest (DLRef')	Das lokale Krypto-Modul ruft die ArchiveRetrieval-Funktion an der TR-S.5'-Schnittstelle des lokalen ECM/Langzeitspeichers auf und übergibt die Referenz, DLRef', auf das zuvor lokal abgelegte Klartextdokument D'.

Nr.	Aufruf	Beschreibung
59.	Return(D')	Der lokale ECM/Langzeitspeicher liefert das Klartextdokument D' an das lokale Krypto-Modul zurück.
60.	HashRequest(D') $\rightarrow H(D')$	Das lokale Krypto-Modul ruft die eigene Hash-Funktion auf und berechnet den Hashwert $H(D')$ über das Klartextdokument D' .
61.	Return(H(D'))	Das lokale Krypto-Modul liefert den Hashwert $H(D')$ an das zentrale Krypto-Modul der zentralen Middleware zurück.
62.	Return(H(D'))	Das zentrale Krypto-Modul liefert den Hashwert $H(D')$ an das zentrale ArchiSig-Modul zurück.
63.	Speichern(AOID, H(Denc'), H(D'), H(VR(D')), H(ReAOID'))	Das zentrale ArchiSig-Modul speichert die $AOID$, $H(Denc')$, $H(D')$, $H(VR(D'))$, $H(ReAOID')$ in internen Datenstrukturen und aktualisiert den Hashbaum.
64.	ArchiveUpdateRequest(DLXAIP++)	Das zentrale ArchiSig-Modul ruft die <code>ArchiveUpdate</code> -Funktion an der TR-S.2 -Schnittstelle des zentralen ECM/Langzeitspeichers auf, um die $DLXAIP++$ im Speicher persistent abzulegen.
65.	Speichern(DLXAIP++) $\rightarrow V002$	Der zentrale ECM-Langzeitspeicher speichert $DLXAIP++$ und liefert die neue Version $V002$ als Bestätigung.
66.	Return(V002)	Der zentrale ECM/Langzeitspeicher liefert die $V002$ als Bestätigung der erfolgreichen Speicherung des korrespondierenden $DLXAIP++$ an das zentrale ArchiSig-Modul zurück.
67.	Return(V002)	Das zentrale ArchiSig-Modul liefert die $V002$ als Bestätigung der erfolgreichen Verarbeitung des korrespondierenden $DLXAIP++$ an das zentrale ArchiSafe-Modul zurück.
68.	Return(V002)	Das zentrale ArchiSafe-Modul liefert die $V002$ an den XML-Adapter zurück, als Bestätigung der erfolgreichen Ablage vom $DLXAIP++$ in der zentralen Middleware.
69.	Return(AOID, V002)	Der XML-Adapter liefert die $AOID$ und $V002$ an die Applikation zurück.
70.	Return(AOID, V002)	Die Applikation liefert die $AOID$ und $V002$ an den Nutzer zurück.

Tabelle 4: Änderung der bewahrten Daten – Beschreibung zur Abbildung 10

3.4.4 Rückgabe der technischen Beweisdaten

Die technischen Beweisdaten werden unter Verwendung des Standardprozesses der zentralen TR-ESOR-Middleware ermittelt (vgl. [TR-ESOR], Kap. 7.5.4). Es müssen dabei folgende Evidence Records abgerufen werden:

- Der zu dem D-AIP zugehörige Evidence Record, vgl. hierzu das D-AIP mit der AOID $AOID-DOK-001$ in der Abbildung 7,

- Die zu den allen Z-AIPs zugehörigen Evidence Records, die aus dem führenden D-AIP referenziert werden, vgl. hierzu die beiden Z-AIPs mit den AOIDs AOID-RI-100 und AOID-RI-102 in der Abbildung 7.

Bezogen auf die oben zitierte Abbildung 7 müssen alle drei Evidence Records abgerufen werden, damit die gesamte Konstellation D-AIP und zugehörige Z-AIPs verifiziert werden können:

- ER-AOID-DOK-001 – schützt das D-AIP, mit der AOID: AOID-DOK-001,
- ER-AOID-RI-100 – schützt das Z-AIP mit der AOID: AOID-RI-100,
- ER-AOID-RI-102 – schützt das Z-AIP, mit der AOID: AOID-RI-102.

Bezogen auf den Ablauf aus der Abbildung 8 sind das zwei Evidence Records:

- ER-V1-LXAIP++ – schützt das initiale (die Version V001) LXAIP++ (D-AIP),
- ER-V1-ReXAIP+ – schützt das initiale (die Version V001) ReXAIP (Z-AIP).

3.4.5 Prüfen der beweisrelevanten Daten und technischen Beweisdaten

Da für die Prüfung der technischen Beweisdaten auch die Hashwerte für die Klartextdokumente berechnet werden müssen, muss bei der Verwendung der Verify- bzw. ValidateEvidence-Funktion an der **TR-S.4**, bzw. **TR-S.512**-Schnittstelle der zentralen Middleware der in der Abbildung 11 dargestellte Ablauf angewandt werden.

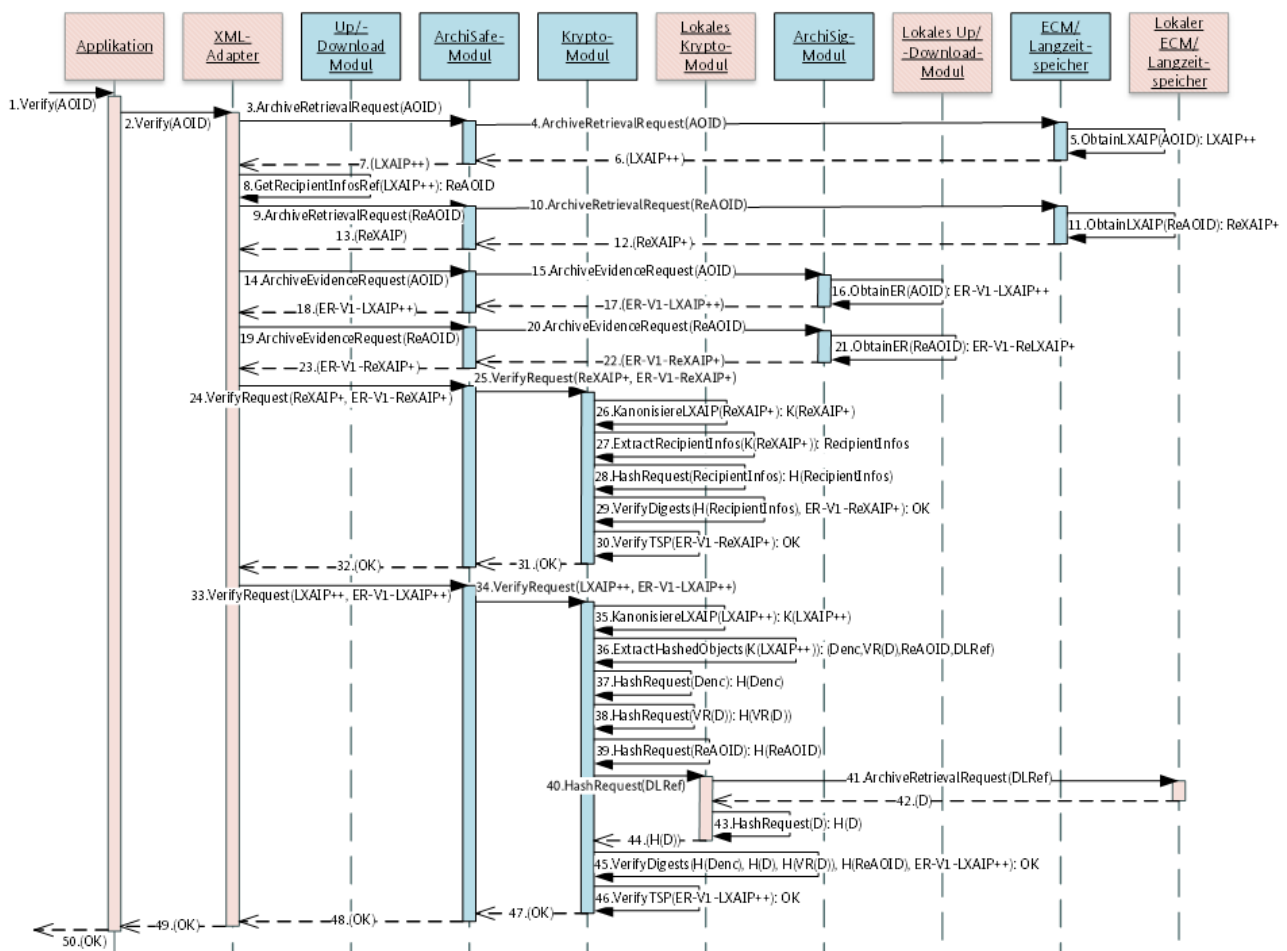


Abbildung 11: Verifikation der Beweisdaten gem. [TR-ESOR-ENC].

Die nachfolgende Tabelle 5 beschreibt die einzelnen Schritte, dargestellt in der Abbildung 11.

Nr.	Aufruf	Beschreibung
1.	Verify(AOID)	Der Nutzer initiiert an der Applikation-Schnittstelle eine Prüfung der letzten Version eines D-AIP, die mit der Hilfe einer AOID <code>AOID</code> referenziert wird.
2.	Verify(AOID)	Die Applikation leitet diesen Aufruf an den XML-Adapter transparent weiter.
3.	ArchiveRetrievalRequest(AOID)	Der XML-Adapter ruft die <code>ArchiveRetrieval</code> -Funktion an der TR-S.4 -Schnittstelle des zentralen ArchiSafe-Moduls auf und übergibt die <code>AOID</code> .
4.	ArchiveRetrievalRequest(AOID)	Das zentrale ArchiSafe-Modul ruft die <code>ArchiveRetrieval</code> -Funktion an der TR-S.5 -Schnittstelle des zentralen ECM/Langzeitspeichers auf und übergibt die <code>AOID</code> .
5.	ObtainLXAIP(AOID) → LXAIP++	Der zentrale ECM/Langzeitspeicher ermittelt im Speicher das zur <code>AOID</code> zugehörige D-AIP, nämlich <code>LXAIP++</code> .
6.	Return(LXAIP++)	Der zentrale ECM/Langzeitspeicher liefert das <code>LXAIP++</code> an das zentrale ArchiSafe-Modul zurück.
7.	Return(LXAIP++)	Das zentrale ArchiSafe-Modul liefert das <code>LXAIP++</code> an den XML-Adapter zurück.
8.	GetRecipientInfosRef(LXAIP++) → ReAOID	Der XML-Adapter ermittelt die Referenzen auf zugehörige Z-AIPs aus dem <code>LXAIP++</code> . In diesem Fall wird einzelne Referenz ermittelt: <code>ReAOID</code> .
9.	ArchiveRetrievalRequest(ReAOID)	Der XML-Adapter ruft die <code>ArchiveRetrieval</code> -Funktion an der TR-S.4 -Schnittstelle des zentralen ArchiSafe-Moduls auf und übergibt die <code>ReAOID</code> .
10.	ArchiveRetrievalRequest(ReAOID)	Das zentrale ArchiSafe-Modul ruft die <code>ArchiveRetrieval</code> -Funktion an der TR-S.5 -Schnittstelle des zentralen ECM/Langzeitspeichers auf und übergibt die <code>ReAOID</code> .
11.	ObtainLXAIP(ReAOID) → ReXAIP+	Der zentrale ECM/Langzeitspeicher ermittelt im Speicher das zur <code>ReAOID</code> zugehörige D-AIP, nämlich <code>ReXAIP+</code> .
12.	Return(ReXAIP+)	Der zentrale ECM/Langzeitspeicher liefert das <code>ReXAIP+</code> an das zentrale ArchiSafe-Modul zurück.
13.	Return(ReXAIP+)	Das zentrale ArchiSafe-Modul liefert das <code>ReXAIP+</code> an den XML-Adapter zurück.
14.	ArchiveEvidenceRequest(AOID)	Der XML-Adapter ruft die <code>ArchiveEvidence</code> -Funktion an der TR-S.4 -Schnittstelle des zentralen ArchiSafe-Moduls auf und übergibt die <code>AOID</code> .
15.	ArchiveEvidenceRequest(AOID)	Das zentrale ArchiSafe-Modul ruft die <code>ArchiveEvidence</code> -Funktion an der TR-S.6 -Schnittstelle des zentralen ArchiSig-Moduls auf und übergibt die <code>AOID</code> .

Nr.	Aufruf	Beschreibung
16.	ObtainER (AOID) → ER-V1-LXAIP++	Das zentrale ArchiSig-Modul ermittelt den zur AOID gehörenden Evidence Record, ER-V1-LXAIP++. Der Evidence Record schützt die letzte Version v001 des LXAIP++
17.	Return (ER-V1-LXAIP++)	Das zentrale ArchiSig-Modul liefert den ermittelten Evidence Record ER-V1-LXAIP++ an das zentrale ArchiSafe-Modul zurück.
18.	Return (ER-V1-LXAIP++)	Das zentrale ArchiSafe-Modul liefert den Evidence Record ER-V1-LXAIP++ an den XML-Adapter zurück.
19.	ArchiveEvidenceRequest (ReAOID)	Der XML-Adapter ruft die ArchiveEvidence-Funktion an der TR-S.4-Schnittstelle des zentralen ArchiSafe-Moduls auf und übergibt die ReAOID.
20.	ArchiveEvidenceRequest (ReAOID)	Das zentrale ArchiSafe-Modul ruft die ArchiveEvidence-Funktion an der TR-S.6-Schnittstelle des zentralen ArchiSig-Moduls auf und übergibt die ReAOID.
21.	ObtainER (ReAOID) → ER-V1-ReXAIP+	Das zentrale ArchiSig-Modul ermittelt den zur ReAOID gehörenden Evidence Record, ER-V1-ReXAIP+. Der Evidence Record schützt die letzte Version v001 des ReXAIP+
22.	Return (ER-V1-ReXAIP+)	Das zentrale ArchiSig-Modul liefert den ermittelten Evidence Record ER-V1-ReXAIP+ an das zentrale ArchiSafe-Modul zurück.
23.	Return (ER-V1-ReXAIP+)	Das zentrale ArchiSafe-Modul liefert den Evidence Record ER-V1-ReXAIP+ an den XML-Adapter zurück.
24.	VerifyRequest (ReXAIP+, ER-V1-ReXAIP+)	Der XML-Adapter ruft die Verify-Funktion an der TR-S.4-Schnittstelle des zentralen ArchiSafe-Moduls auf und übergibt das im Schritt 13 ermittelte ReXAIP+ zusammen mit dem im Schritt 23 ermittelten ER-V1-ReXAIP+.
25.	VerifyRequest (ReXAIP+, ER-V1-ReXAIP+)	Das zentrale ArchiSafe-Modul ruft die Verify-Funktion an der TR-S.1-Schnittstelle des zentralen Krypto-Moduls auf und übergibt das ReXAIP+ mit den zugehörigen Evidence Record ER-V1-ReXAIP+.
26.	KanonisiereLXAIP (ReXAIP+) → K(ReXAIP+)	Das zentrale Krypto-Modul kanonisiert das ReXAIP+, es entsteht ein K(ReXAIP+).
27.	ExtractRecipientInfos (K(ReXAIP+)) → RecipientInfos	Das zentrale Krypto-Modul extrahiert das RecipientInfos-Element aus dem K(ReXAIP+).
28.	HashRequest (RecipientInfos) → H(RecipientInfos)	Das zentrale Krypto-Modul berechnet den Hashwert über das in Schritt 27 extrahierte RecipientInfos-Element: H(RecipientInfos).

Nr.	Aufruf	Beschreibung
29.	VerifyDigests (H(RecipientInfos), ER-V1-ReXAIP+) → OK	Das zentrale Krypto-Modul verifiziert den in Schritt 28 berechneten Hashwert $H(\text{RecipientInfos})$ gegenüber dem übergebenen Evidence Record ER-V1-ReXAIP+ . Es wird die mathematische Korrektheit des Hashbaumes geprüft. Das Prüfergebnis ist positiv.
30.	VerifyTSP (ER-V1-ReXAIP+) → OK	Das zentrale Krypto-Modul verifiziert den Zeitstempel des Evidence Records ER-V1-ReXAIP+ , inkl. Prüfung der Zertifikatskette bis zum Vertrauensanker ¹⁴ . Das Prüfergebnis ist positiv.
31.	Return (OK)	Da die beiden Prüfergebnisse aus den Schritten 29 und 30 positiv ausgefallen sind, liefert das zentrale Krypto-Modul ein positives Prüfergebnis an das zentrale ArchiSafe-Modul zurück.
32.	Return (OK)	Das zentrale ArchiSafe-Modul liefert das positive Prüfergebnis an den XML-Adapter zurück. Die Unversehrtheit des Z-AIP ReXAIP+ wurde bestätigt.
33.	VerifyRequest (LXAIP++, ER-V1-LXAIP++)	Der XML-Adapter ruft die <i>Verify</i> -Funktion an der TR-S.4 -Schnittstelle des zentralen ArchiSafe-Moduls auf und übergibt das in Schritt 7 ermittelte LXAIP++ zusammen mit dem in Schritt 18 ermittelten ER-V1-LXAIP++ .
34.	VerifyRequest (LXAIP++, ER-V1-LXAIP++)	Das zentrale ArchiSafe-Modul ruft die <i>Verify</i> -Funktion an der TR-S.1 -Schnittstelle des zentralen Krypto-Moduls auf und übergibt das LXAIP++ mit dem zugehörigen Evidence Record ER-V1-LXAIP++ .
35.	KanonisiereLXAIP (LXAIP++) → K(LXAIP++)	Das zentrale Krypto-Modul kanonisiert das LXAIP++ , es entsteht ein $K(\text{LXAIP++})$.
36.	ExtractHashedObjects (K(LXAIP++)) → (Denc, VR(D), ReAOID, DLRef)	Das zentrale Krypto-Modul extrahiert die durch die <i>ProtectedObjectPointer</i> -Elemente referenzierten Objekte aus dem $K(\text{LXAIP++})$. Es werden folgende Elemente extrahiert: • Denc – das verschlüsselte Klartextdokument D, • $\text{VR}(D)$ – Prüfbericht zu den im Klartextdokument D enthaltenen Signaturen, • ReAOID – Referenz auf die zugehörige Z-AIPs, • DLRef – Referenz auf das lokal gespeicherte Klartextdokument D.
37.	HashRequest (Denc) → H(Denc)	Das zentrale Krypto-Modul berechnet den Hashwert über das in Schritt 36 extrahierte Objekt Denc : $H(\text{Denc})$.
38.	HashRequest (VR(D)) → H(VR(D))	Das zentrale Krypto-Modul berechnet den Hashwert über das in Schritt 36 extrahierte Objekt $\text{VR}(D)$: $H(\text{VR}(D))$.

¹⁴ Die Prüfung der Kette kann innerhalb der Middleware durch eine Verifikationskomponente erfolgen, oder an ein Verifikationsdienst (VDA) delegiert werden.

Nr.	Aufruf	Beschreibung
39.	HashRequest (ReAOID) $\rightarrow$ H(ReAOID)	Das zentrale Krypto-Modul berechnet den Hashwert über das in Schritt 36 extrahierte Objekt <code>ReAOID</code> : $H(\text{ReAOID})$.
40.	HashRequest (DLRef)	Das zentrale Krypto-Modul ruft die Hash-Funktion an der TR-S.3' -Schnittstelle des lokalen Krypto-Moduls auf und übergibt die Referenz <code>DLRef</code> auf das lokal gespeicherte Klartextdokument <code>D</code> . Der benötigte Hashwert über das Klartextdokument <code>D</code> muss lokal berechnet werden.
41.	ArchiveRetrievalRequest (DLRef)	Das lokale Krypto-Modul ruft die <code>ArchiveRetrieval</code> -Funktion an der TR-S.5' -Schnittstelle des lokalen ECM/Langzeitspeichers auf, um das durch die <code>DLRef</code> referenzierte Klartextdokument <code>D</code> zu ermitteln.
42.	Return (D)	Das lokale ECM/Langzeitspeicher liefert das angefragte Klartextdokument <code>D</code> an das lokale Krypto-Modul zurück.
43.	HashRequest (D) $\rightarrow$ H(D)	Das lokale Krypto-Modul berechnet den Hashwert über das in Schritt 42 ermittelte Klartextdokument <code>D</code> , $H(D)$.
44.	Return (H(D))	Das lokale Krypto-Modul liefert den in Schritt 43 berechneten Hashwert $H(D)$ an das zentrale Krypto-Modul zurück.
45.	VerifyDigests (H(Denc), H(D), H(VR(D)), H(ReAOID), ER-V1-LXAIP++) $\rightarrow$ OK	Das zentrale Krypto-Modul verifiziert die in den Schritten 37, 38, 39 und 43 berechneten Hashwerte $H(\text{Denc})$, $H(\text{VR}(D))$, $H(\text{ReAOID})$, $H(D)$ gegenüber den übergebenen Evidence Record <code>ER-V1-LXAIP++</code> . Es wird die mathematische Korrektheit des Hashbaumes geprüft. Das Prüfergebnis ist positiv.
46.	VerifyTSP (ER-V1-LXAIP++) $\rightarrow$ OK	Das zentrale Krypto-Modul verifiziert den Zeitstempel des Evidence Records <code>ER-V1-LXAIP++</code> , inkl. Prüfung der Zertifikatskette bis zum Vertrauensanker ¹⁴ . Das Prüfergebnis ist positiv.
47.	Return (OK)	Da die beiden Prüfergebnisse aus den Schritten 45 und 46 positiv ausgefallen sind, liefert das zentrale Krypto-Modul ein positives Prüfgesamtergebnis an das zentrale ArchiSafe-Modul zurück.
48.	Return (OK)	Das zentrale ArchiSafe-Modul liefert das positive Prüfergebnis an den XML-Adapter zurück. Die Unversehrtheit des D-AIP <code>LXAIP++</code> wurde bestätigt.
49.	Return (OK)	Da die Unversehrtheit sowohl des D-AIP (Schritt 48) als auch des zugehörigen Z-AIP (Schritt 32) bestätigt worden sind, liefert der XML-Adapter ein positives Prüfergebnis der Verifikation an die Applikation zurück.

Nr.	Aufruf	Beschreibung
50.	Return(OK)	Die Applikation leitet das positive Prüfergebnis an den Nutzer weiter.

Tabelle 5: Verifikation der Beweisdaten – Beschreibung zur Abbildung 11.

Grundsätzlich kann die Unversehrtheit der einzelnen Klartextdokumente auch lokal erfolgen. Für diesen Grund muss der zum korrespondierenden D-AIP zugehörnde Evidence Record aus der zentralen Middleware abgerufen werden (siehe dazu Kap. 3.4.4) und zur Prüfung des besagten Klartextdokuments mit Hilfe eines lokalen verfügbaren Verifikationswerkzeugs (z.B. BSI-ErVerifyTool) herangezogen werden. Bezogen auf die **TR-S.4**-Schnittstelle und den Beispielablauf aus der Abbildung 8 ergibt das die folgenden Schritte (Klartextdokument **D** liegt bereits lokal vor):

- ArchiveEvidence(AOID, V001) → ER-V1-LXAIP++,
- Verify(D, ER-V1-LXAIP++) → ResultMajor: OK.

3.4.6 Vernichten von Archivinformationspaketen, inkl. Daten

Da die relevanten Daten verteilt gespeichert werden (sowohl in lokalen als auch in zentralen ECM/Langzeitspeichern) muss die führende Applikation die entsprechende Logik umsetzen, um die AIPs inkl. zugehörigen Daten restlos zu vernichten.

A.15 Um die lokal aufbewahrten Klartextdokumente löschen zu können muss die Schnittstelle des lokalen Up/Download-Moduls um eine Delete-Funktion erweitert werden (vgl. hierzu Kap. 3.5.2)¹⁵.

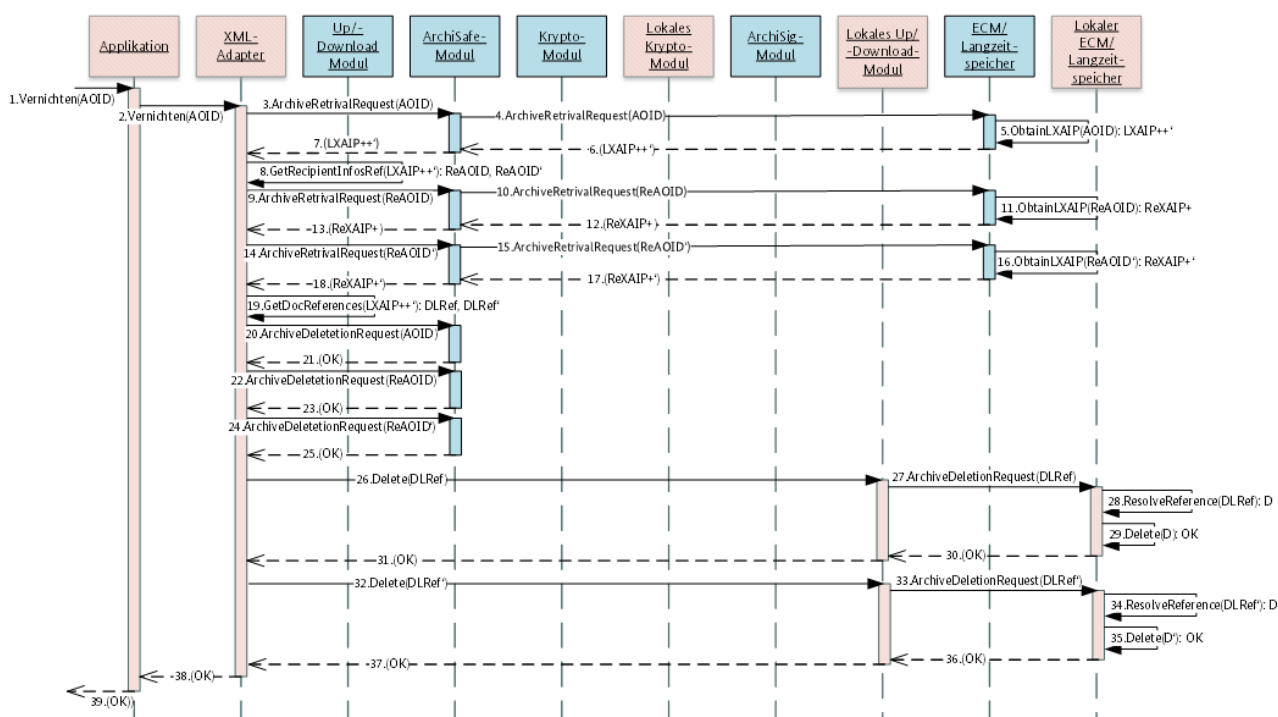


Abbildung 12: Vernichten von AIPs inkl. zugehörigen Daten gem. [TR-ESOR-ENC].

Die nachfolgende Tabelle 6 beschreibt die einzelnen Schritte, dargestellt in der Abbildung 12.

¹⁵ Ein direkter Zugriff auf den lokalen ECM/Langzeitspeicher ist zwar gegeben, es ist aber ausschließlich ein lesender Zugriff gestattet (vgl. Abbildung 1 und Abbildung 2)

Nr.	Aufruf	Beschreibung
1.	Vernichten(AOID)	Nutzer initiiert das Vernichten eines D-AIP mit der AOID, AOID in der Applikation Diese Operation impliziert die Vernichtung aller zugehörigen Z-AIPs sowie aller lokal gespeicherten Klartextdokumente. Die Gesamtoperation muss durch die Applikation bzw. XML-Adapter (als deren Teil) orchestriert werden.
2.	Vernichten(AOID)	Die Applikation ruft die <code>Vernichten</code> -Funktion an der Schnittstelle zum XML-Adapter auf und übergibt die AOID.
3.	ArchiveRetrievalRequest(AOID)	Der XML-Adapter ruft die <code>ArchiveRetrieval</code> -Funktion an der TR-S.4 -Schnittstelle des zentralen ArchiSafe-Moduls auf und übergibt die AOID.
4.	ArchiveRetrievalRequest(AOID)	Das zentrale ArchiSafe-Modul ruft die <code>ArchiveRetrieval</code> -Funktion an der TR-S.5 -Schnittstelle des zentralen ECM/Langzeitspeichers auf und übergibt die AOID.
5.	ObtainLXAIP(AOID) → LXAIP++'	Der zentrale ECM/Langzeitspeicher ermittelt die zugehörige LXAIP++', das die Versionen v001 und v002 beinhaltet.
6.	Return(LXAIP++')	Der zentrale ECM/Langzeitspeicher gibt das ermittelte LXAIP++' an das zentrale ArchiSafe-Modul zurück.
7.	Return(LXAIP++')	Das zentrale ArchiSafe-Modul gibt das ermittelte LXAIP++' an den XML-Adapter zurück.
8.	GetRecipientInfosRef(LXAIP++') → ReAOID, ReAOID'	Der XML-Adapter ermittelt aus dem LXAIP++' die Referenzen auf die zugehörigen Z-AIPs. Für jede enthaltene Version wird eine Referenz ermittelt: ReAOID und ReAOID'.
9.	ArchiveRetrievalRequest(ReAOID)	Der XML-Adapter ruft die <code>ArchiveRetrieval</code> -Funktion an der TR-S.4 -Schnittstelle des zentralen ArchiSafe-Moduls auf und übergibt die ReAOID.
10.	ArchiveRetrievalRequest(ReAOID)	Das zentrale ArchiSafe-Modul ruft die <code>ArchiveRetrieval</code> -Funktion an der TR-S.5 -Schnittstelle des zentralen ECM/Langzeitspeichers auf und übergibt die ReAOID.
11.	ObtainLXAIP(ReAOID) → ReXAIP+	Der zentrale ECM/Langzeitspeicher ermittelt die zugehörige ReXAIP+.
12.	Return(ReXAIP+)	Der zentrale ECM/Langzeitspeicher gibt das ermittelte ReXAIP+ an das lokale ArchiSafe-Modul zurück.
13.	Return(ReXAIP+)	Das zentrale ArchiSafe-Modul gibt das ermittelte ReXAIP+ an den XML-Adapter zurück.
14.	ArchiveRetrievalRequest(ReAOID')	Der XML-Adapter ruft die <code>ArchiveRetrieval</code> -Funktion an der TR-S.4 -Schnittstelle des zentralen ArchiSafe-Moduls auf und übergibt die ReAOID'.

Nr.	Aufruf	Beschreibung
15.	ArchiveRetrievalRequest (ReAOID‘)	Das zentrale ArchiSafe-Modul ruft die <code>ArchiveRetrieval</code> -Funktion an der TR-S.5 -Schnittstelle des zentralen ECM/Langzeitspeichers auf und übergibt die <code>ReAOID</code> ‘.
16.	ObtainLXAIP (ReAOID‘) → ReXAIP+‘	Der zentrale ECM/Langzeitspeicher ermittelt die zugehörige <code>ReXAIP+</code> ‘.
17.	Return (ReXAIP+‘)	Der zentrale ECM/Langzeitspeicher gibt das ermittelte <code>ReXAIP+</code> ‘ an das zentrale ArchiSafe-Modul zurück.
18.	Return (ReXAIP+‘)	Das zentrale ArchiSafe-Modul gibt das ermittelte <code>ReXAIP+</code> ‘ an den XML-Adapter zurück.
19.	GetDocReferences (LXAIP++‘) → DLRef, DLRef‘	Der XML-Adapter ermittelt alle in <code>LXAIP++</code> ‘ vorkommenden Referenzen auf lokal gespeicherte Klartextdokumente. Das sind <code>DLRef</code> aus der Version <code>V001</code> und <code>DLRef</code> ‘ aus der Version <code>V002</code> .
20.	ArchiveDeleteRequest (AOID)	Der XML-Adapter ruft die <code>ArchiveDelete</code> -Funktion an der TR-S.4 -Schnittstelle des zentralen ArchiSafe-Moduls auf und übergibt die <code>AOID</code> . Ab hier wird der Standardablauf angewandt, vgl. [TR-ESOR] , Kap. 7.5.5, Schritt 2.
21.	Return (OK)	Das zentrale ArchiSafe-Modul bestätigt die erfolgreiche Löschung des D-AIPs mit der <code>AOID</code> <code>AOID</code> . Es wurden alle Versionen des D-AIPs vernichtet.
22.	ArchiveDeletionRequest (ReAOID)	Der XML-Adapter ruft die <code>ArchiveDeletion</code> -Funktion an der TR-S.4 -Schnittstelle des zentralen ArchiSafe-Moduls auf und übergibt die <code>ReAOID</code> . Ab hier wird der Standardablauf angewandt, vgl. [TR-ESOR] , Kap. 7.5.5, Schritt 2.
23.	Return (OK)	Das zentrale ArchiSafe-Modul bestätigt die erfolgreiche Löschung des Z-AIP mit der <code>AOID</code> <code>ReAOID</code> .
24.	ArchiveDeletionRequest (ReAOID‘)	Der XML-Adapter ruft die <code>ArchiveDeletion</code> -Funktion an der TR-S.4 -Schnittstelle des zentralen ArchiSafe-Moduls auf und übergibt die <code>ReAOID</code> ‘. Ab hier wird der Standardablauf angewandt, vgl. [TR-ESOR] , Kap. 7.5.5, Schritt 2.
25.	Return (OK)	Das zentrale ArchiSafe-Modul bestätigt die erfolgreiche Löschung des Z-AIP mit der <code>AOID</code> <code>ReAOID</code> ‘.
26.	Delete (DLRef)	Der XML-Adapter ruft die <code>Delete</code> -Funktion des lokalen Up/Download-Moduls auf und übergibt die Referenz <code>DLRef</code> auf ein lokal gespeichertes Klartextdokument <code>D</code> .
27.	ArchiveDeletionRequest (DLRef)	Das lokale Up/Download-Modul ruft die <code>ArchiveDeletion</code> -Funktion an der TR-S.5 ‘-Schnittstelle des lokalen ECM/Langzeitspeichers auf und übergibt die Referenz <code>DLRef</code> auf ein lokal gespeichertes Klartextdokument <code>D</code> .

Nr.	Aufruf	Beschreibung
28.	ResolveReference (DLRef) → D	Der lokale ECM/Langzeitspeicher löst die Referenz <code>DLRef</code> auf und ermittelt das zugehörige Klartextdokument <code>D</code> im Speicher.
29.	Delete (D) → OK	Der lokale ECM/Langzeitspeicher löscht das Klartextdokument <code>D</code> erfolgreich aus dem Speicher.
30.	Return (OK)	Der lokale ECM/Langzeitspeicher bestätigt gegenüber dem lokalen Up/Download-Modul die erfolgreiche Löschung der referenzierten Klartextdokument <code>D</code> .
31.	Return (OK)	Das lokale Up/Download-Modul bestätigt gegenüber dem XML-Adapter die erfolgreiche Löschung der referenzierten Klartextdokument <code>D</code> .
32.	Delete (DLRef')	Der XML-Adapter ruft die <code>Delete</code> -Funktion des lokalen Up/Download-Moduls auf und übergibt die Referenz <code>DLRef'</code> auf ein lokal gespeichertes Klartextdokument <code>D'</code> .
33.	ArchiveDeletionRequest (DLRef')	Das lokale Up/Download-Modul ruft die <code>ArchiveDeletion</code> -Funktion an der TR-S.5' -Schnittstelle des lokalen ECM/Langzeitspeichers auf und übergibt die Referenz <code>DLRef'</code> auf ein lokal gespeichertes Klartextdokument <code>D'</code> .
34.	ResolveReference (DLRef') → D'	Der lokale ECM/Langzeitspeicher löst die Referenz <code>DLRef'</code> auf und ermittelt das zugehörige Klartextdokument <code>D'</code> im Speicher.
35.	Delete (D') → OK	Der lokale ECM/Langzeitspeicher löscht das Klartextdokument <code>D'</code> erfolgreich aus dem Speicher.
36.	Return (OK)	Der lokale ECM/Langzeitspeicher bestätigt gegenüber dem lokalen Up/Download-Modul die erfolgreiche Löschung der referenzierten Klartextdokument <code>D'</code> .
37.	Return (OK)	Das lokale Up/Download-Modul bestätigt gegenüber dem XML-Adapter die erfolgreiche Löschung der referenzierten Klartextdokument <code>D'</code> .
38.	Return (OK)	Alle AIPs und zugehörige Daten (lokal und zentral) wurden erfolgreich gelöscht. Der XML-Adapter bestätigt gegenüber der Applikation die erfolgreiche Vernichtung der Daten.
39.	Return (OK)	Die Applikation bestätigt dem Nutzer die erfolgreiche Vernichtung der Daten.

Tabelle 6: Vernichten der aufbewahrten Daten – Beschreibung zur Abbildung 12.

3.4.7 Erneuerung des Hashbaums

Im Falle der Hashbaumeerneuerung müssen die Hashwerte aller geschützten Daten, sowohl zentral als auch lokal gespeicherten, mit Hilfe eines neuen Hashalgorithmus berechnet werden. Das impliziert, dass bedingt durch den Speicherort der Daten auch die Werteberechnungen zentral bzw. lokal erfolgen müssen.

Der in der Abbildung 13 dargestellte exemplarische Ablauf der Hashwertberechnung bei der Hashbaumerneuerung erweitert den im [TR-ESOR-M.3], Kap. 2.4.4 beschriebenen Prozess.

- A.16** Der 2. Schritt im ursprünglichen Ablauf gem. [TR-ESOR-M.3], Kap. 2.4.4 muss gem. dem Ablauf in der Abbildung 13, für jede bekannte AOID erfolgen. Übrigen Schritte des Ablaufs aus [TR-ESOR-M.3], Kap. 2.4.4, zuvor und nach dem 2. Schritt bleiben unberührt.

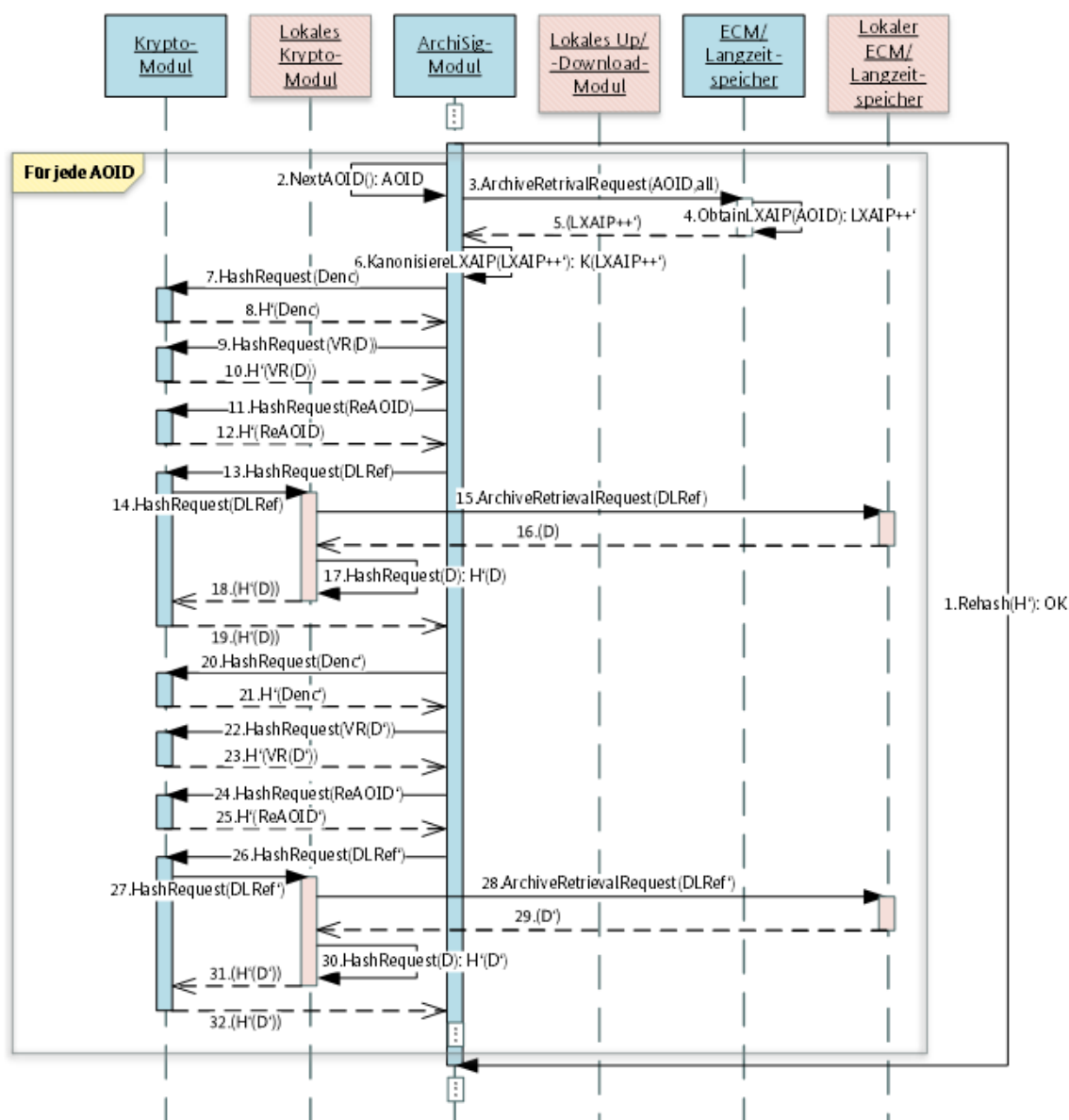


Abbildung 13: Hashwertberechnung bei Hashbaumerneuerung gem. [TR-ESOR-ENC].

Die nachfolgende Tabelle 7 beschreibt die einzelnen Schritte, dargestellt in der Abbildung 13.

Nr.	Aufruf	Beschreibung
1.	Rehash(H') → OK	Die Rehash-Funktion des zentralen ArchiSig-Moduls wird initiiert, um mit Hilfe des vorgegebenen Hashalgorithmus H' den gesamten gespeicherten Hashbaum zu erneuern.
2.	NextAOID() → AOID	Die NextAOID-Funktion des zentralen ArchiSig-Moduls liefert die AOID des nächsten (L)XAIP, hier AOID, um dessen Hashwerte zu erneuern.

Nr.	Aufruf	Beschreibung
3.	ArchiveRetrievalRequest (AOID, all)	Das zentrale ArchiSig-Modul ruft die <code>ArchiveRetrieval</code> -Funktion an der TR-S.2 -Schnittstelle vom ECM/Langzeitspeicher auf und übergibt die <code>AOID</code> . Es werden alle Versionen angefordert.
4.	ObtainLXAIP (AOID) → LXAIP++‘	Der ECM/Langzeitspeicher ermittelt intern das zu der übergebenen <code>AOID</code> <code>AOID</code> zugehörige <code>LXAIP++‘</code> , inkl. aller Versionen <code>V001</code> und <code>V002</code> .
5.	Return (LXAIP++)	Der ECM/Langzeitspeicher liefert das ermittelte <code>LXAIP++‘</code> an das zentrale ArchiSig-Modul zurück.
6.	KanonisiereLXAIP (LXAIP++‘) → K(LXAIP++‘)	Das zentrale ArchiSig-Modul kanonisiert das <code>LXAIP++‘</code> . Es entsteht ein <code>K(LXAIP++‘)</code> .
7.	HashRequest (Denc)	Das zentrale ArchiSig-Modul ruft die <code>Hash</code> -Funktion an der TR-S.3 -Schnittstelle des zentralen Krypto-Moduls auf und übergibt das verschlüsselte Dokument <code>Denc</code> .
8.	Return (H‘(Denc))	Das zentrale Krypto-Modul liefert den Hashwert <code>H‘(Denc)</code> über das verschlüsselte Element <code>Denc</code> zurück.
9.	HashRequest (VR(D))	Das zentrale ArchiSig-Modul ruft die <code>Hash</code> -Funktion an der TR-S.3 -Schnittstelle des zentralen Krypto-Moduls auf und übergibt das Verifikationsprotokoll <code>VR(D)</code> über das Klardokument <code>D</code> .
10.	Return (H‘(VR(D)))	Das zentrale Krypto-Modul liefert den Hashwert über das Verifikationsprotokoll <code>VR(D)</code> vom Klartextdokument <code>D</code> , <code>H‘(VR(D))</code> , zurück.
11.	HashRequest (ReAOID)	Das zentrale ArchiSig-Modul ruft die <code>Hash</code> -Funktion an der TR-S.3 -Schnittstelle des zentralen Krypto-Moduls auf und übergibt die <code>ReAOID</code> , vom zuvor abgelegten <code>ReXAIP</code> , das die Zugriffsberechtigung auf die verschlüsselten Inhalte hält.
12.	Return (H‘(ReAOID))	Das zentrale Krypto-Modul liefert den Hashwert <code>H‘(ReAOID)</code> über die <code>ReAOID</code> , vom zuvor abgelegten <code>ReXAIP</code> , welches die Zugriffsberechtigung auf die verschlüsselten Inhalte hält.
13.	HashRequest (DLRef)	Das zentrale ArchiSig-Modul ruft die <code>Hash</code> -Funktion an der TR-S.3 -Schnittstelle des zentralen Krypto-Moduls auf und übergibt die Referenz, <code>DLRef</code> , auf das zuvor lokal abgelegte Klartextdokument <code>D</code> .
14.	HashRequest (DLRef)	Das zentrale Krypto-Modul ruft die <code>Hash</code> -Funktion an der TR-S.3’ -Schnittstelle des lokalen Krypto-Moduls auf und übergibt die Referenz, <code>DLRef</code> , auf das zuvor lokal abgelegte Klartextdokument <code>D</code> .

Nr.	Aufruf	Beschreibung
15.	ArchiveRetrievalRequest (DLRef)	Das lokale Krypto-Modul ruft die <code>ArchiveRetrieval</code> -Funktion an der TR-S.5' -Schnittstelle des lokalen ECM/Langzeitspeichers auf und übergibt die Referenz, <code>DLRef</code> , auf das zuvor lokal abgelegte Klartextdokument <code>D</code> .
16.	Return (D)	Der lokale ECM/Langzeitspeicher liefert das Klartextdokument <code>D</code> an das lokale Krypto-Modul zurück.
17.	HashRequest (D) → H'(D)	Das lokale Krypto-Modul ruft die eigene <code>Hash</code> -Funktion auf und berechnet den Hashwert <code>H'(D)</code> über das Klartextdokument <code>D</code> .
18.	Return (H'(D))	Das lokale Krypto-Modul liefert den Hashwert <code>H'(D)</code> an das zentrale Krypto-Modul der zentralen Middleware zurück.
19.	Return (H'(D))	Das zentrale Krypto-Modul liefert den Hashwert <code>H'(D)</code> an das zentrale ArchiSig-Modul zurück.
20.	HashRequest (Denc')	Das zentrale ArchiSig-Modul ruft die <code>Hash</code> -Funktion an der TR-S.3 -Schnittstelle des zentralen Krypto-Moduls auf und übergibt das verschlüsselte Dokument <code>Denc'</code> .
21.	Return (H'(Denc'))	Das zentrale Krypto-Modul liefert den Hashwert <code>H'(Denc')</code> über das verschlüsselte Element <code>Denc'</code> zurück.
22.	HashRequest (VR(D'))	Das zentrale ArchiSig-Modul ruft die <code>Hash</code> -Funktion an der TR-S.3 -Schnittstelle des Krypto-Moduls auf und übergibt das Verifikationsprotokoll <code>VR(D')</code> über das Klardokument <code>D'</code> .
23.	Return (H'(VR(D')))	Das zentrale Krypto-Modul liefert den Hashwert <code>H'(VR(D'))</code> über das Verifikationsprotokoll <code>(VR(D'))</code> vom Klartextdokument <code>D'</code> zurück.
24.	HashRequest (ReAOID')	Das zentrale ArchiSig-Modul ruft die <code>Hash</code> -Funktion an der TR-S.3 -Schnittstelle des zentralen Krypto-Moduls auf und übergibt die <code>ReAOID'</code> , vom zuvor abgelegten <code>ReXAIP'</code> , das die Zugriffsberechtigung auf die verschlüsselten Inhalte hält.
25.	Return (H'(ReAOID'))	Das zentrale Krypto-Modul liefert den Hashwert <code>H'(ReAOID')</code> über die <code>ReAOID'</code> , vom zuvor abgelegten <code>ReXAIP'</code> , welches die Zugriffsberechtigung auf die verschlüsselten Inhalte hält.
26.	HashRequest (DLRef')	Das zentrale ArchiSig-Modul ruft die <code>Hash</code> -Funktion an der TR-S.3 -Schnittstelle des zentralen Krypto-Moduls auf und übergibt die Referenz, <code>DLRef'</code> , auf das zuvor lokal abgelegte Klartextdokument <code>D'</code> .
27.	HashRequest (DLRef')	Das zentrale Krypto-Modul ruft die <code>Hash</code> -Funktion an der TR-S.3' -Schnittstelle des lokalen Krypto-Moduls auf und übergibt die Referenz, <code>DLRef'</code> , auf das zuvor lokal abgelegte Klartextdokument <code>D'</code> .

Nr.	Aufruf	Beschreibung
28.	ArchiveRetrievalRequest (DLRef')	Das lokale Krypto-Modul ruft die <code>ArchiveRetrieval</code> -Funktion an der TR-S.5' -Schnittstelle des lokalen ECM/Langzeitspeichers auf und übergibt die Referenz, DLRef', auf das zuvor lokal abgelegte Klartextdokument D'.
29.	Return (D')	Der lokale ECM/Langzeitspeicher liefert das Klartextdokument D' an das lokale Krypto-Modul zurück.
30.	HashRequest (D') → H'(D')	Das lokale Krypto-Modul ruft die eigene <code>Hash</code> -Funktion auf und berechnet den Hashwert H'(D') über das Klartextdokument D'.
31.	Return (H'(D'))	Das lokale Krypto-Modul liefert den Hashwert H'(D') an das zentrale Krypto-Modul der zentralen Middleware zurück.
32.	Return (H'(D'))	Das zentrale Krypto-Modul liefert den Hashwert H'(D') an das zentrale ArchiSig-Modul zurück.

Tabelle 7: Hashwertberechnung bei der Hashbaumerneuerung – Beschreibung zur Abbildung 13.

Nach dem erfolgreichen Abschluss des 32. Schritts erfolgt die Weiterverarbeitung der Neuberechneten Hashwerte gem. den Vorgaben aus [TR-ESOR-M.3], Kap. 2.4.4, beginnend mit dem 3. Schritt aus dem dortigen Prozess.

3.4.8 Erneuerung des Schlüsselmaterials

Im Laufe der Zeit verlieren kryptographische Algorithmen ihre Sicherheitseignung und müssen erneuert werden. Die Überwachung und Erneuerung der kryptographischen Algorithmen für die Hashbaumbildung und Archivzeitstempel werden durch die TR-ESOR-Middleware übernommen – das ist eine Kernfunktion der Middleware. Die bei der Zugriffsteuerung (Verschlüsselung) verwendeten symmetrischen und asymmetrischen Schlüssel werden durch die Middleware nicht überwacht. Aus dem Grund muss die führende Applikation diese Aufgabe übernehmen, vgl. hierzu auch die Anforderungen A.5 und A.6.

Nachfolgende Kapitel gehen auf diese beiden Aspekte näher ein.

3.4.8.1 Erneuerung der symmetrischen Verschlüsselungsschlüssel

Gem. der Anforderung A.6 muss die eingesetzte symmetrische Kryptographie überwacht werden und, bevor die eingesetzten Algorithmen ihre Sicherheitseignung verlieren, entsprechend erneuert werden.

Die Erneuerung der symmetrischen Schlüssel besteht aus den folgenden groben Etappen:

- Die Klartextdokumente müssen mit neuen sicherheitsgeeigneten symmetrischen Schlüsseln verschlüsselt werden,
- Die betroffenen D-AIPs müssen in neue D-AIPs migriert und die ursprüngliche D-AIPs müssen im Nachgang vernichtet werden¹⁶, die neuen D-AIPs müssen zentral abgelegt werden,
- Die korrespondierenden Z-AIPs müssen neu erstellt und zentral abgelegt werden, die ursprünglichen Z-AIPs müssen vernichtet werden.

¹⁶ Gem. [TR-ESOR] kann ein Dokument nur logisch in einer neuen Version ausgeblendet werden, bleibt jedoch nach wie vor in der vorherigen Version bestehen und kann somit abgerufen werden. Da die Kryptographie schwach geworden ist, kann ggf. diese gebrochen werden und der Zugriffsschutz umgegangen werden.

Eine Migration eines vorhandenen D-AIP in ein neues D-AIP* muss entsprechend vorbereitet werden. Da das verschlüsselte Dokument `Denc` wegen der Schwäche des verwendeten Verschlüsselungsalgorithmus nicht migriert werden darf, würde dieses Datenobjekt für die Validierung der aus dem zugehörigen D-AIP referenzierten gesamten Datenobjektgruppe fehlen. Aus diesem Grund müssen die einzelnen Inhalte aus dem ursprünglichen D-AIP in das neue D-AIP* abgelegt und der zugehörige Evidence Record darf nicht die gesamte Gruppe, also `VersionManifest`-Element im ursprünglichen D-AIP, ansprechen, sondern muss die migrierten Elemente einzeln referenzieren. Eine beispielhafte Migration eines `D-AIP-001.xml` in das `D-AIP-001*.xml` kann der Abbildung 14 entnommen werden.

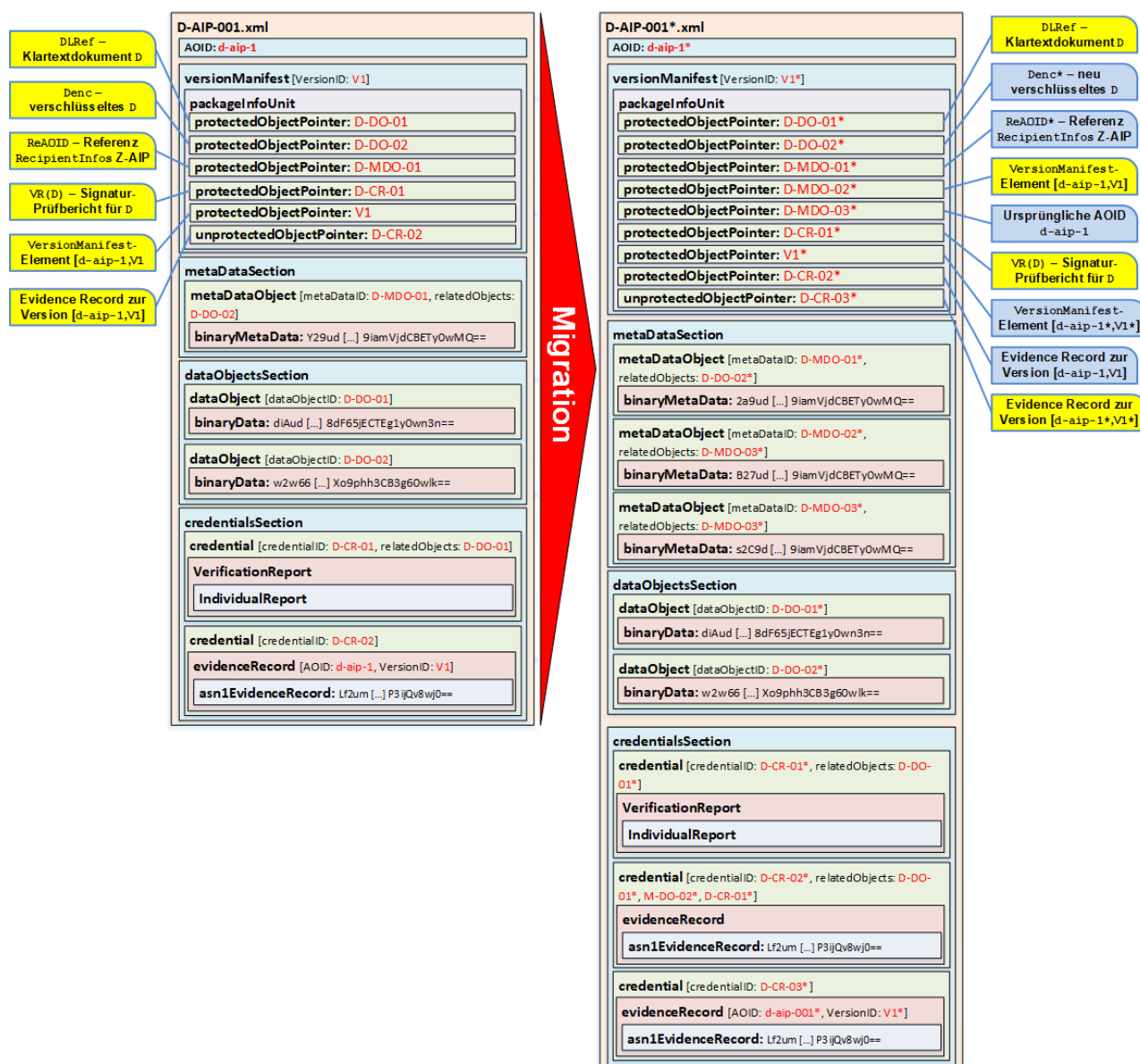


Abbildung 14: Migration eines D-AIP im Zuge der Erneuerung der symmetrischen Schlüssel - Beispiel.

Das `D-AIP-001.xml` (linke Seite in der Abbildung 14) beinhaltet fünf `protectedObjectPointer`-Elemente:

- `D-DO-01` – referenziert ein `dataObject`-Element, das die Referenz `DLRef` auf das lokal gespeicherte Klartextdokument `D` beinhaltet,
- `D-DO-02` – referenziert ein `dataObject`-Element, das die verschlüsselte Version `Denc` des Klartextdokuments `D` beinhaltet,
- `D-MDO-01` – referenziert ein `metaDataObject`-Element, das die Referenz `ReAOID` auf das zugehörige Z-AIP enthält,
- `D-CR-01` – referenziert ein `credential`-Element, das den Signaturprüfbericht `VR(D)` für Klartextdokument `D` enthält,

- $v1^{17}$ – referenziert das `VersionManifest`-Element in der Version $v1$ in diesem D-AIP.

Darüber hinaus enthält das `D-AIP-001.xml` ein `unprotectedObjectPointer`-Element:

- `D-CR-02` – referenziert ein `credential`-Element, das den Evidence Record zur Version $v1$ dieses D-AIP enthält.

Nach der Migration enthält das `D-AIP-001*.xml` (rechte Seite in der Abbildung 14) bereits acht `protectedObjectPointer`-Elemente:

- `D-DO-01*` – **[migriert]** – referenziert ein `dataObject`-Element, das die Referenz `DLRef` auf das lokal gespeicherte Klartextdokument D beinhaltet,
- `D-DO-02*` – **[neu]** – referenziert ein `dataObject`-Element, das die verschlüsselte Version `Denc*` des Klartextdokuments D beinhaltet,
- `D-MDO-01*` – **[neu]** – referenziert ein `metaDataObject`-Element, das die Referenz `ReAOID*` auf das zugehörige Z-AIP enthält,
- `D-MDO-02*` – **[migriert]** – referenziert das `VersionManifest`-Element in der Version $v1$ aus dem ursprünglichen `D-AIP-001.xml` mit AOID `d-aip-1`,
- `D-MDO-03*` – **[neu]** – referenziert das `AOID`-Element des ursprünglichen `D-AIP-001.xml`,
- `D-CR-01*` – **[migriert]** – referenziert ein `credential`-Element, das den Signaturprüfbericht $VR(D)$ für Klartextdokument D enthält,
- $v1*$ – **[neu]** – referenziert das `VersionManifest`-Element in der Version $v1*$ in diesem D-AIP,
- `D-CR-02*` – **[migriert]** – referenziert ein `credential`-Element, das den Evidence Record zur Version $v1$ des ursprünglichen `D-AIP-001.xml` enthält. Das `relatedObjects`-Attribut verweist auf die Elemente mit entsprechenden IDs `D-DO-01*`, `M-DO-02*`, `D-CR-01*`, was bedeutet, dass jedes der besagten Elemente durch den Evidence Record geschützt ist. Die Prüfung muss aber einzeln erfolgen, da aus der ursprünglichen Datenobjektgruppe in `D-AIP-001.xml` die beiden Elemente mit ID `D-DO-02` und `D-MDO-01` nicht mehr vorhanden sind.

Darüber hinaus enthält das `D-AIP-001*.xml` ein `unprotectedObjectPointer`-Element:

- `D-CR-03*` – referenziert ein `credential`-Element, das den Evidence Record zur Version $v1*$ dieses D-AIP enthält.

In dem o. g. Migrationsbeispiel verfügt die ursprüngliche `D-AIP-001.xml` lediglich über eine Version $v1$. Im Falle, dass mehr als eine Version vorhanden ist, muss die Migration für jede Version entsprechend dem o. g. Muster erfolgen.

Die nachfolgende Abbildung 15 stellt einen beispielhaften Ablauf einer Erneuerung der symmetrischen Schlüssel dar.

¹⁷ Die Versionswerte $v1$ und $v001$ sowie $v2$ und $v002$ sind als Synonyme zu verstehen.

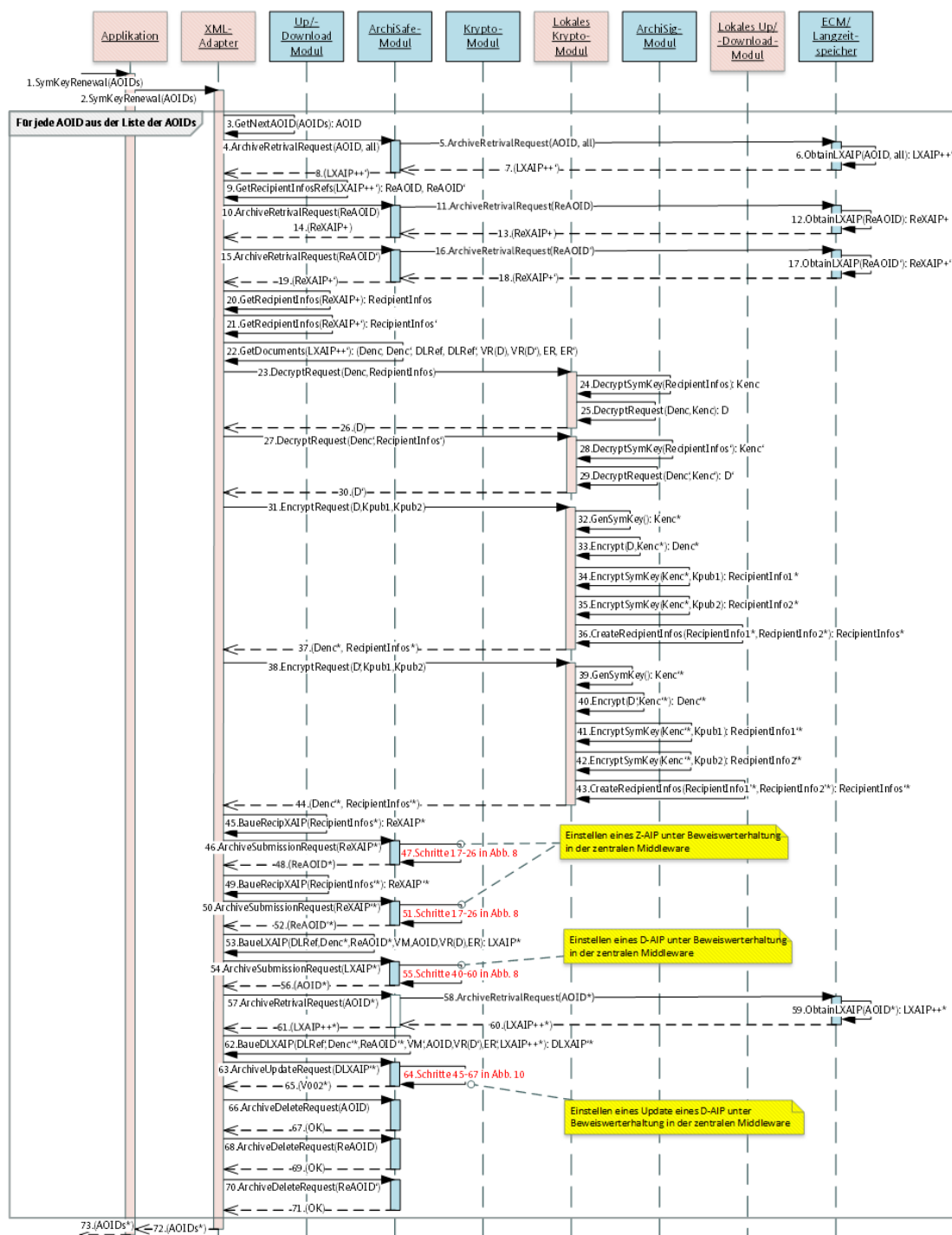


Abbildung 15: Erneuerung der symmetrischen Schlüssel gem. [TR-ESOR-ENC].

Die nachfolgende Tabelle 8 beschreibt die einzelnen Schritte, dargestellt in der Abbildung 15.

Nr.	Aufruf	Beschreibung
1.	SymKeyRenewal(AOIDs)	Das Überwachungssystem initiiert die Erneuerung der symmetrischen Schlüssel für alle betroffenen D-AIPs. Eine Liste der AOIDs $AOIDs$ wird an die Applikation übergeben.
2.	SymKeyRenewal(AOIDs)	Die Applikation initiiert die Erneuerung der symmetrischen Schlüssel für alle betroffenen D-AIPs. Eine Liste der AOIDs $AOIDs$ wird an den XML-Adapter übergeben.

Nr.	Aufruf	Beschreibung
3.	GetNextAOID (AOIDs) → AOID	Der XML-Adapter ermittelt eine AOID aus der übergebenen Liste der AOIDs.
4.	ArchiveRetrievalRequest (AOID, all)	Der XML-Adapter ruft die ArchiveRetrieval-Funktion an der TR-S.4 -Schnittstelle des zentralen ArchiSafe-Moduls auf und übergibt die AOID. Um alle Versionen anzufordern, wird das VersionID-Element übergeben, mit dem Wert all.
5.	ArchiveRetrievalRequest (AOID, all)	Das zentrale ArchiSafe-Modul ruft die ArchiveRetrieval-Funktion an der TR-S.5 -Schnittstelle des zentralen ECM/Langzeitspeichers auf und übergibt die AOID. Um alle Versionen anzufordern wird das VersionID-Element übergeben, mit dem Wert all.
6.	ObtainLXAIP (AOID, all) → LXAIP++‘	Der zentrale ECM/Langzeitspeicher ermittelt im Speicher das zur AOID zugehörige D-AIP, nämlich LXAIP++ ‘, inkl. alle Versionen.
7.	Return (LXAIP++‘)	Der zentrale ECM/Langzeitspeicher liefert das LXAIP++ ‘ an das zentrale ArchiSafe-Modul zurück.
8.	Return (LXAIP++‘)	Das zentrale ArchiSafe-Modul liefert das LXAIP++ ‘ an den XML-Adapter zurück.
9.	GetRecipientInfosRefs (LXAIP++‘) → ReAOID, ReAOID‘	Der XML-Adapter ermittelt die Referenzen auf zugehörige Z-AIPs aus dem LXAIP++ ‘. In diesem Fall werden einzelne Referenzen je Version ermittelt: ReAOID für Version V001 und ReAOID ‘ für Version V002.
10.	ArchiveRetrievalRequest (ReAOID)	Der XML-Adapter ruft die ArchiveRetrieval-Funktion an der TR-S.4 -Schnittstelle des zentralen ArchiSafe-Moduls auf und übergibt die ReAOID.
11.	ArchiveRetrievalRequest (ReAOID)	Das zentrale ArchiSafe-Modul ruft die ArchiveRetrieval-Funktion an der TR-S.5 -Schnittstelle des zentralen ECM/Langzeitspeichers auf und übergibt die ReAOID.
12.	ObtainLXAIP (ReAOID) → ReXAIP+	Der zentrale ECM/Langzeitspeicher ermittelt im Speicher das zur ReAOID zugehörige D-AIP, nämlich ReXAIP+.
13.	Return (ReXAIP+)	Der zentrale ECM/Langzeitspeicher liefert das ReXAIP+ an das zentrale ArchiSafe-Modul zurück.
14.	Return (ReXAIP+)	Das zentrale ArchiSafe-Modul liefert das ReXAIP+ an den XML-Adapter zurück.
15.	ArchiveRetrievalRequest (ReAOID‘)	Der XML-Adapter ruft die ArchiveRetrieval-Funktion an der TR-S.4 -Schnittstelle des zentralen ArchiSafe-Moduls auf und übergibt die ReAOID ‘.
16.	ArchiveRetrievalRequest (ReAOID‘)	Das zentrale ArchiSafe-Modul ruft die ArchiveRetrieval-Funktion an der TR-S.5 -Schnittstelle des zentralen ECM/Langzeitspeichers auf und übergibt die ReAOID ‘.

Nr.	Aufruf	Beschreibung
17.	ObtainLXAIP (ReAOID') → ReXAIP+'	Der zentrale ECM/Langzeitspeicher ermittelt im Speicher das zur ReAOID' zugehörige D-AIP, nämlich ReXAIP+'.
18.	Return (ReXAIP+')	Der zentrale ECM/Langzeitspeicher liefert das ReXAIP+' an das zentrale ArchiSafe-Modul zurück.
19.	Return (ReXAIP+')	Das zentrale ArchiSafe-Modul liefert das ReXAIP+' an den XML-Adapter zurück.
20.	GetRecipientInfos (ReXAIP+) → RecipientInfos	Der XML-Adapter ermittelt aus dem vorliegenden ReXAIP+ das korrespondierende RecipientInfos-Element: RecipientInfos.
21.	GetRecipientInfos (ReXAIP+') → RecipientInfos'	Der XML-Adapter ermittelt aus dem vorliegenden ReXAIP+' das korrespondierende RecipientInfos-Element: RecipientInfos'.
22.	GetDocuments (LXAIP++') → (Denc, Denc', DLRef, DLRef', VR(D), VR(D'), ER, ER')	Der XML-Adapter ermittelt aus dem vorliegenden LXAIP++' die für die Migration relevanten Daten:
23.	DecryptRequest (Denc, RecipientInfos)	Der XML-Adapter ruft die Decrypt-Funktion an der TR-S.3' -Schnittstelle des lokalen Krypto-Moduls auf und übergibt das verschlüsselte Klartextdokument D sowie das korrespondierende RecipientInfos-Element: RecipientInfos.
24.	DecryptSymKey (RecipientInfos) → Kenc	Das lokale Krypto-Modul entschlüsselt den symmetrischen Schlüssel Kenc aus dem RecipientInfos-Element.
25.	DecryptRequest (Denc, Kenc) → D	Das lokale Krypto-Modul entschlüsselt das verschlüsselte Klartextdokument D.
26.	Return (D)	Das lokale Krypto-Modul liefert das entschlüsselte Klartextdokument D an den XML-Adapter zurück.
27.	DecryptRequest (Denc', RecipientInfos')	Der XML-Adapter ruft die Decrypt-Funktion an der TR-S.3' -Schnittstelle des lokalen Krypto-Moduls auf und übergibt das verschlüsselte Klartextdokument D', sowie das korrespondierende RecipientInfos-Element: RecipientInfos'.
28.	DecryptSymKey (RecipientInfos') → Kenc'	Das lokale Krypto-Modul entschlüsselt den symmetrischen Schlüssel Kenc' aus dem RecipientInfos'-Element.
29.	DecryptRequest (Denc', Kenc') → D'	Das lokale Krypto-Modul entschlüsselt das verschlüsselte Klartextdokument D' (Update des Klartextdokuments D).
30.	Return (D')	Das lokale Krypto-Modul liefert das entschlüsselte Klartextdokument D' an den XML-Adapter zurück.

Nr.	Aufruf	Beschreibung
31.	EncryptRequest (D, Kpub1, Kpub2)	Der XML-Adapter steuert das lokale Krypto-Modul an, um die hybride Verschlüsselung des Dokuments via die TR-S.3' -Schnittstelle zu initiieren. Es werden das Klartextdokument D und zwei öffentliche Schlüssel (Zertifikat) der berechtigten Nutzer Kpub1 und Kpub2 übergeben.
32.	GenSymKey () → Kenc*	Das lokale Krypto-Modul leitet einen symmetrischen Schlüssel für die Verschlüsselung ab. Ausgabe Kenc* – symmetrische Verschlüsselungsschlüssel.
33.	Encrypt (D, Kenc*) → Denc*	Das lokale Krypto-Modul verschlüsselt D mit dem generierten Schlüssel Kenc*. Das Ergebnis ist ein verschlüsseltes Dokument Denc*.
34.	EncryptSymKey (Kenc*, Kpub1) → RecipientInfo1*	Das lokale Krypto-Modul verschlüsselt den generierten symmetrischen Verschlüsselungsschlüssel Kenc* mit dem asymmetrischen öffentlichen Schlüssel des Nutzers Kpub1. Das Ergebnis ist eine Struktur RecipientInfo1*.
35.	EncryptSymKey (Kenc*, Kpub2) → RecipientInfo2*	Das lokale Krypto-Modul verschlüsselt den generierten symmetrischen Verschlüsselungsschlüssel Kenc* mit dem asymmetrischen öffentlichen Schlüssel des Nutzers Kpub2. Das Ergebnis ist eine Struktur RecipientInfo2*.
36.	CreateRecipientInfos (RecipientInfo1*, RecipientInfo2*) → RecipientInfos*	Das lokale Krypto-Modul legt die beiden erzeugten RecipientInfo1* und RecipientInfo2* in eine gemeinsame Struktur RecipientInfos*, inkl. weiterer Daten, wie z. B. Schlüsselverschlüsselungsalgorithmus.
37.	Return (Denc*, RecipientInfos*)	Als Antwort auf den Schritt 31. liefert das lokale Krypto-Modul das verschlüsselte Dokument D als Denc* und die korrespondierende RecipientInfo-Struktur RecipientInfos* als Zugriffsberechtigung an die Applikation zurück.
38.	EncryptRequest (D', Kpub1, Kpub2)	Der XML-Adapter steuert das lokale Krypto-Modul an, um die hybride Verschlüsselung des Dokuments via die TR-S.3' -Schnittstelle zu initiieren. Es wird das Klartextdokument D' und zwei öffentliche Schlüssel (Zertifikat) der berechtigten Nutzer Kpub1 und Kpub2 übergeben.
39.	GenSymKey () → Kenc'*	Das lokale Krypto-Modul leitet einen symmetrischen Schlüssel für die Verschlüsselung ab. Ausgabe Kenc'* – symmetrische Verschlüsselungsschlüssel.
40.	Encrypt (D', Kenc'*) → Denc'*	Das lokale Krypto-Modul verschlüsselt D' mit dem generierten Schlüssel Kenc'*. Das Ergebnis ist ein verschlüsseltes Dokument Denc'*.

Nr.	Aufruf	Beschreibung
41.	EncryptSymKey (Kenc [*] , Kpub1) → RecipientInfo1 [*]	Das lokale Krypto-Modul verschlüsselt den generierten symmetrischen Verschlüsselungsschlüssel Kenc [*] mit dem asymmetrischen öffentlichen Schlüssel des Nutzers Kpub1. Das Ergebnis ist eine Struktur RecipientInfo1 [*] .
42.	EncryptSymKey (Kenc [*] , Kpub2) → RecipientInfo2 [*]	Das lokale Krypto-Modul verschlüsselt den generierten symmetrischen Verschlüsselungsschlüssel Kenc [*] mit dem asymmetrischen öffentlichen Schlüssel des Nutzers Kpub2. Das Ergebnis ist eine Struktur RecipientInfo2 [*] .
43.	CreateRecipientInfos (RecipientInfo1 [*] , RecipientInfo2 [*]) → RecipientInfos [*]	Das lokale Krypto-Modul legt die beiden erzeugten RecipientInfo1 [*] und RecipientInfo2 [*] in eine gemeinsame Struktur RecipientInfos [*] , inkl. weiterer Daten, wie z. B. Schlüsselverschlüsselungsalgorithmus.
44.	Return (Denc [*] , RecipientInfos [*])	Antwort auf den Schritt 38.: Das lokale Krypto-Modul liefert das verschlüsselte Dokument D [*] als Denc [*] und die korrespondierende RecipientInfo-Struktur RecipientInfos [*] als Zugriffsberechtigung an die Applikation zurück.
45.	BaueRecipXAIP (RecipientInfos [*]) → ReXAIP [*]	Der XML-Adapter erzeugt ein Z-AIP (ReXAIP [*]) und legt darin die erhaltene RecipientInfo-Struktur RecipientInfos [*] ab.
46.	ArchiveSubmissionRequest (ReXAIP [*])	Der XML-Adapter ruft die ArchiveSubmission-Funktion an der TR-S.4 -Schnittstelle des zentralen ArchiSafe-Moduls auf und übergibt das im 45. Schritt erzeugte ReXAIP [*] .
47.	<i>Schritte 17. – 26., in Abbildung 8</i>	Das ReXAIP [*] wird im zentralen ECM/Langzeitspeicher abgelegt und eine AOID, ReAOID [*] wird zurückgeliefert. Es werden dafür die Schritte 17. bis 26. in der Abbildung 8 durchlaufen, in den das Z-AIP ReXAIP [*] unter der Beweiswerterhaltung im zentralen ArchiSig-Modul gestellt wird.
48.	Return (ReAOID [*])	Das zentrale ArchiSafe-Modul liefert die korrespondierende AOID, ReAOID [*] an den XML-Adapter zurück.
49.	BaueRecipXAIP (RecipientInfos [*]) → ReXAIP [*]	Der XML-Adapter erzeugt ein Z-AIP (ReXAIP [*]) und legt darin die erhaltene RecipientInfo-Struktur RecipientInfos [*] ab.
50.	ArchiveSubmissionRequest (ReXAIP [*])	Der XML-Adapter ruft die ArchiveSubmission-Funktion an der TR-S.4 -Schnittstelle des zentralen ArchiSafe-Moduls auf und übergibt das im 49. Schritt erzeugte ReXAIP [*] .
51.	<i>Schritte 17. – 26., in Abbildung 8</i>	Das ReXAIP [*] wird im zentralen ECM/Langzeitspeicher abgelegt und eine AOID, ReAOID [*] wird zurückgeliefert. Es werden dafür die Schritte 17. bis 26. in der Abbildung 8 durchlaufen.

Nr.	Aufruf	Beschreibung
52.	Return (ReAOID*)	Das zentrale ArchiSafe-Modul liefert die korrespondierende AOID, ReAOID`* an den XML-Adapter zurück.
53.	BaueLXAIP (DLRef, Denc*, ReAOID*, VM, AOID, VR(D), ER) → LXAIP*	Der XML-Adapter baut ein LXAIP* zusammen, welches die Referenz auf das lokal gespeicherte Klartextdokument D (DLRef), das verschlüsselte Klartextdokument Denc*, die Referenz auf die zuvor abgelegte Z-AIP mit den Zugriffsberechtigungen (ReXAIP*), nämlich ReAOID*, VersionManifest-Element VM aus dem D-AIP LXAIP++, AOID des D-AIP LXAIP++, AOID, Signaturprüfbericht zum Klartextdokument D aus dem D-AIP LXAIP++ und Evidence Record ER zur Version V001 aus dem D-AIP LXAIP++ beinhaltet.
54.	ArchiveSubmissionRequest (LXAIP*)	Der XML-Adapter ruft die ArchiveSubmission-Funktion an der TR-S.4 -Schnittstelle des zentralen ArchiSafe-Moduls auf und übergibt das im 53. Schritt erzeugte LXAIP*.
55.	<i>Schritte 40. – 60., in Abbildung 8</i>	Das LXAIP* wird im zentralen ECM/Langzeitspeicher abgelegt und eine AOID, AOID* wird zurückgeliefert. Es werden dafür die Schritte 40. ¹⁸ bis 60. in der Abbildung 8 durchlaufen.
56.	Return (AOID*)	Das zentrale ArchiSafe-Modul liefert die korrespondierende AOID, ReAOID`* an den XML-Adapter zurück.
57.	ArchiveRetrivalRequest (AOID*)	Der XML-Adapter ruft die ArchiveRetrieval-Funktion an der TR-S.4 -Schnittstelle des zentralen ArchiSafe-Moduls auf und übergibt die AOID*.
58.	ArchiveRetrivalRequest (AOID*)	Das zentrale ArchiSafe-Modul ruft die ArchiveRetrieval-Funktion an der TR-S.5 -Schnittstelle des zentralen ECM/Langzeitspeichers auf und übergibt die AOID*.
59.	ObtainLXAIP (AOID*) → LXAIP++*	Der zentrale ECM/Langzeitspeicher ermittelt das zur AOID* zugehörige D-AIP LXAIP+++ in der Version V001*.
60.	Return (LXAIP++*)	Der zentrale ECM/Langzeitspeicher liefert das im 59. Schritt ermittelte LXAIP+++ an das zentrale ArchiSafe-Modul zurück.
61.	Return (LXAIP++*)	Das zentrale ArchiSafe-Modul liefert das LXAIP+++ an den XML-Adapter zurück.

¹⁸ Hinweis: Die Validierung der Signaturen in D ist nicht erforderlich, da diese Schritte bereits bei der ersten Ablage erfolgreich ausgeführt worden sind. Der zugehörige Prüfbericht liegt vor und ist bereits durch die technischen Beweisdaten geschützt.

Nr.	Aufruf	Beschreibung
62.	BaueDLXAIP (DLRef', Denc', ReAOID', - VM', AOID, VR(D'), ER', LXAIP++) → DLXAIP'	Der XML-Adapter erstellt, basierend auf LXAIP++, ein DLXAIP' für die Version V002* und legt darin folgende Objekte ab: • DLRef' – lokale Referenz auf Klartextdokument D', • Denc' – verschlüsseltes Klartextdokument D', • ReAOID' – Referenz auf die zugehörige Z-AIP, • VM' – VersionManifest-Element vom Version V002 des D-AIP LXAIP++, • AOID – die AOID, AOID, des D-XAIP LXAIP++, • VR(D') – Signaturprüfbericht zum Klartextdokument D', ER' – der Evidence Record zu Version V002 des D-AIP LXAIP++.
63.	ArchiveUpdateRequest (DLXAIP')	Der XML-Adapter ruft die ArchiveUpdate-Funktion an der TR-S.4 -Schnittstelle des zentralen ArchiSafe-Moduls auf und übergibt das im 62. Schritt erstellte DLXAIP'.
64.	<i>Schritte 45. – 67., in Abbildung 10</i>	Das durch AOID* referenzierte LXAIP++ wird um die mit DLXAIP' übergebenen Daten erweitert, im zentralen ECM/Langzeitspeicher als LXAIP++, abgelegt und eine neue Version V002* wird zurückgeliefert. Es werden dafür die Schritte 45. ¹⁹ bis 67. in der Abbildung 10 durchlaufen.
65.	Return (V002*)	Das zentrale ArchiSafe-Modul liefert eine neue Version V002* an den XML-Adapter zurück.
66.	ArchiveDeleteRequest (AOID)	Der XML-Adapter ruft die ArchiveDelete-Funktion an der TR-S.4-Schnittstelle des zentralen ArchiSafe-Moduls auf und übergibt die AOID. Ab hier wird der Standardablauf angewandt, vgl. [TR-ESOR], Kap. 7.5.5, Schritt 2.
67.	Return (OK)	Das zentrale ArchiSafe-Modul bestätigt die erfolgreiche Löschung des D-AIPs mit der AOID AOID. Es wurden alle Versionen des D-AIPs vernichtet.
68.	ArchiveDeleteRequest (ReAOID)	Der XML-Adapter ruft die ArchiveDelete-Funktion an der TR-S.4-Schnittstelle des zentralen ArchiSafe-Moduls auf und übergibt die ReAOID. Ab hier wird der Standardablauf angewandt, vgl. [TR-ESOR], Kap. 7.5.5, Schritt 2.
69.	Return (OK)	Das zentrale ArchiSafe-Modul bestätigt die erfolgreiche Löschung des Z-AIP mit der AOID ReAOID.

¹⁹ Hinweis: Die Validierung der Signaturen in D ist nicht erforderlich, da diese Schritte bereits bei der ersten Aktualisierung erfolgreich ausgeführt worden sind. Der zugehörige Prüfbericht liegt vor und ist bereits durch die technischen Beweisdaten geschützt.

Nr.	Aufruf	Beschreibung
70.	ArchiveDeleteRequest (ReAOID ²⁰)	Der XML-Adapter ruft die <code>ArchiveDelete</code> -Funktion an der TR-S.4 -Schnittstelle des zentralen ArchiSafe-Moduls auf und übergibt die ReAOID ²⁰ . Ab hier wird der Standardablauf angewandt, vgl. [TR-ESOR], Kap. 7.5.5, Schritt 2.
71.	Return (OK)	Das zentrale ArchiSafe-Modul bestätigt die erfolgreiche Löschung des Z-AIP mit der AOID ReAOID ²⁰ .
72.	Return (AOIDs*)	Der XML-Adapter erstellt eine Liste AOIDs* mit dem Bezug zur ursprünglichen auf die neuen AOIDs (z. B. AOID → AOID*) und übergibt diese Liste an die Applikation zurück, um die ursprünglichen AOIDs zu ersetzen.
n73.	Return (AOIDs*)	Die Applikation liefert die im 72. Schritt erstellte Liste AOIDs* an das Überwachungssystem zurück.

Tabelle 8: Erneuerung der symmetrischen Schlüssel - Beschreibung zur Abbildung 15.

3.4.8.2 Erneuerung der asymmetrischen Zugriffsberechtigungsschlüssel

Sollten die Zugriffsregeln auf ein verschlüsseltes Klartextdokument verändert werden, so sind die korrespondierenden Z-AIPs einer D-AIP zu erneuern.

Es gelten folgende zusätzliche Anforderungen für diesen Prozess:

- A.17** Im Zuge der Erneuerung müssen die asymmetrischen Schlüssel aller korrespondierenden Z-AIPs²⁰ entsprechend angepasst werden.
- A.18** Die verwendeten symmetrischen Schlüssel müssen gleichwohl erneuert werden. Siehe hierzu auch Kap. 3.4.8.1.

Die nachfolgende Abbildung 16 illustriert einen beispielhaften Ablauf der Erneuerung der asymmetrischen Schlüssel für ein einzelnes D-AIP mit zwei zugehörigen Z-AIPs.

²⁰ Das referenzierte D-AIP beinhaltet die notwendigen Referenzen (ReAOID) auf die korrespondierenden Z-AIPs.

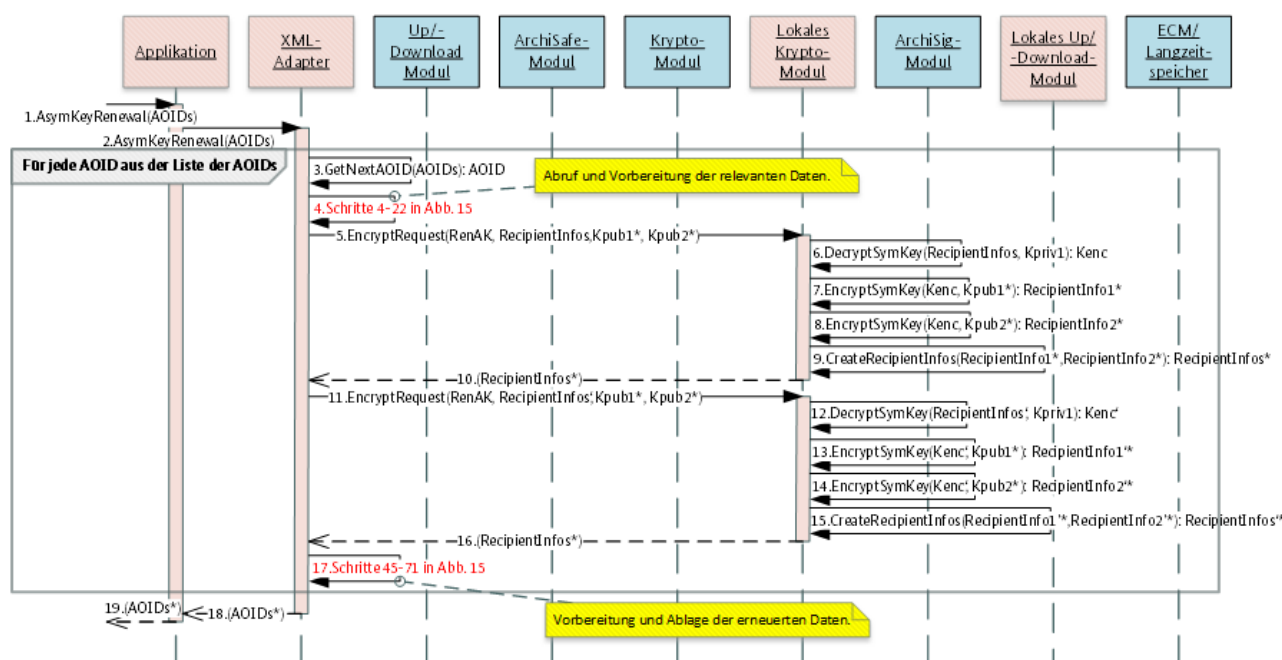


Abbildung 16: Erneuerung der asymmetrischen Schlüssel gem. [TR-ESOR-ENC].

Die nachfolgende Tabelle 9 beschreibt die einzelnen Schritte, dargestellt in der Abbildung 16.

Nr.	Aufruf	Beschreibung
1.	AsymKeyRenewal(AOIDs)	Das Überwachungssystem initiiert die Erneuerung der asymmetrischen Schlüssel für alle betroffenen D-AIPs. Eine Liste der AOIDs $AOIDs$ wird an die Applikation übergeben.
2.	AsymKeyRenewal(AOIDs)	Die Applikation initiiert die Erneuerung der asymmetrischen Schlüssel für alle betroffenen D-AIPs. Eine Liste der AOIDs $AOIDs$ wird an den XML-Adapter übergeben.
3.	GetNextAOID(AOIDs) → AOID	Der XML-Adapter ermittelt eine AOID aus der im 2. Schritt übergebenen Liste $AOIDs$.
4.	<i>Schritte 4. – 22., in Abbildung 15</i>	Es werden die Schritte 4. bis 22. in der Abbildung 15 ausgeführt. Die benötigten Daten werden aus den D-AIP (AOID) und zugehörigen Z-AIPs ($ReAOID$ und $ReAOID'$) ermittelt: • $DLRef$ – Referenz auf das Klartextdokument D, • $Denc$ – verschlüsseltes Klartextdokument D, • $VR(D)$ – Signaturprüfbericht zum D, • ER – Evidence Record über die Version $V001$, • $DLRef'$ – Referenz auf das Klartextdokument D', • $Denc'$ – verschlüsseltes Klartextdokument D', • $VR(D')$ – Signaturprüfbericht zum D', • ER' – Evidence Record über die Version $V002$, • $RecipientInfos$ – Zugriffsberechtigung für Version $V001$, • $RecipientInfos'$ – Zugriffsberechtigung für die Version $V002$.

Nr.	Aufruf	Beschreibung
5.	EncryptRequest (RenAK, RecipientInfos, Kpub1*, Kpub2*)	Der XML-Adapter ruft die <code>Encrypt</code> -Funktion an der TR-S.3' -Schnittstelle des lokalen Krypto-Moduls auf und übergibt das zu erneuernde <code>RecipientInfos</code> -Element sowie die neuen asymmetrischen öffentlichen Schlüssel (Zertifikate) <code>Kpub1*</code> und <code>Kpub2*</code> . Das <code>RenAK</code> -Attribut signalisiert dem lokalen Krypto-Modul gegenüber, dass es sich um die Erneuerung der asymmetrischen Schlüssel handelt.
6.	DecryptSymKey (RecipientInfos, Kpriv1) → Kenc	Das lokale Krypto-Modul entschlüsselt ein <code>RecipientInfo</code> -Element aus dem <code>RecipientInfos</code> -Element mit dem vorhandenen privaten asymmetrischen Schlüssel <code>Kpriv1</code> ²¹ und ermittelt den symmetrischen Schlüssel <code>Kenc</code> .
7.	EncryptSymKey (Kenc, Kpub1*) → RecipientInfo1*	Das lokale Krypto-Modul verschlüsselt den im 6. Schritt ermittelten symmetrischen Schlüssel <code>Kenc</code> mit dem asymmetrischen öffentlichen Schlüssel des Nutzers <code>Kpub1*</code> . Das Ergebnis ist eine Struktur <code>RecipientInfo1*</code> .
8.	EncryptSymKey (Kenc, Kpub2*) → RecipientInfo2*	Das lokale Krypto-Modul verschlüsselt den im 6. Schritt ermittelten symmetrischen Schlüssel <code>Kenc</code> mit dem asymmetrischen öffentlichen Schlüssel des Nutzers <code>Kpub2*</code> . Das Ergebnis ist eine Struktur <code>RecipientInfo2*</code> .
9.	CreateRecipientInfos (RecipientInfo1*, RecipientInfo2*) → RecipientInfos*	Das lokale Krypto-Modul legt die beiden erzeugten <code>RecipientInfo1*</code> und <code>RecipientInfo2*</code> in eine gemeinsame Struktur <code>RecipientInfos*</code> , inkl. weiterer Daten, wie z. B. Schlüsselverschlüsselungsalgorithmus etc..
10.	Return (RecipientInfos*)	Das lokale Krypto-Modul liefert die neu erzeugte Struktur <code>RecipientInfos*</code> an den XML-Adapter zurück.
11.	EncryptRequest (RenAK, RecipientInfos', Kpub1*, Kpub2*)	Der XML-Adapter ruft die <code>Encrypt</code> -Funktion an der TR-S.3' -Schnittstelle des lokalen Krypto-Moduls auf und übergibt das zu erneuernde <code>RecipientInfos'</code> -Element sowie die neuen asymmetrischen öffentlichen Schlüssel (Zertifikate) <code>Kpub1*</code> und <code>Kpub2*</code> . Das <code>RenAK</code> -Attribut signalisiert dem lokalen Krypto-Modul gegenüber, dass es sich um die Erneuerung der asymmetrischen Schlüssel handelt.
12.	DecryptSymKey (RecipientInfos', Kpriv1) → Kenc'	Das lokale Krypto-Modul entschlüsselt ein <code>RecipientInfo'</code> -Element aus dem <code>RecipientInfos'</code> -Element mit dem vorhandenen privaten asymmetrischen Schlüssel <code>Kpriv1</code> ²² und ermittelt den symmetrischen Schlüssel <code>Kenc'</code> .

²¹ Da alle `RecipientInfo`-Elemente den gleichen symmetrischen Schlüssel `Kenc` beinhalten, reicht es aus, wenn nur ein solches Element entschlüsselt wird.

²² Da alle `RecipientInfo'`-Elemente den gleichen symmetrischen Schlüssel `Kenc'` beinhalten, reicht es aus, wenn nur ein solches Element entschlüsselt wird.

Nr.	Aufruf	Beschreibung
13.	EncryptSymKey (Kenc', Kpub1*) → RecipientInfo1'	Das lokale Krypto-Modul verschlüsselt den im 12. Schritt ermittelten symmetrischen Schlüssel Kenc' mit dem asymmetrischen öffentlichen Schlüssel des Nutzers Kpub1*. Das Ergebnis ist eine Struktur RecipientInfo1'.
14.	EncryptSymKey (Kenc', Kpub2*) → RecipientInfo2'	Das lokale Krypto-Modul verschlüsselt den im 12. Schritt ermittelten symmetrischen Schlüssel Kenc' mit dem asymmetrischen öffentlichen Schlüssel des Nutzers Kpub2*. Das Ergebnis ist eine Struktur RecipientInfo2'.
15.	CreateRecipientInfos (RecipientInfo1', RecipientInfo2') → RecipientInfos'	Das lokale Krypto-Modul legt die beiden erzeugten RecipientInfo1' und RecipientInfo2' in eine gemeinsame Struktur RecipientInfos', inkl. weiterer Daten, wie z. B. Schlüsselverschlüsselungsalgorithmus etc..
16.	Return (RecipientInfos')	Das lokale Krypto-Modul liefert die neu erzeugte Struktur RecipientInfos' an den XML-Adapter zurück.
17.	<i>Schritte 45. – 71., in Abbildung 15</i>	Es erfolgt die Migration der Daten in ein neues D-AIP und entsprechend neue Z-AIPs (im Sinne von Kap. 3.4.8.1).
18.	Return (AOIDs*)	Der XML-Adapter erstellt eine Liste AOIDs* mit dem Bezug zur ursprünglichen auf die neuen AOIDs (z. B. AOID → AOID*) und übergibt diese Liste an die Applikation zurück, um die ursprünglichen AOIDs zu ersetzen.
19.	Return (AOIDs*)	Die Applikation liefert die im 18. Schritt erstellte Liste AOIDs* an das Überwachungssystem zurück.

Tabelle 9: Erneuerung der asymmetrischen Schlüssel - Beschreibung zur Abbildung 15

3.4.9 Änderung der Zugriffsberechtigung inkl. Entfernung der alten Zugriffsberechtigung

Die Änderung der Zugriffsberechtigung muss in Abhängigkeit von deren Art unterschiedlich erfolgen:

- **Erweiterung** der vorhandenen Zugriffsberechtigung – siehe Kap. 3.4.9.1,
- **Einschränkung** der vorhandenen Zugriffsberechtigung – siehe Kap. 3.4.9.2.

3.4.9.1 Erweiterung der Zugriffsberechtigung

Da in diesem Falle ausschließlich neue Berechtigungen dazukommen, besteht nicht das in **A.15** zitierte Risiko eines unbefugten Zugriffs. Dementsprechend ist es hinreichend, die zu betreffenden D-AIP korrespondierenden Z-AIPs mit Hilfe der `ArchiveUpdate`-Funktion zu erweitern. Die letzte Version der entsprechenden Z-AIPs steuert den aktuellen Zugriff auf die passenden Anteile des tangierten D-AIP.

Die nachfolgende Abbildung 17 stellt einen exemplarischen Ablauf einer Zugriffsberechtigungserweiterung dar.

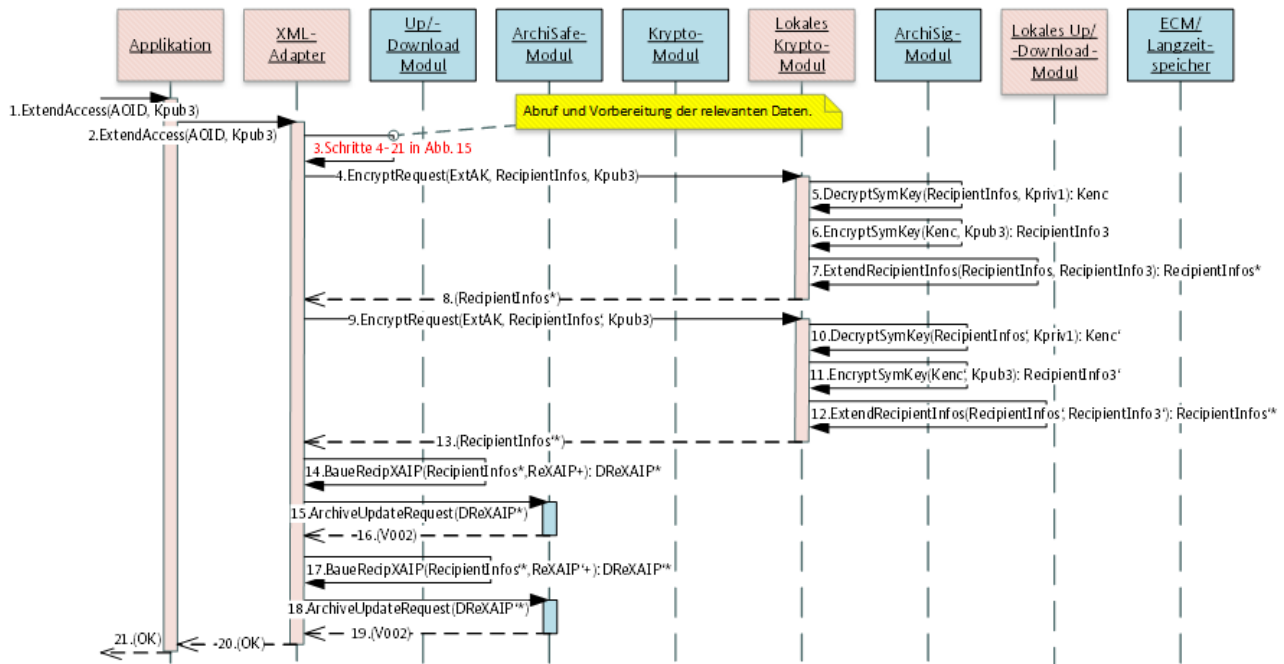


Abbildung 17: Erweiterung der Zugriffsberechtigung gem. [TR-ESOR-ENC].

Die nachfolgende Tabelle 10 beschreibt die einzelnen Schritte, dargestellt in der Abbildung 17.

Nr.	Aufruf	Beschreibung
1.	ExtendAccess (AOID, Kpub3)	Der Nutzer initiiert in der Applikation die Erweiterung der Zugriffsberechtigung für D-AIP mit der AOID $AOID$ für den Inhaber des öffentlichen Schlüssels K_{pub3} .
2.	ExtendAccess (AOID, Kpub3)	Die Applikation initiiert im XML-Adapter die Erweiterung der Zugriffsberechtigung für D-AIP mit der AOID $AOID$ für den Inhaber des öffentlichen Schlüssels K_{pub3} .
3.	<i>Schritte 4. – 21., in Abbildung 15</i>	Es werden die Schritte 4. bis 22. in der Abbildung 15 ausgeführt. Die benötigten Daten werden aus den D-AIP (AOID) und zugehörigen Z-AIPs ($ReAOID$ und $ReAOID'$) ermittelt, insbesondere: • $ReXAIP$ – Z-AIP mit dem $RecipientInfo$-Element, • $ReXAIP'$ – Z-AIP mit dem $RecipientInfo$-Element, • $RecipientInfos$ – Zugriffsberechtigung für Version V001, • $RecipientInfos'$ – Zugriffsberechtigung für die Version V002.
4.	EncryptRequest (ExtAK, RecipientInfos, Kpub3)	Der XML-Adapter ruft die <code>Encrypt</code> -Funktion an der TR-S.3'-Schnittstelle des lokalen Krypto-Moduls auf und übergibt das zu erweiternden $RecipientInfos$ -Element sowie die zusätzlichen asymmetrischen öffentlichen Schlüssel (Zertifikat) K_{pub3} . Das <code>ExtAK</code> -Attribut signalisiert dem lokalen Krypto-Modul gegenüber, dass es sich um die Erweiterung der Zugriffsberechtigung handelt.

Nr.	Aufruf	Beschreibung
5.	DecryptSymKey (RecipientInfos, Kpriv1) → Kenc	Das lokale Krypto-Modul entschlüsselt ein RecipientInfo-Element aus dem RecipientInfos-Element mit dem vorhandenen privaten asymmetrischen Schlüssel Kpriv1 ²¹ und ermittelt den symmetrischen Schlüssel Kenc.
6.	EncryptSymKey (Kenc, Kpub3) → RecipientInfo3	Das lokale Krypto-Modul verschlüsselt den im 5. Schritt ermittelten symmetrischen Schlüssel Kenc mit dem asymmetrischen öffentlichen Schlüssel Kpub3. Das Ergebnis ist eine Struktur RecipientInfo3.
7.	ExtendRecipientInfos (RecipientInfos, RecipientInfo3) → RecipientInfos*	Das lokale Krypto-Modul legt die im 6. Schritt ermittelte Struktur RecipientInfo3 in dem RecipientInfos-Element zusätzlich ab. Es entsteht ein neues RecipientInfos*-Element, das die erweiterte Zugriffsberechtigung abbildet.
8.	Return (RecipientInfos*)	Das lokale Krypto-Modul liefert das neu erzeugte RecipientInfos*-Element an den XML-Adapter zurück.
9.	EncryptRequest (ExtAK, RecipientInfos', Kpub3)	Der XML-Adapter ruft die Encrypt-Funktion an der TR-S.3' -Schnittstelle des lokalen Krypto-Moduls auf und übergibt das zu erweiternden RecipientInfos'-Element sowie die zusätzlichen asymmetrischen öffentlichen Schlüssel (Zertifikat) Kpub3. Das ExtAK-Attribut signalisiert dem lokalen Krypto-Modul gegenüber, dass es sich um die Erweiterung der Zugriffsberechtigung handelt.
10.	DecryptSymKey (RecipientInfos', Kpriv1) → Kenc'	Das lokale Krypto-Modul entschlüsselt ein RecipientInfo-Element aus dem RecipientInfos'-Element mit dem vorhandenen privaten asymmetrischen Schlüssel Kpriv1 ²² und ermittelt den symmetrischen Schlüssel Kenc'.
11.	EncryptSymKey (Kenc', Kpub3) → RecipientInfo3'	Das lokale Krypto-Modul verschlüsselt den im 10. Schritt ermittelten symmetrischen Schlüssel Kenc' mit dem asymmetrischen öffentlichen Schlüssel Kpub3. Das Ergebnis ist eine Struktur RecipientInfo3'.
12.	ExtendRecipientInfos (RecipientInfos', RecipientInfo3') → RecipientInfos**	Das lokale Krypto-Modul legt die im 12. Schritt ermittelte Struktur RecipientInfo3' in dem RecipientInfos'-Element zusätzlich ab. Es entsteht ein neues RecipientInfos**-Element, das die erweiterte Zugriffsberechtigung abbildet.
13.	Return (RecipientInfos**)	Das lokale Krypto-Modul liefert das neu erzeugte RecipientInfos**-Element an den XML-Adapter zurück.
14.	BaueRecipXAIP (RecipientInfos*, ReXAIP+) → DReXAIP*	Der XML-Adapter erzeugt ein DReXAIP*, um die Version V002 im Z-AIP ReXAIP+ mit dem im 8. Schritt empfangenen RecipientInfo*-Element abzulegen und somit die gewünschte Zugriffsrechteerneuerung zu finalisieren.

Nr.	Aufruf	Beschreibung
15.	ArchiveUpdateRequest (DReXAIP*)	Der XML-Adapter ruft die <code>ArchiveUpdate</code> -Funktion an der TR-S.4 -Schnittstelle des zentralen ArchiSafe-Moduls auf und übergibt das im 14. Schritt erzeugte DReXAIP*. Ab hier wird der Standardablauf angewandt, vgl. [TR-ESOR], Kap. 7.5.2, Schritt 5.
16.	Return (V002)	Das zentrale ArchiSafe-Modul bestätigt die erfolgreiche Aktualisierung des Z-AIP mit der AOID <code>ReAOID</code> . Das entstandene ReXAIP* beinhaltet zwei Versionen: V001 und V002.
17.	BaueRecipXAIP (RecipientInfos*, ReXAIP'+) → DReXAIP*	Der XML-Adapter erzeugt ein DReXAIP'', um die Version V002 im Z-AIP <code>ReXAIP'+</code> mit dem im 13. Schritt empfangenen <code>RecipientInfo''</code> -Element abzulegen und somit die gewünschte Zugriffsrechteerneuerung zu finalisieren
18.	ArchiveUpdateRequest (DReXAIP'*)	Der XML-Adapter ruft die <code>ArchiveUpdate</code> -Funktion an der TR-S.4 -Schnittstelle des zentralen ArchiSafe-Moduls auf und übergibt das im 14. Schritt erzeugte DReXAIP'*. Ab hier wird der Standardablauf angewandt, vgl. [TR-ESOR], Kap. 7.5.2, Schritt 5.
19.	Return (V002)	Das zentrale ArchiSafe-Modul bestätigt die erfolgreiche Aktualisierung des Z-AIP mit der AOID <code>ReOID'</code> . Das entstandene ReXAIP'* beinhaltet zwei Versionen: V001 und V002.
20.	Return (OK)	Alle relevanten Z-AIPs (<code>ReXAIP</code> und <code>ReXAIP'</code>) wurden erfolgreich aktualisiert. Es entstand eine neue Zugriffsberechtigung mit dem öffentlichen Schlüssel <code>Kpub3</code> . Der XML-Adapter bestätigt gegenüber der Applikation den erfolgreichen Abschluss der Operation.
21.	Return (OK)	Die Applikation bestätigt dem Nutzer die erfolgreiche Erweiterung der Zugriffsberechtigung.

Tabelle 10: Erweiterung der Zugriffsberechtigung - Beschreibung zur Abbildung 17.

3.4.9.2 Einschränkung der Zugriffsberechtigung

In diesem Fall kommt das in **A.15** zitierte Risiko zur Geltung und es gelten daher die Anforderungen **A.17** und **A.18**.

Die nachfolgende Abbildung 18 zeigt einen beispielhaften Ablauf einer Zugriffsberechtigungseinschränkung, in dem einem Nutzer (einer Rolle) mit dem öffentlichen Schlüssel `Kpub2` der Zugriff auf das D-AIP `LXAIP++'` entzogen wird.

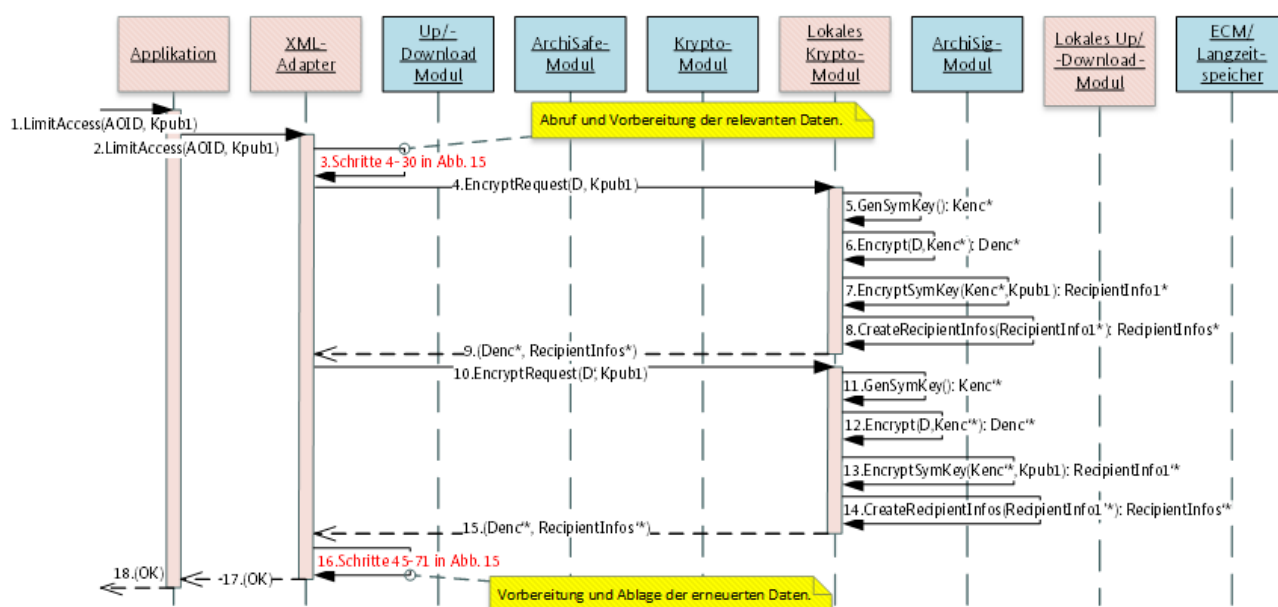


Abbildung 18: Einschränkung der Zugriffsberechtigung gem. [TR-ESOR-ENC].

Die nachfolgende Tabelle 11 beschreibt die einzelnen Schritte, dargestellt in der Abbildung 18.

Nr.	Aufruf	Beschreibung
1.	LimitAccess(AOID, Kpub1)	Der Nutzer initiiert in der Applikation die Einschränkung der Zugriffsberechtigung für D-AIP mit der AOID $AOID$ für den Inhaber des öffentlichen Schlüssels K_{pub2} .
2.	LimitAccess (AOID, Kpub1)	Die Applikation initiiert im XML-Adapter die Einschränkung der Zugriffsberechtigung für D-AIP mit der AOID $AOID$ für den Inhaber des öffentlichen Schlüssels K_{pub2} .
3.	<i>Schritte 4.–30., in Abbildung 15</i>	Es werden die Schritte 4. bis 30. in der Abbildung 15 ausgeführt. Die benötigten Daten werden aus den D-AIP (AOID) und zugehörigen Z-AIPs ($ReAOID$ und $ReAOID'$) ermittelt, insbesondere: • D – Klartextdokument D, • D' – Klartextdokument D', • $RecipientInfos$ – Zugriffsberechtigung für Version V001 von LXAIP++', • $RecipientInfos'$ – Zugriffsberechtigung für die Version V002 von LXAIP++'.
4.	EncryptRequest(D, Kpub1)	Der XML-Adapter ruft die <code>Encrypt</code> -Funktion an der TR-S.3' -Schnittstelle des lokalen Krypto-Moduls auf und übergibt das Klartextdokument D und den asymmetrischen öffentlichen Schlüssel (Zertifikat) K_{pub1} . Der asymmetrische Schlüssel K_{pub2} fehlt.
5.	GenSymKey() → Kenc*	Das lokale Krypto-Modul generiert einen neuen symmetrischen Schlüssel K_{enc*} .

Nr.	Aufruf	Beschreibung
6.	Encrypt (D, Kenc*) → Denc*	Das lokale Krypto-Modul verschlüsselt das Klartextdokument D mit dem im 5. Schritt neu generierten symmetrischen Schlüssel Kenc*. Es entsteht das verschlüsselte Klartextdokument Denc*.
7.	EncryptSymKey (Kenc*, Kpub1) → RecipientInfo1*	Das lokale Krypto-Modul verschlüsselt den generierten symmetrischen Verschlüsselungsschlüssel Kenc* mit dem asymmetrischen öffentlichen Schlüssel des Nutzers Kpub1. Das Ergebnis ist eine Struktur RecipientInfo1*.
8.	CreateRecipientInfos (RecipientInfo1*) → RecipientInfos*	Das lokale Krypto-Modul legt das im 7. Schritt erzeugte RecipientInfo1* in eine Struktur RecipientInfos*, inkl. weiterer Daten, wie z.B. Algorithmus für die Schlüsselverschlüsselung, ab.
9.	Return (Denc*, RecipientInfos*)	Antwort auf den 4. Schritt: Das lokale Krypto-Modul liefert das verschlüsselte Dokument D als Denc* und die korrespondierende RecipientInfo-Struktur RecipientInfos* als Zugriffsberechtigung an den XML-Adapter zurück.
10.	EncryptRequest (D', Kpub1)	Der XML-Adapter ruft die Encrypt-Funktion an der TR-S.3'-Schnittstelle des lokalen Krypto-Moduls auf und übergibt das Klartextdokument D' und den asymmetrischen öffentlichen Schlüssel (Zertifikat) Kpub1. Der asymmetrische Schlüssel Kpub2 fehlt.
11.	GenSymKey () → Kenc'*	Das lokale Krypto-Modul generiert einen neuen symmetrischen Schlüssel Kenc*.
12.	Encrypt (D', Kenc'*) → Denc'*	Das lokale Krypto-Modul verschlüsselt das Klartextdokument D' mit dem im 11. Schritt neu generierten symmetrischen Schlüssel Kenc*. Es entsteht das verschlüsselte Klartextdokument Denc*.
13.	EncryptSymKey (Kenc'* , Kpub1) → RecipientInfo1'*	Das lokale Krypto-Modul verschlüsselt den generierten symmetrischen Verschlüsselungsschlüssel Kenc'* mit dem asymmetrischen öffentlichen Schlüssel des Nutzers Kpub1. Das Ergebnis ist eine Struktur RecipientInfo1'*.
14.	CreateRecipientInfos (RecipientInfo1'*) → RecipientInfos'*	Das lokale Krypto-Modul legt das im 13. Schritt erzeugte RecipientInfo1'* in eine Struktur RecipientInfos'*, inkl. weiterer Daten, wie z.B. Algorithmus für die Schlüsselverschlüsselung, ab.
15.	Return (Denc'* , RecipientInfos'*)	Antwort auf den 10. Schritt: Das lokale Krypto-Modul liefert das verschlüsselte Dokument D' als Denc'* und die korrespondierende RecipientInfo-Struktur RecipientInfos'* als Zugriffsberechtigung an den XML-Adapter zurück.

Nr.	Aufruf	Beschreibung
16.	<i>Schritte 45. – 71., in Abbildung 15</i>	Es erfolgt die Migration der Daten in ein neues D-AIP und in die entsprechend neuen Z-AIPs (im Sinne von Kap. 3.4.8.1).
17.	Return(OK)	Der XML-Adapter bestätigt die erfolgreiche Durchführung der Zugriffseinschränkungsoperation gegenüber der Applikation.
18.	Return(OK)	Die Applikation bestätigt die erfolgreiche Durchführung der Zugriffseinschränkungsoperation gegenüber dem Nutzer.

Tabelle 11: Einschränkung der Zugriffsberechtigung – Beschreibung zur Abbildung 18.

3.5 Profilierung der verwendeten Schnittstellen [TR-ESOR-E]

Die in der Abbildung 3 für die TR-S.4 und in der Abbildung 4 für TR-S.512 in [TR-ESOR-E], Kap. 2 dargestellte Umsetzung der jeweiligen IT-Referenzarchitektur auf Basis des eCard-API-Frameworks ist nachfolgend um die Aspekte dieses Profils in der Abbildung 19 erweitert und in der Abbildung 20 dargestellt worden.

Hinweis!

Um die Lesbarkeit zu verbessern, sind die in diesem Profil zusätzlich definierten Artefakte in der Farbe **Lila** und die angepassten Artefakte in der Farbe **Grün** abgebildet. Die konkrete Ausgestaltung ist den nachfolgenden Kapiteln zu entnehmen.

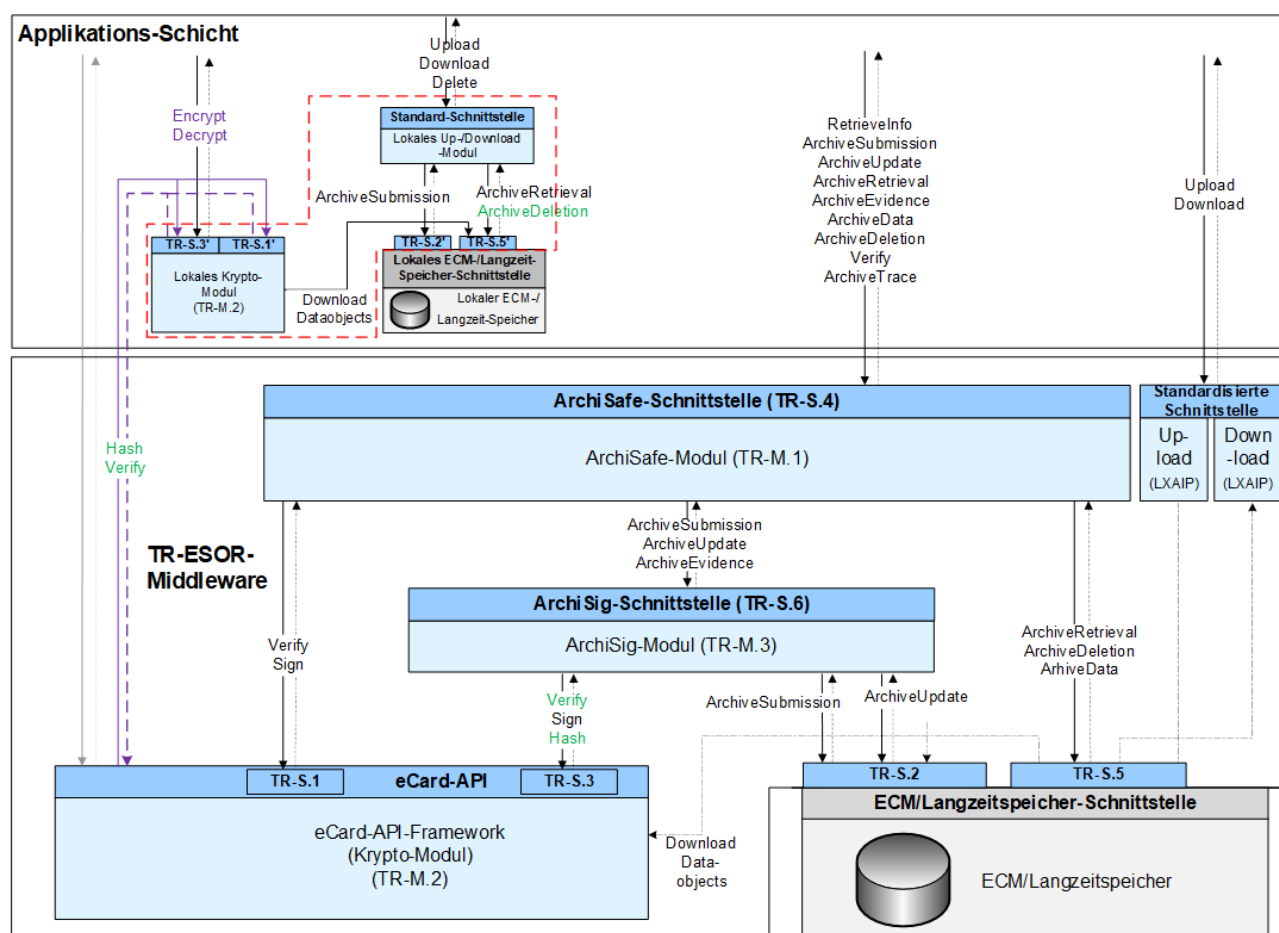


Abbildung 19: Umsetzung der IT-Referenzarchitektur von [TR-ESOR-ENC] auf Basis von TR-S.4.

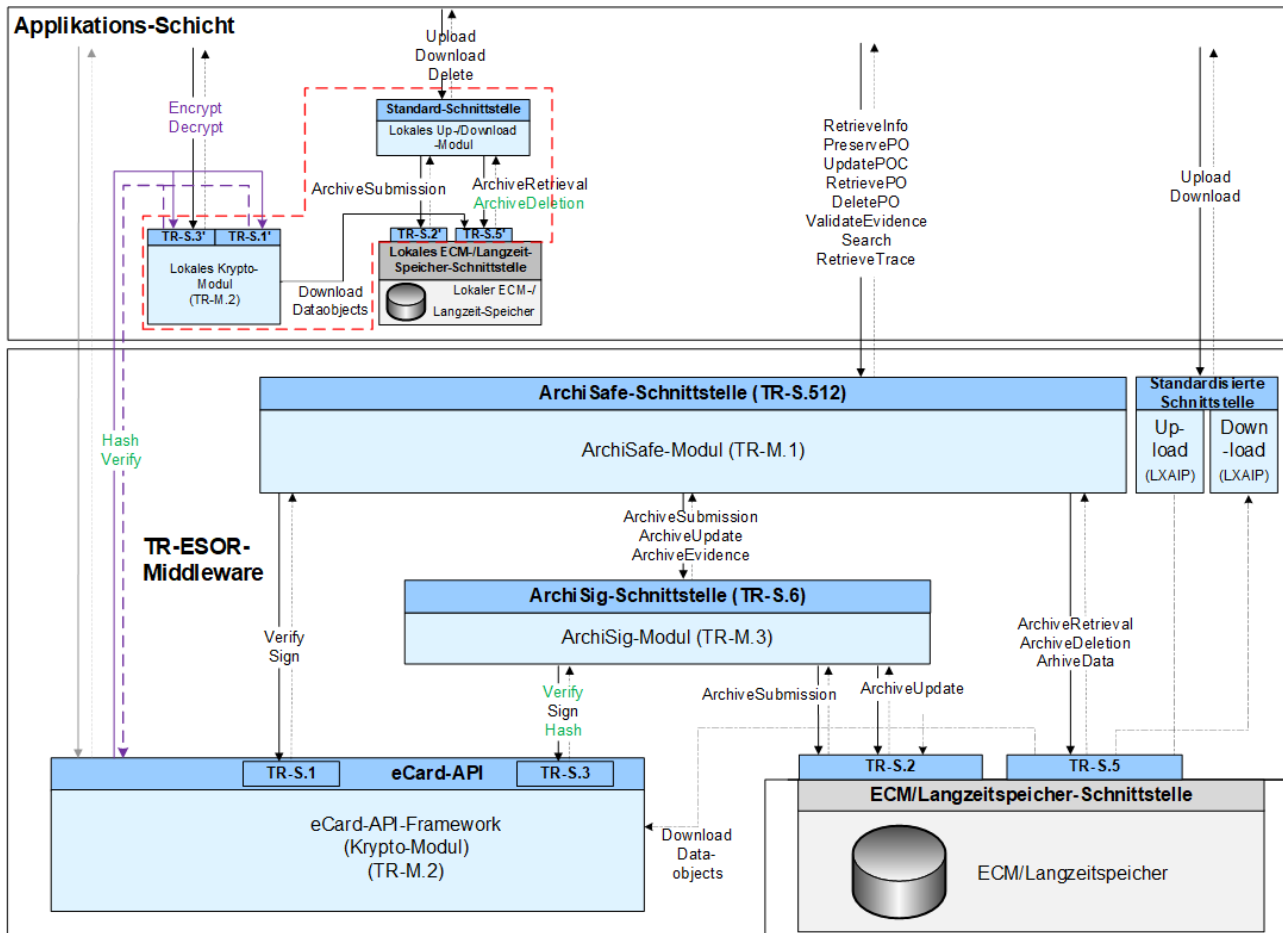


Abbildung 20: Umsetzung der IT-Referenzarchitektur von [TR-ESOR-ENC] auf Basis von TR-S.512.

Hinweis!

Die in diesem Dokument zusätzlich eingeführten XML-Datenstrukturen wurden im folgenden Namensraum definiert: `xmlns:enc=http://www.bsi.bund.de/tr-esor/enc` (vgl. hierzu Kap. 7).

3.5.1 Die Schnittstelle TR-S.3'

Die Schnittstelle **TR-S.3'** des lokalen Krypto-Moduls setzt auf die Definition der Schnittstelle **TR-S.3** (vgl. Kap. 5.3, [TR-ESOR-E]) auf und wird um die folgenden Funktionen erweitert:

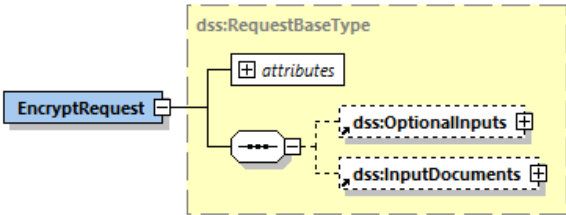
- (1) Verschlüsselung von Daten: `Encrypt`-Funktion (vgl. Kap. 3.5.1.1),
- (2) Entschlüsselung von verschlüsselten Daten: `Decrypt`-Funktion (vgl. Kap. 3.5.1.2).

Die beiden Funktionen orientieren sich an den entsprechenden Funktionen aus dem eCardAPI-Framework gem. [eCard-2], Kap. 3.3.

3.5.1.1 `Encrypt`-Funktion: Verschlüsselung von Daten

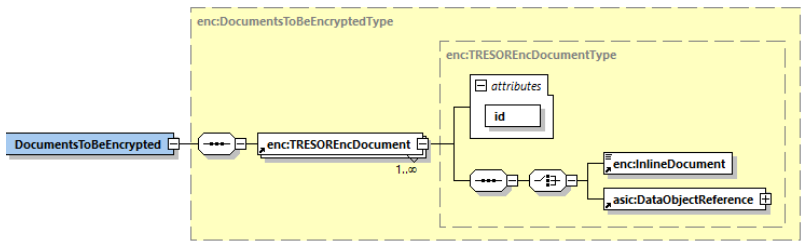
Zur Verschlüsselung der Daten ist in **TR-S.3'** (vgl. Abbildung 1 und Abbildung 2) die Anfrage `EncryptRequest` und die Antwort `EncryptResponse` aus [eCard-2], Kap. 3.3.1 vorgesehen.

3.5.1.1.1 EncryptRequest

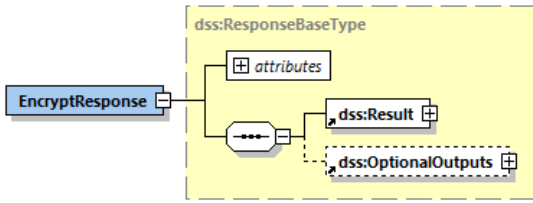
Name	<i>EncryptRequest</i>
Beschreibung	Mit Hilfe der <code>Encrypt</code>-Funktion werden im Sinne dieser Profilierung folgende Anwendungsfälle ausgelöst: (1) Verschlüsselung²³ von Klartextdokumenten und entsprechende Vorbereitung der zugehörigen <code>RecipientInfos</code>-Elemente (vgl. Kap. 3.6.1 und Schritte 1 bis 8 in der Abbildung 8), (2) Neuverschlüsselung von <code>RecipientInfo</code>-Elementen bei der Erneuerung der asymmetrischen Zugriffsberechtigungsschlüssel (vgl. Kap. 3.4.8.2) (3) Erweiterung der korrespondierenden <code>RecipientInfos</code>-Elemente bei der Erweiterung der Zugriffsberechtigung (vgl. Kap. 3.4.9.1)
Details	Der Eingabeparameter <code>EncryptRequest</code> im Kontext der TR-S.3'-Schnittstelle weist folgenden Aufbau auf und kann wie folgt parametrisiert werden. 

²³ Die zu verwendenden kryptographischen Algorithmen (inkl. Längeneingaben) können per Konfiguration festgelegt werden.

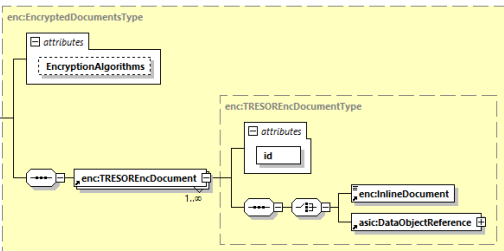
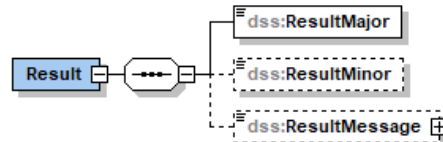
Name	Beschreibung der <i>EncryptRequest</i> -Parameter
dss:OptionalInputs	Das optionale Element dss:OptionalInputs <u>muss</u> vorhanden sein und beinhaltet folgende weitere Elemente: - Das optionale TRESOREncMode-Element, das die Art der Verschlüsselungsoperation definiert (siehe Beschreibung weiter oben). <pre> graph LR TRESOREncMode[TRESOREncMode] </pre> - Es können folgende Parameter definiert werden: http://www.bsi.bund.de/DE/tr-esor/enc-mode/encdoc - definiert das o.g. Szenario (1) und stellt die Standardbelegung dar, http://www.bsi.bund.de/DE/tr-esor/enc-mode/renasym - definiert das o.g. Szenario (2), http://www.bsi.bund.de/DE/tr-esor/enc-mode/extaccs - definiert das o.g. Szenario (3). Wenn dieses Element fehlt, dann wird das Szenario (1) durchgeführt. Abhängig vom gewählten Szenario werden folgende weitere Elemente übergeben: - Szenario (1): - Das xaip:certificateValues-Pflichtelement, das die X.509-Zertifikate der Zugriffsberechtigten Rollen enthält: <pre> graph LR certValues[certificateValues] --> attributes[attributes] certValues --> EncapsulatedX509Certificate[EncapsulatedX509Certificate] certValues --> OtherCertificate[OtherCertificate] EncapsulatedX509Certificate --- OtherCertificate subgraph "0..∞" EncapsulatedX509Certificate OtherCertificate end </pre> Es <u>dürfen hier ausschließlich</u> die Elemente EncapsulatedX509Certificate verwendet werden. - Szenario (2): - Das RecipientInfos-Pflichtelement, das die akt. Zugriffsberechtigung enthält, - Das xaip:certificatesValues-Pflichtelement, das die neuen Zertifikate enthält. Die Anzahl der RecipientInfo-Elemente und der EncapsulatedX509Certificate-Elemente aus dem xaip:certificatesValues-Element <u>muss</u> gleich sein. - Szenario (3): - Das RecipientInfos-Pflichtelement, das die akt. Zugriffsberechtigung enthält, - Das xaip:certificatesValues-Pflichtelement, das die zusätzlichen Zertifikate enthält.

Name	Beschreibung der <i>EncryptRequest</i> -Parameter
dss:InputDocuments	Es <u>muss</u> ausschließlich im Szenario (1) ein DocumentsToBeEncrypted-Element mit <u>mindestens</u> einem Klartextdokument (als TRESOREncDocument-Element abgelegt), dessen Zugriffsbestimmung gem. den Vorgaben aus dem xaip:certificateValues-Element geregelt werden <u>müssen</u>, enthalten werden. Das TRESOREncDocument-Element beinhaltet entweder die binären Daten innerhalb vom InlineDocument-Element oder eine Referenz auf zuvor hochgeladenen Daten innerhalb vom asic:DataObjectReference-Element (vgl. hierzu [TR-ESOR-F], Kap. 3.2.1).  Das id-Attribut identifiziert eindeutig ein Klartextdokument und wird mit dem korrespondierenden verschlüsselten Dokument zurückgegeben.

3.5.1.1.2 EncryptResponse

Name	<i>EncryptResponse</i>
Beschreibung	Als Antwort auf einen EncryptRequest (vgl. Kap. 3.5.1.1.1) wird von der TR-S.3'-Schnittstelle des lokalen Krypto-Moduls zu jedem übergebenen Klartextdokument D eine verschlüsselte Version D_{enc} zurückgeliefert. Darüber hinaus wird eine Instanz des RecipientInfos-Elements zurückgeleitet, mit deren Hilfe die Zugriffsberechtigung auf die verschlüsselten Dokumente geregelt wird. Wenn die Anfrage nicht erfolgreich ausgeführt werden kann, <u>muss</u> eine Fehlermeldung zurückgegeben werden.
Name	Der EncryptResponse-Ausgabeparameter weist folgenden Aufbau auf und wird wie folgt parametrisiert. 

Name	Beschreibung der <i>EncryptResponse</i> -Ausgabeparameter
dss:Result	Enthält die Statusinformationen und ggf. die Fehler zu einer durchgeführten Aktion. Die Struktur dieses Elements und die möglichen Fehlercodes sind in Abs. 4.1.2 von [eCard-1] und in Abs. 3.2.1 von [eCard-2] beschrieben.

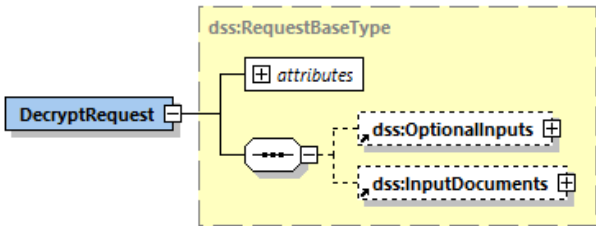
Name	Beschreibung der <i>EncryptResponse</i> -Ausgabeparameter
dss:OptionalOutputs	Das optionale dss:OptionalOutputs-Element <u>muss</u> im Erfolgsfall vorhanden sein und enthält, in Abhängigkeit von dem ausgeführten Szenario, folgende Elemente: • Szenario (1): ○ Das RecipientInfos-Element <u>muss</u> vorhanden sein, ○ Das EncryptedDocuments-Element, das die verschlüsselten Klartextdokumente enthält <u>muss</u> enthalten sein:  Die verschlüsselten Daten werden innerhalb des TRESEncDocument-Element binäre als InlineDocument-Element, oder als Referenz auf diese innerhalb vom asic:DataObjectReference-Element (vgl. [TRESOR_F], Kap. 3.2.1) abgelegt. Das optionale EncryptionAlgorithm-Attribut kann einen Hinweis auf die verwendete Verschlüsselungsart beinhalten. Der Wert des id-Attributs wird aus dem zugehörigen DocumentToBeEncrypted-Element kopiert. • Szenario (2): ○ Ein RecipientInfos-Element mit erneuerten Zugriffsberechtigungen muss zurückgegeben werden. • Szenario (3): ○ Ein RecipientInfos-Element mit den entsprechenden RecipientInfo-Elementen für die zusätzliche Zugriffsberechtigungen (Zertifikate) muss zurückgegeben werden.
dss:Result	Statusinformationen und Fehler beim Aufruf der Funktion Delete-Funktion (vgl. [eCard-1] Abs. 4.1 und Abs. 4.2). 

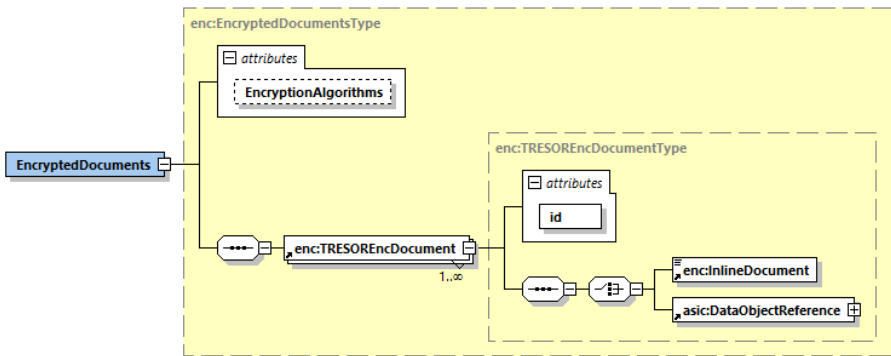
Name	Fehlercode in <i>Result</i> für <i>EncryptResponse</i>
ResultMajor	• <u>/resultmajor#ok</u> • <u>/resultmajor#error</u>
ResultMinor	Siehe [eCard-2], Kap. 3.3.1.

3.5.1.2 Decrypt-Funktion: Entschlüsselung von verschlüsselten Daten

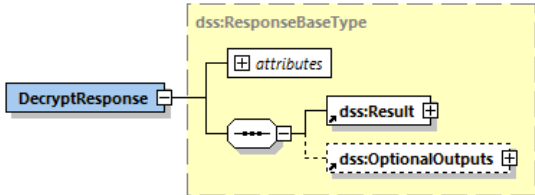
Zur Entschlüsselung der verschlüsselten Daten ist in **TR-S.3'** (vgl. Abbildung 1 und Abbildung 2) die Anfrage `DecryptRequest` und die Antwort `DecryptResponse` aus **[eCard-2]**, Kap. 3.3.2 vorgesehen.

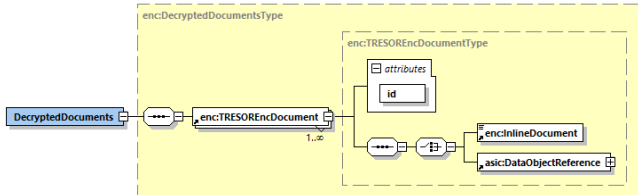
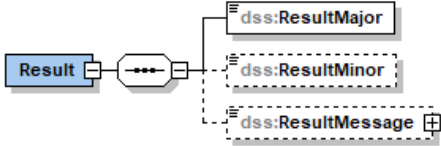
3.5.1.2.1 DecryptRequest

Name	<i>DecryptRequest</i>
Beschreibung	Mit Hilfe der <code>Decrypt</code> -Funktion wird im Sinne dieser Profilierung die Entschlüsselung von zuvor lokal verschlüsselten Klartextdokumenten initiiert.
Details	Der Eingabeparameter <code>DecryptRequest</code> im Kontext der TR-S.3'-Schnittstelle weist folgenden Aufbau auf und kann wie folgt parametrisiert werden. 

Name	Beschreibung der <i>DecryptRequest</i> -Eingabeparameter
<code>dss:OptionalInputs</code>	Das optionale Element <code>dss:OptionalInputs</code> <u>muss</u> vorhanden sein und beinhaltet folgende weitere Elemente: Das <code>RecipientInfos</code>-Pflichtelement, das die akt. Zugriffsberechtigungen enthält.
<code>dss:InputDocuments</code>	Das optionale <code>dss:InputDocuments</code>-Element <u>muss</u> vorhanden sein und beinhaltet das <code>EncryptedDocuments</code>-Element, das die einzelnen verschlüsselten Klartextdokumente (jeweils als ein <code>TRESEncDocument</code>-Element kodiert) beherbergt:  Die verschlüsselten Daten werden innerhalb des <code>TRESEncDocument</code>-Element binäre als <code>InlineDocument</code>-Element, oder als Referenz auf diese innerhalb vom <code>asic:DataObjectReference</code>-Element (vgl. [TRESOR_F], Kap. 3.2.1) abgelegt. Das optionale <code>EncryptionAlgorithm</code>-Attribut kann einen Hinweis auf die verwendete Verschlüsselungsart beinhalten. Das obligatorische <code>id</code>-Attribut referenziert eindeutig ein verschlüsseltes Klartextdokument.

3.5.1.2.2 DecryptResponse

Name	<i>DecryptResponse</i>
Beschreibung	Als Antwort auf einen DecryptRequest (vgl. Kap. 3.5.1.2.1) wird von der TR-S.3'-Schnittstelle des lokalen Krypto-Moduls zu jedem übergebenen verschlüsselten Klartextdokument <i>Denc</i> ein Klartextdokument <i>D</i> zurückgeliefert. Wenn die Anfrage nicht erfolgreich ausgeführt werden kann, <u>muss</u> eine Fehlermeldung zurückgegeben werden.
Details	Der DecryptResponse-Ausgabeparameter weist folgenden Aufbau auf und wird wie folgt parametrisiert. 

Name	dss:Result
dss:Result	Enthält die Statusinformationen und die Fehler zu einer durchgeführten Aktion. Die Struktur dieses Elements und die möglichen Fehlercodes sind in Abs. 4.1.2 von [eCard-1] und in Abs. 3.2.1 von [eCard-2] beschrieben.
dss:OptionalOutputs	Das optionale dss:OptionalOutputs-Element <u>muss</u> im Erfolgsfall vorhanden sein und enthält das DecryptedDocuments-Element, das die entschlüsselten Klartextdokumente, die jeweils in Form eines TRESOREncDocument-Element abgelegt worden sind:  Die Dokumente können entweder binär in einem InlineDocument-Element oder als eine Referenz in einem asic:DataObjectReference-Element (vgl. [TR-ESOR-F], Kap. 3.2.1) abgelegt werden. Der Wert des id-Attributs wird aus dem zugehörigen EncrypteDocument-Element des DecryptRequest-Elements kopiert.
Statusinformationen und Fehlercodes	Statusinformationen und Fehler beim Aufruf der Funktion Delete-Funktion (vgl. [eCard-1] Abs. 4.1 und Abs. 4.2). 

Name	dss:Result
Name	Fehlercode
ResultMajor	• /resultmajor#ok • /resultmajor#error
ResultMinor	Siehe [eCard-2], Kap. 3.3.1.

3.5.2 Schnittstelle des lokalen Up-Download-Moduls

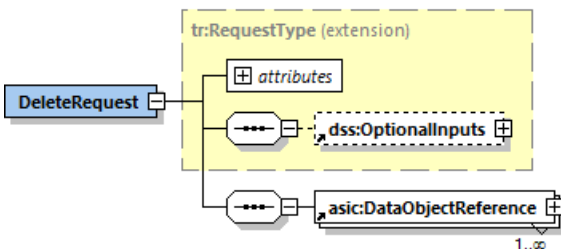
Gem. [TR-ESOR-E], Kap. 6 liegt die konkrete Ausgestaltung der Upload/Download-Schnittstelle in Verantwortung eines Herstellers und es werden lediglich die wichtigsten zu beachtenden Aspekte dargelegt.

Um den in Kap. 3.4.6 definierten Prozess umsetzen zu können, muss die lokale Upload/Download-Schnittstelle eine zusätzliche `Delete`-Funktion enthalten. Die wichtigsten Aspekte dieser Funktion sind dem Kap. 3.5.2.1 zu entnehmen.

3.5.2.1 Delete-Funktion

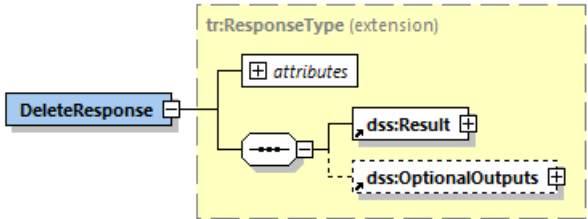
Mit Hilfe der `Delete`-Funktion der lokalen Upload/Download-Schnittstelle können die zuvor unter der Verwendung der `Upload`-Funktion hochgeladenen Dateien (vgl. hierzu [TR-ESOR-E], Kap. 6.1) gelöscht werden.

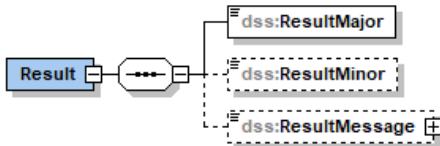
3.5.2.1.1 DeleteRequest

Name	<i>DeleteRequest</i>
Beschreibung	Mit Hilfe des <code>DeleteRequest</code> wird (werden) ein (oder mehrere) zuvor im lokalen ECM/Langzeitspeicher gespeichert(e) und in einem LXAIP referenziertes/-en, lokales/-en Datenobjekt(e) gelöscht.
Details	Folgende Darstellung einer möglichen <code>Delete</code>-Anfrage in Form des <code>DeleteRequest</code>-Elements stellt eine Empfehlung für die Umsetzung dar. 

Name	Beschreibung von <i>DeleteRequest</i>
<code>dss:OptionalInputs</code>	Das optionale Element <code>dss:OptionalInputs</code> ist nicht vorhanden.
<code>asic:DataObjectReference</code>	Enthält mindestens eine Instanz des <code>asic:DataObjectReference</code> -Elements gem. [TR-ESOR-F], Kap. 3.2.1, welches das zuvor übermittelte und im zugehörigen LXAIP referenzierte lokale Datenobjekt eindeutig beschreibt.

3.5.2.1.2 DeleteResponse

Name	<i>DeleteResponse</i>
Beschreibung	Als Antwort auf eine <i>DeleteRequest</i> (vgl. Kap. 3.5.2.1.1) wird von der lokalen Upload/Download-Schnittstelle zu jedem, mit einer Instanz des <i>asic:DataObjectReference</i>-Elements angefragten lokalen Datenobjekt, der Status der erfolgten Löschoperation zurückgeliefert. Wenn die Anfrage nicht erfolgreich ausgeführt werden kann, muss eine Fehlermeldung zurückgegeben werden.
Details	Folgende Darstellung einer möglichen <i>DeleteResponse</i> in Form des <i>DeleteResponse</i>-Elements stellt eine Empfehlung für die Umsetzung im Falle eines Fehlers dar. 

Name	Spezifikation des <i>DeleteResponse</i> -Elements
<i>dss:Result</i>	Enthält die Statusinformationen und ggf. die Fehlermeldung zu einer durchgeführten Aktion. Die Struktur dieses Elements und die möglichen Fehlercodes sind in Abs. 4.1.2 von [eCard-1] und in Abs. 3.2.1 von [eCard-2] beschrieben.
<i>dss:OptionalOutputs</i>	Das optionale <i>dss:OptionalOutputs</i> -Element ist nicht vorhanden.
Statusinformationen und Fehler	Statusinformationen und Fehler beim Aufruf der Funktion <i>Delete</i>-Funktion (vgl. [eCard-1] Abs. 4.1 und Abs. 4.2). 

Fehlercode-Name	Fehlercodes für <i>DeleteResponse</i>
ResultMajor	/resultmajor#ok /resultmajor#error
ResultMinor	/resultminor/ar/unknownDataObjectReference

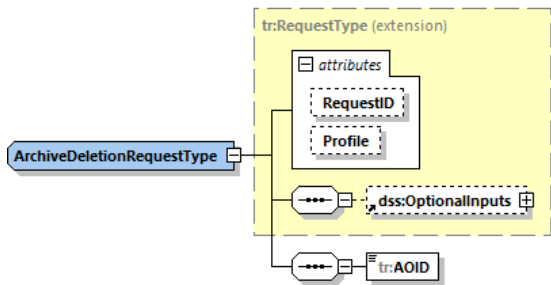
3.5.3 Schnittstelle TR-S.5'

Die TR-S.5-Schnittstelle bietet gem. [TR-ESOR-E], Kap. 5.4 folgende Funktionen an:

- ArchiveRetrieval
- ArchiveDeletion
- ArchiveData.

Um die Löschung der lokal abgelegten Klartextdokumente an der TR-S.5'-Schnittstelle aus dem lokalen ECM/Langzeitspeicher zu ermöglichen, wird der in [TR-ESOR-E], Kap. 5.4.3 definierte ArchiveDeletionRequest-Eingabeparameter der ArchiveDeletion-Funktion entsprechend profiliert

3.5.3.1 Eingabeparameter: ArchiveDeletionRequest

Name	ArchiveDeletionRequest	
Beschreibung	Mit Hilfe der ArchiveDeletionRequest-Anfrage wird ein (oder mehrere) zuvor im lokalen ECM/Langzeitspeicher und in einem LXAIP referenziertes/-en lokales/-en Datenobjekt(e) aus dem lokalen ECM/Langzeitspeicher gelöscht werden.	
Details	Der Eingabeparameter ArchiveDeletionRequest weist folgende Aufbau auf und wird in dem hier beschriebenen Anwendungsfall wie folgt parametrisiert. 	
	Name	Beschreibung
	dss:OptionalInputs	Das optionale Element dss:OptionalInputs <u>muss</u> vorhanden sein und muss mindestens eine Instanz des asic:DataObjectReference-Elements gem. [TR-ESOR-F], Kap. 3.2.1 beinhalten, das das zuvor übermittelte und im zugehörigen LXAIP referenzierte lokale Datenobjekt eindeutig beschreibt, das gelöscht werden soll.
	tr:AOID	Dieses verpflichtende Element <u>muss</u> vorhanden sein und <u>muss</u> leer sein.

3.6 Profilierung der notwendigen Datenformate [TR-ESOR-F]

In diesem Kapitel werden zusätzliche Datenformate definiert, die für die Umsetzung der im Kap. 3.4 beschriebenen fachlichen Prozesse benötigt werden. Es werden dabei die im Kap. 3.3 dargelegten Konzepte konkretisiert und die bestehende XAIP-Definition (XML-Schema) erweitert.

Hinweis!

Die in diesem Dokument zusätzlich eingeführten XML-Datenstrukturen wurden im folgenden Namensraum definiert: xmlns:enc=http://www.bsi.bund.de/tr-esor/enc (vgl. hierzu Kap. 7).

3.6.1 Rollenreferenzen für Zugriffssteuerung

Eine Rollenreferenz beinhaltet eine eindeutige Referenz auf ein PKI-Zertifikat, das für die asynchrone Verschlüsselung des symmetrischen Schlüssels, mit dem das korrespondierende Dokument verschlüsselt worden ist, verwendet wurde (vgl. auch Kap. 3.3). In der Abbildung 21 wird die Definition der Rollenreferenz als `xaip:RecipientInfo`-Element dargestellt.

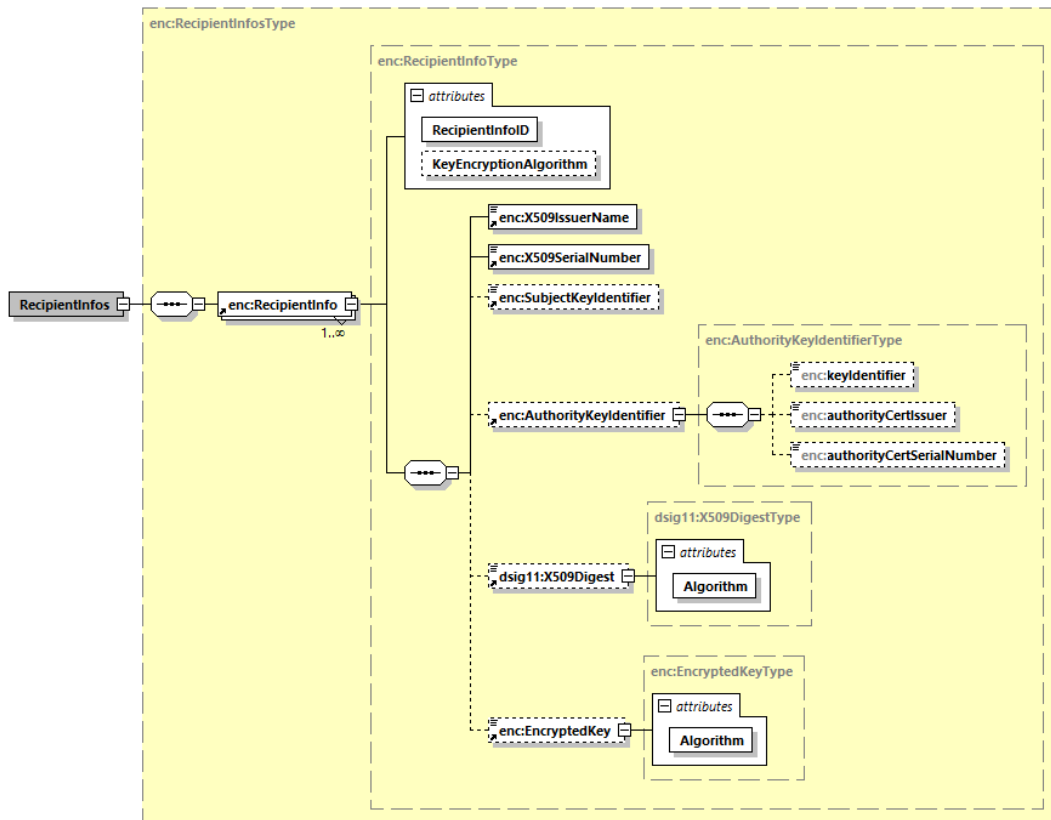


Abbildung 21: Definition der Rollenreferenzen als `xaip:RecipientInfos`-XML-Struktur.

Das `xaip:RecipientInfos`-Element enthält folgende Elemente:

`<RecipientInfo>` [required, unbounded]

Das `<RecipientInfo>`-Element beschreibt eine Referenz auf ein Zertifikat, mit dem der für die symmetrische Verschlüsselung des korrespondierenden Dokuments verwendeten symmetrische Schlüssel `kenc` verschlüsselt worden ist.

Die Struktur eines `xaip:RecipientInfo`-Elements ist wie folgt definiert:

`<X509IssuerName>` [required]

Das `<X509IssuerName>`-Element beinhaltet den Namen der Herausgeber des zugehörigen Zertifikats, z. B.: „CN=PCA-1-Verwaltung-03, O=PKI-1-Verwaltung, C=DE“

`<X509SerialNummber>` [required]

Das `<X509SerialNummber>`-Element beinhaltet die Seriennummer des zugehörigen Zertifikats.

`<SubjectKeyIdentifier>` [optional]

Das `<SubjectKeyIdentifier>`-Element erlaubt die Identifizierung eines bestimmten Zertifikats (vgl. [RFC3280], Kap. 4.2.1.2).

`<AuthorityKeyIdentifier>` [optional]

Das `<AuthorityKeyIdentifier>`-Element beinhaltet eine Referenz zum Zertifikat, mit dessen zugehörigem privaten Schlüssel dieses Zertifikat signiert worden ist (vgl. [RFC3280], Kap. 4.2.1.1).

<dsig11:X509Digest> [optional]

Das <dsig11:X509Digest>-Element beinhaltet einen Hashwert des Zertifikats.

<EncryptedKey> [optional]

Das <EncryptedKey>-Element beinhaltet den verschlüsselten symmetrischen Schlüssel.

[@RecipientInfoID] [required]

Das <RecipientInfoID>-Attribut beinhaltet einen frei wählbaren Identifikator für diese Instanz des xaip:RecipientInfo-Elements.

[@KeyEncryptionAlgorithm] [optional]

Das <KeyEncryptionAlgorithm>-Attribut beinhaltet eine URI, die den für die Verschlüsselung des symmetrischen Schlüssels verwendete Algorithmus bestimmt, z. B.:

- <http://www.w3.org/2001/04/xmlenc#rsa-oaep-mgf1p> or
- <http://www.w3.org/2009/xmlenc11#rsa-oaep>, etc.

Die Struktur eines AuthorityKeyIdentifier-Elements ist wie folgt definiert:

<keyIdentifier> [optional]

Das <keyIdentifier>-Element beinhaltet eine Referenz zum Zertifikat der herausgebenden CA (vgl. [RFC3280], Kap. 4.2.1.1).

<authorityCertIssuer> [optional]

Das <authorityCertIssuer>-Element beinhaltet den Namen des Herausgebers des referenzierten Herausgeberzertifikats

<authorityCertSerialNumber> [optional]

Das <authorityCertSerialNumber>-Element beinhaltet die Seriennummer des Herausgeberzertifikats.

Die Struktur eines dsig11:X509Digest-Element ist wie nachfolgend definiert:

[@Algorithm] [required]

Das <Algorithm>-Attribut beinhaltet eine URI, die den verwendeten Hashalgorithmus identifiziert, z. B.:

- <http://www.w3.org/2001/04/xmlenc#sha512>, or
- <http://www.w3.org/2007/05/xmldsig-more#sha3-512>, etc.

Die Struktur eines EncryptedKey-Elements ist wie folgt definiert:

[@Algorithm] [required]

Das <Algorithm>-Attribut beinhaltet eine URI, die den Algorithmus identifiziert, der für die symmetrische Verschlüsselung verwendet wird.

3.6.2 Referenz auf ein Z-AIP

Die Rollenreferenzen für eine Zugriffssteuerung (vgl. Kap. 3.6.1) werden in einem Z-AIP gehalten (vgl. Kap. 3.6.3). Aus einem D-AIP (vgl. Kap. 3.6.4) heraus muss auf die korrespondierenden Z-AIPs verwiesen werden. Die Definition, wie ein solcher Verweis aufgebaut wird, ist der Abbildung 22 zu entnehmen.

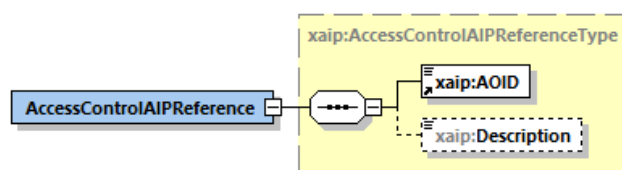


Abbildung 22: Definition des AccessControlAIPReference-Elements.

Das xaip:AccessControlAIPReference-Element enthält folgende weitere Elemente:

<AOID> [required]

Das <AOID>-Element enthält die AOID von dem korrespondierenden Z-AIP, das das entsprechende xaip:RecipientInfos-Element beherbergt.

<Description> [optional]

Das <Description>-Element kann eine für Menschen lesbare Beschreibung beinhalten.

3.6.3 Aufbau eines Z-AIP

Der Aufbau eines Z-AIP wird in der Abbildung 23 beispielhaft dargestellt.

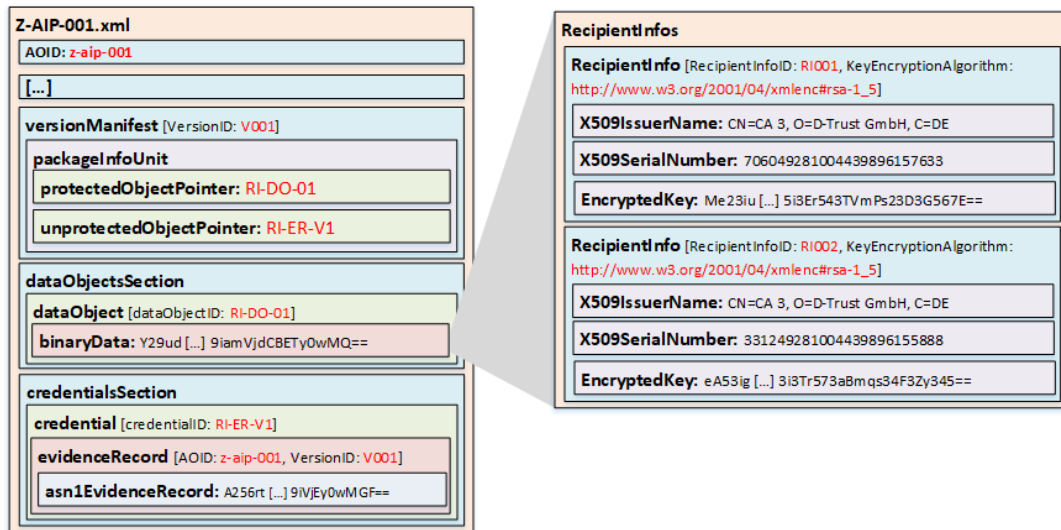


Abbildung 23: Struktur eines Z-AIP – Beispiel (vereinfachte Darstellung)

Das Beispiel-Z-AIP beinhaltet ein dataObject-Element, in dem das RecipientInfos-Element binär gespeichert worden ist. Der in einem credential-Element abgelegte Evidence Record bezieht sich auf das versionManifest-Element in der Version V001 und schützt das RecipientInfos-Element.

Das RecipientInfos-Element beinhaltet zwei RecipientInfo-Elemente, die jeweils die zugriffsberechtigte Rolle durch die X509IssuerName-Element und X509SerialNumber-Element referenzieren. In beiden Fällen weist das KeyEncryptionAlgorithm-Attribut den gleichen Wert auf, nämlich http://www.w3.org/2001/04/xmlenc#rsa-1_5, was der für die Verschlüsselung des symmetrischen Schlüssels verwendete Algorithmus beschreibt. In den jeweiligen EncryptedKey-Elementen wurde der verschlüsselte symmetrische Schlüssel abgelegt.

3.6.4 Aufbau von D-AIP

Der Aufbau eines D-AIP wird in der Abbildung 24 dargestellt.

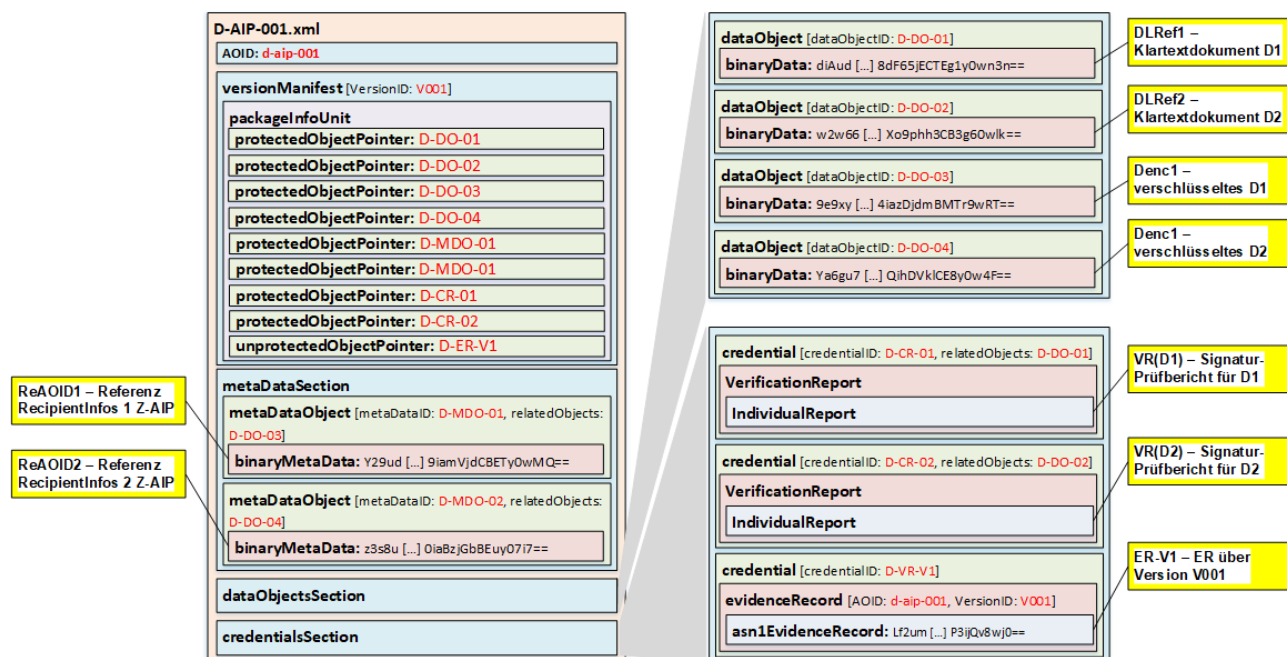


Abbildung 24: Struktur eines D-AIP – Beispiel.

Das oben dargestellte Beispiel eines D-AIP illustriert eine Ablage von zwei Klartextdokumenten D1 und D2 inkl. zusätzlicher Daten. Das D-AIP beinhaltet acht `protectedObjectPointer`-Elemente und ein `unprotectedObjectPointer`-Element.

Im `metaDataSection`-Element wurden zwei `metaDataObject`-Elemente abgelegt:

- **D-MDO-01** – beinhaltet ein `binaryMetaData`-Element, in dem die Referenz auf das zugehörige Z-AIP (vgl. Kap. 3.6.3) mit der AOID `ReAOID1` gespeichert ist. Das `relatedObjects`-Attribut referenziert das `dataObject`-Element mit dem `dataObjectID` **D-DO-03**,
- **D-MDO-02** – beinhaltet ein `binaryMetaData`-Element, in dem die Referenz auf das zugehörige Z-AIP (vgl. Kap. 3.6.3) mit der AOID `ReAOID2` gespeichert ist. Das `relatedObjects`-Attribut referenziert das `dataObject`-Element mit dem `dataObjectID` **D-DO-04**.

Das `dataObjectsSection`-Element beinhaltet vier `dataObject`-Elemente:

- **D-DO-01** – beinhaltet ein `dataObject`-Element mit der Base64-kodierten, lokalen Referenz auf das Klartextdokument D1, eingebettet in ein `binaryData`-Element,
- **D-DO-02** – beinhaltet ein `dataObject`-Element mit der Base64-kodierten, lokalen Referenz auf das Klartextdokument D2, eingebettet in ein `binaryData`-Element,
- **D-DO-03** – beinhaltet ein `dataObject`-Element mit der Base64-kodierten Verschlüsselung `Denc1` des Klartextdokuments D1, eingebettet in ein `binaryData`-Element,
- **D-DO-04** – beinhaltet ein `dataObject`-Element mit der Base64-kodierten Verschlüsselung `Denc2` des Klartextdokuments D2, eingebettet in ein `binaryData`-Element.

Das `credentialsSection`-Element beinhaltet drei `credential`-Elemente:

- **D-CR-01** – beinhaltet ein `VerificationReport`-Element, das einen Prüfbericht zu(r) Signatur(en), die das Klartextdokument D1 beinhaltet, enthält,
- **D-CR-02** – beinhaltet ein `VerificationReport`-Element, das einen Prüfbericht zu(r) Signatur(en), die das Klartextdokument D2 beinhaltet, enthält,
- **D-ER-V1** – beinhaltet ein `evidenceRecord`-Element, das die Version `V001` dieses LXAIP mit der AOID `d-aip-001` schützt und in ein `asn1EvidenceRecord`-Element eingebettet ist.

3.6.5 Referenz auf das Klartextdokument (DLRef)

Der Aufbau der beim Hochladen der Klartextdokumente lokalen Referenzen ist in diesem Profil offengehalten und die konkrete Ausgestaltung dem Hersteller überlassen. Weiterhin wird an der Stelle auf [TR-ESOR-F], Kap. 3.2.6 verwiesen.

Darüber hinaus muss folgende Anforderung an die Referenz erfüllt werden:

- A.19** Es muss sichergestellt werden, dass das zentrale Krypto-Modul anhand der Referenz den Referenztyp erkennen (z. B. durch die geeignete Belegung des `MimeType`-Attributs im `asic:DataObjectReference-Element`) sowie das zuständige lokale Krypto-Modul identifizieren kann.

4 Abkürzungsverzeichnis

Abkürzung	Auflösung
AIP	Archive Information Package
AOID	Archive Object Identifier
D	Klartextdokument
D-AIP	Dokument-AIP
Denc	Verschlüsseltes Klartextdokument
DLRef	Lokale Referenz auf das Klartextdokument
Kenc	Symmetrischer Schlüssel
Kpriv	Privater asymmetrischer Schlüssel
Kpub	Öffentlicher asymmetrischer Schlüssel
NfD	Nur für den Dienstgebrauch
ReAOID	Eine einem Z-AIP zugehörige AOID
ReXAIP	Ein die Zugriffsberechtigungen enthaltendes XAIP
VDA	Vertrauensdiensteanbieter
VS	Verschlusssache
XAIP	XML-base AIP
LXAIP	Logisches XAIP
Z-AIP	Zugriffsberechtigung-AIP

5 Anhang – Umgang mit VS-NfD eingestufte Information

5.1 Einführung

In diesem Anhang wird ein spezieller Anwendungsfall in der öffentlichen Verwaltung betrachtet. Eine Behörde betreibt eine Fachanwendung in der gem. VSA als VS-NfD eingestufte Informationen verarbeitet werden. Diese Informationen sollen in einem [TR-ESOR]-System beweiswerterhaltend bis zum Ablauf der Aufbewahrungsfrist gespeichert werden. Das [TR-ESOR]-System selbst ist nicht für die Verarbeitung von VS-NfD eingestuften Informationen freigegeben (keine VS-Software).

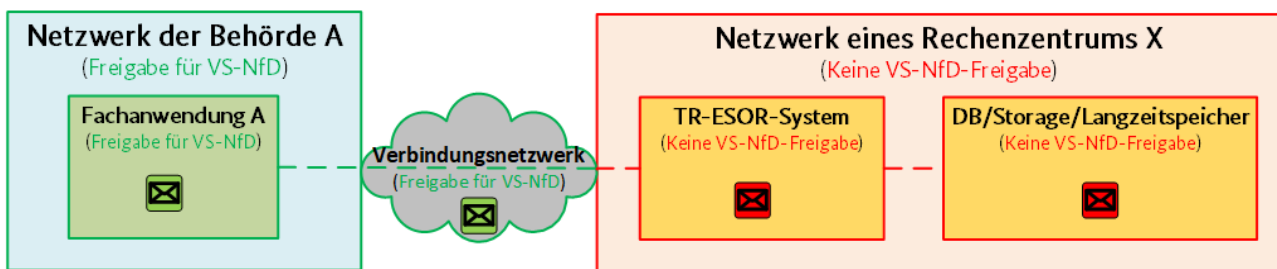


Abbildung 25: Beispielhafter Anwendungsfall: Speicherung VS-NfD-Information in einem nicht VS-TR-ESOR.

Gemäß [TR-ESOR-ENC] (vgl. Kap. 3) kann als ein erster Ansatz die Lösungsarchitektur in der Abbildung 26 herangezogen werden. Dies impliziert, dass es eine lokale TR-ESOR-Middleware (vgl. Kap. 3.2) im dem VS-NfD-Bereich der Behörde A gibt, die selbst auch für die Verarbeitung der VS-NfD-eingestuften Informationen zugelassen worden ist.

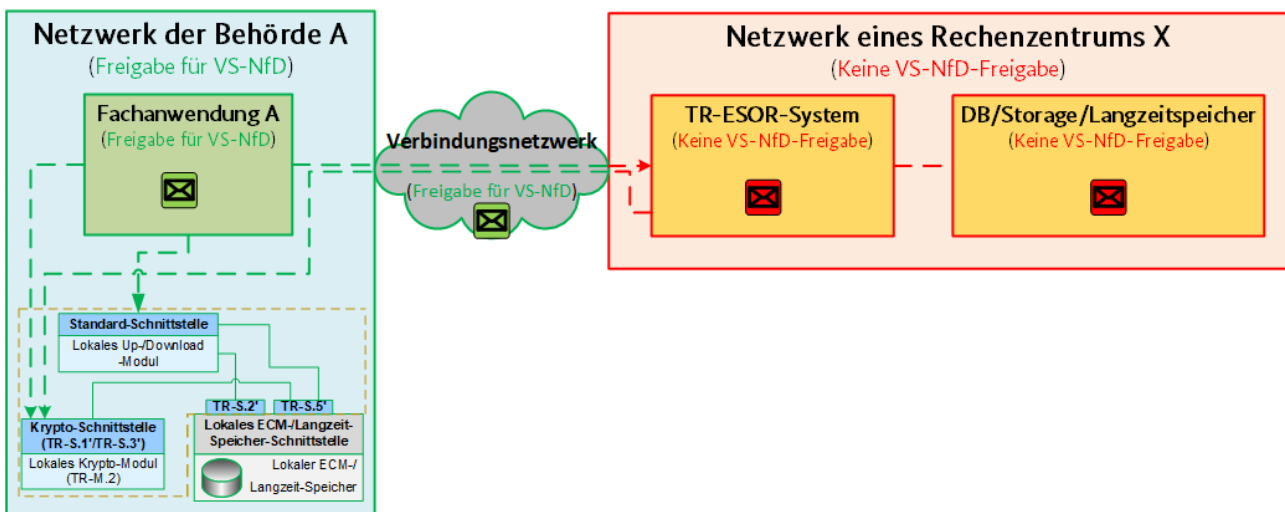


Abbildung 26: Speicherung VS-NfD-Information in einem nicht VS-TR-ESOR gem. [TR-ESOR-ENC] - Standardansatz.

Die auf diesem Ansatz basierenden Abläufe sind direkt dem Kap. 3.4 zu entnehmen und bedürfen keiner Anpassung. Der aufwendigste Teil der Umsetzung ist die Herbeiführung einer VS-NfD-Zulassung für die lokale TR-ESOR-Middleware. Sowohl das lokale Up-/Download-Modul als auch das lokale Krypto-Modul, aber auch der lokale ECM-/Langzeitspeicher, müssen für die Verarbeitung von VS-NfD-Informationen geprüft und freigegeben worden sein (VS-IT).

Um den Umsetzungsaufwand zu reduzieren, kann die Anzahl der Komponenten und somit die Komplexität der lokalen Middleware deutlich reduziert werden. Dieser „reduzierte“ Ansatz gem. [TR-ESOR-ENC] für eine

Lösungsarchitektur ist in der Abbildung 27 dargestellt. Dieser Ansatz kann zur Geltung kommen, falls die Haltung der Klartextdokumente innerhalb der lokalen Middleware nicht notwendig ist, da diese anderweitig (z. B. innerhalb der Fachanwendung) oder gar nicht mehr lokal gehalten werden.

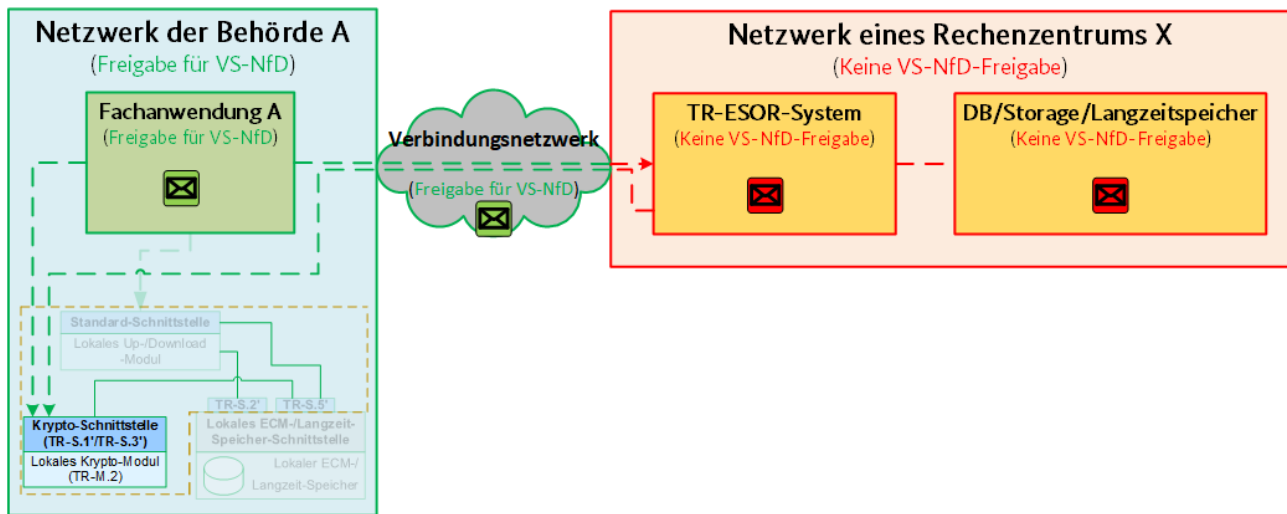


Abbildung 27: Speicherung VS-NfD-Information in einem nicht VS-TR-ESOR gem. [TR-ESOR-ENC], reduzierter Ansatz.

In dem Falle entfallen sowohl das lokale Up-/Download-Modul als auch der lokale ECM-/Langzeitspeicher gänzlich. Das übriggebliebene lokale Krypto-Modul muss aber nach wie vor über eine VS-NfD-Zulassung verfügen, deren Herbeiführung mit einem erheblich reduzierten Aufwand gegenüber dem Standardansatz erfolgen kann.

Hinweis!

Alternativ könnte die lokale TR-ESOR-Middleware, anstatt wie in der Abbildung 26 und in der Abbildung 27 dargestellt, direkt im Netzwerk der Behörde in einem durch die Behörde beauftragten Rechenzentrum platziert werden. Die zusätzlichen Grundvoraussetzungen dafür wären die VS-NfD-Zulassung des Rechenzentrums und ein entsprechend VS-NfD-zugelassenes Verbindungsnetzwerk zwischen der Behörde und dem Rechenzentrum.

5.2 Architektur (VS-NfD reduziert)

Der „reduzierte“ Ansatz führt zu kleinen Änderungen in der in der Abbildung 5 und in der Abbildung 6 dargestellten Gesamtarchitektur (entsprechend bezogen auf **TR-S.4** und **TR-S.512**). Die „reduzierte“ IT-Referenzarchitektur mit **TR-S.4** ist der Abbildung 28 zu entnehmen.

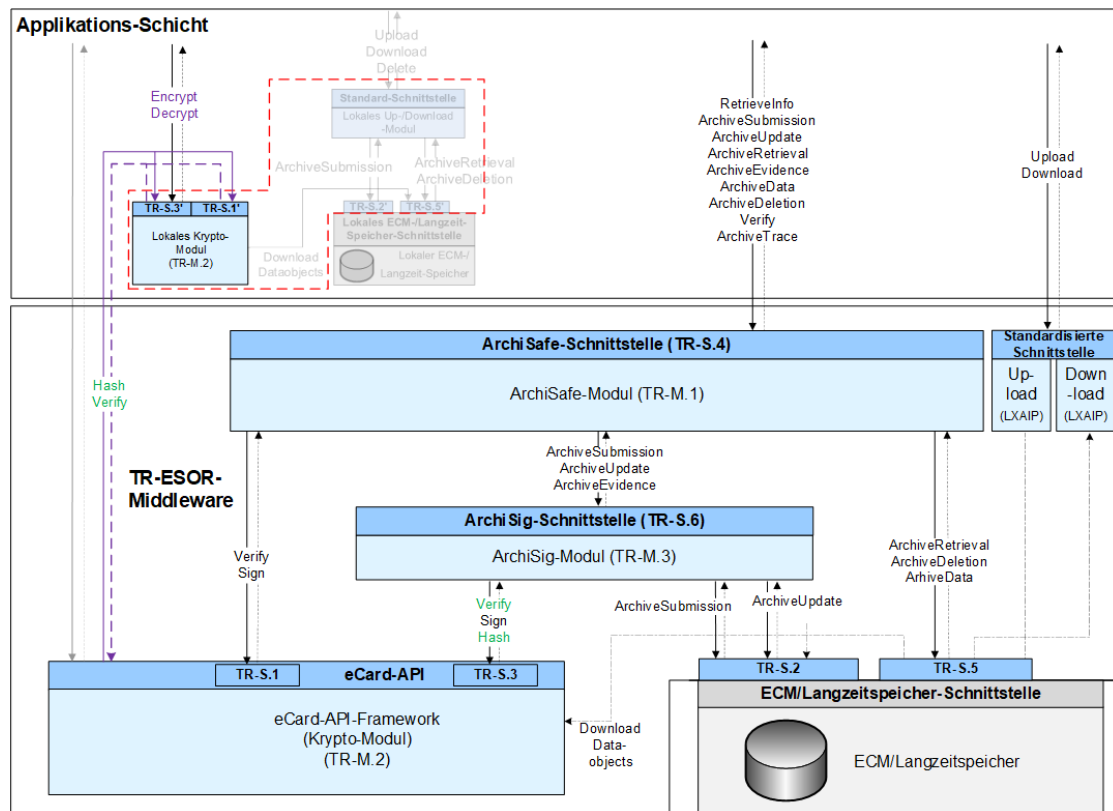


Abbildung 28: Schematische Darstellung der IT-Referenzarchitektur mit TR-S.4 gem. [TR-ESOR-ENC], reduziert.

Die „reduzierte“ IT-Referenzarchitektur mit TR-S.512 ist der Abbildung 29 zu entnehmen.

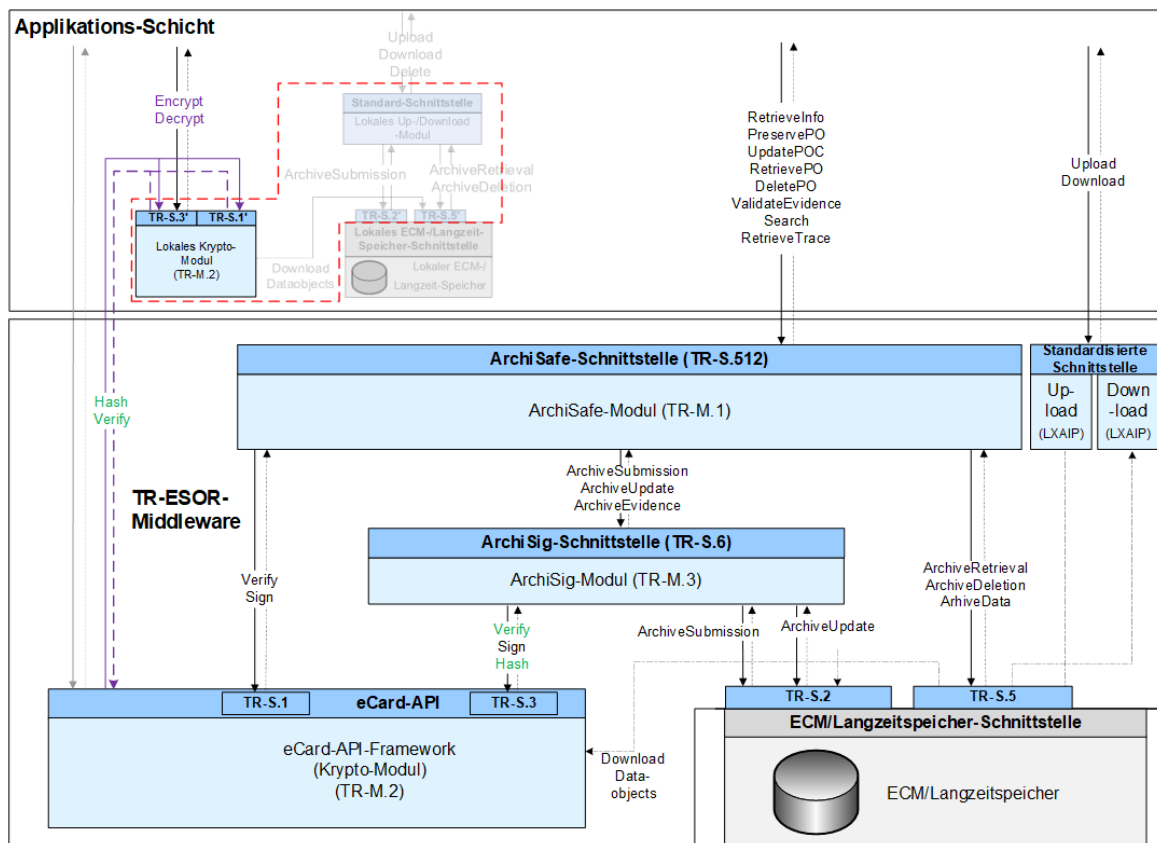


Abbildung 29: Schematische Darstellung der IT-Referenzarchitektur mit TR-S.512 gem. [TR-ESOR-ENC], reduziert.

5.3 Prozesse (VS-NfD reduziert)

5.3.1 Ablage elektronischer Unterlagen (VS-NfD reduziert)

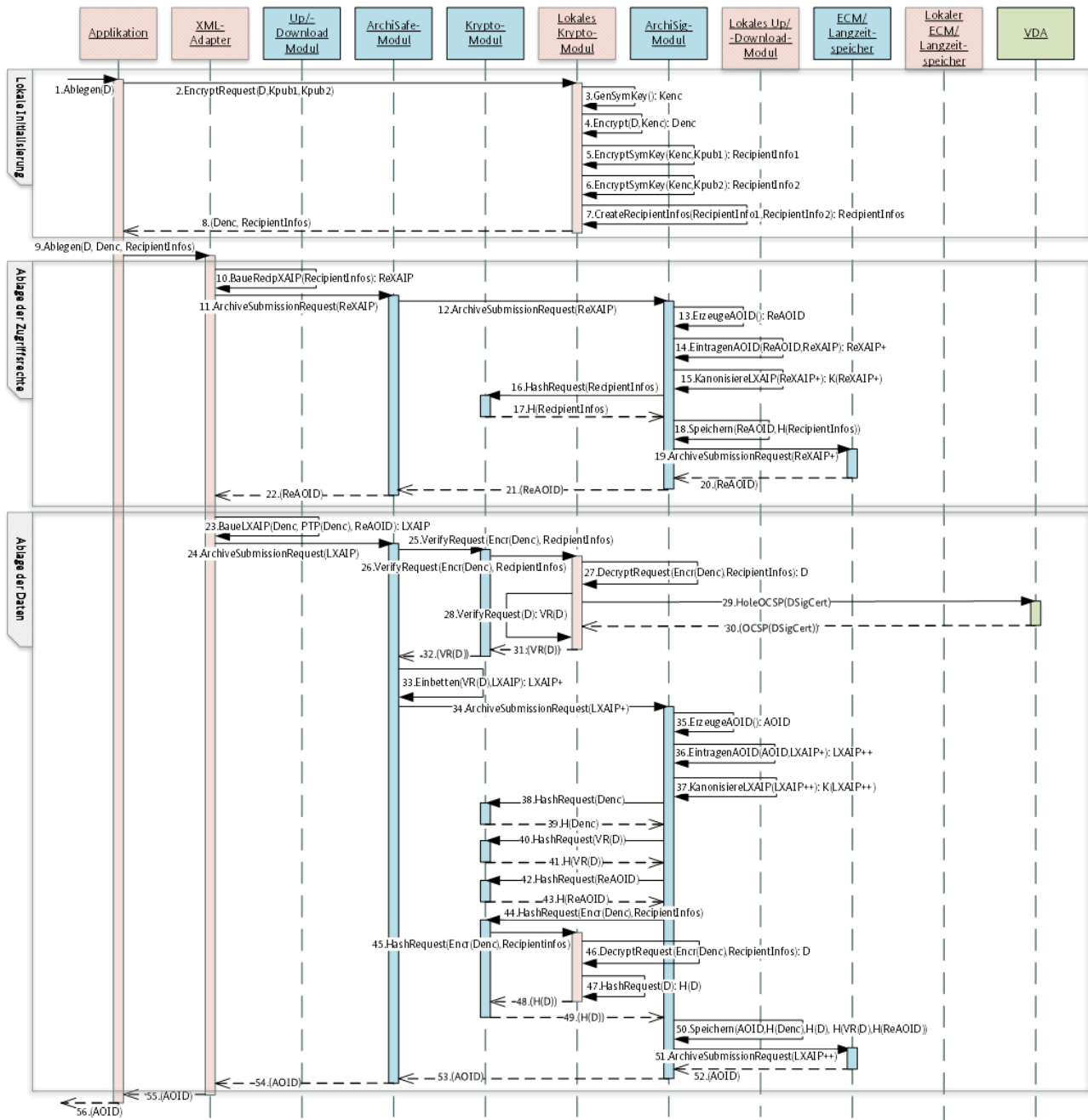


Abbildung 30: Ablage gem. [TR-ESOR-ENC] (VS-NfD reduziert).

Die nachfolgende Tabelle 12 beschreibt die Änderungen in der Abbildung 30 in Bezug auf das ursprünglich in der Abbildung 8 definierte Verhalten bei der Ablage gem. [TR-ESOR-ENC].

Ursprungsschritte	Zielschritte	Änderungsbeschreibung
1 – 6	-	Die Schritte entfallen vollumfänglich, da es keinen lokalen Speicher gibt.
28	23	Die Referenz <code>DLRef</code> wird durch die Referenz <code>PTP(Denc)</code> (vgl. Kap. 5.5.2) ersetzt.

Ursprungsschritte	Zielschritte	Änderungsbeschreibung
30, 31	25, 26	Anstatt von <code>DLRef</code> wird <code>Encr(Denc)</code> ²⁴ mit <code>RecipientInfos</code> übergeben.
50, 51	44, 45	Zusätzlich zur <code>Encr(Denc)</code> -Struktur, die das verschlüsselte Klartextdokument <code>D</code> beinhaltet, muss die zugehörige <code>RecipientInfos</code> -Struktur in der zentralen Middleware aus dem korrespondierenden <code>ReXAIP</code> ermittelt (vor Schritt 25 und 44) und an das lokale Krypto-Modul übergeben werden. Auf diese Weise kann das lokale Krypto-Modul den passenden asymmetrischen privaten Schlüssel (z. B. <code>Kpriv1</code>) ermitteln und den symmetrischen Verschlüsselungsschlüssel (<code>Kenc</code>) entschlüsseln, um damit den Klartextinhalt aus dem <code>Denc</code> zu gewinnen.
32, 33	27	Das Klartextdokument <code>D</code> wird nicht aus dem Speicher, sondern durch die Entschlüsselung von <code>Denc</code> gewonnen.
52, 53	46	

Tabelle 12: Änderungen in der Abbildung 30 bezogen auf die Abbildung 8.

²⁴ Im LXAIP wird mit Hilfe vom `PTP(Denc)` (vgl. Kap. 5.5.2) eine Referenz auf das verschlüsselte Dokument definiert und damit der Hashwert über den dazugehörigen Klartextinhalt angefordert, wofür das Parameter `Encr(Denc)` (vgl. z. B. Kap. 5.4.1.1 oder Kap. 5.4.2.1) erzeugt wird.

5.3.2 Abfrage der bewahrten Daten (VS-NfD reduziert)

Der Ablauf gem. [TR-ESOR-ENC] aus dem Kap. 3.4.2 wird ohne Änderungen angewandt.

5.3.3 Ändern der bewahrten Daten (VS-NfD reduziert)

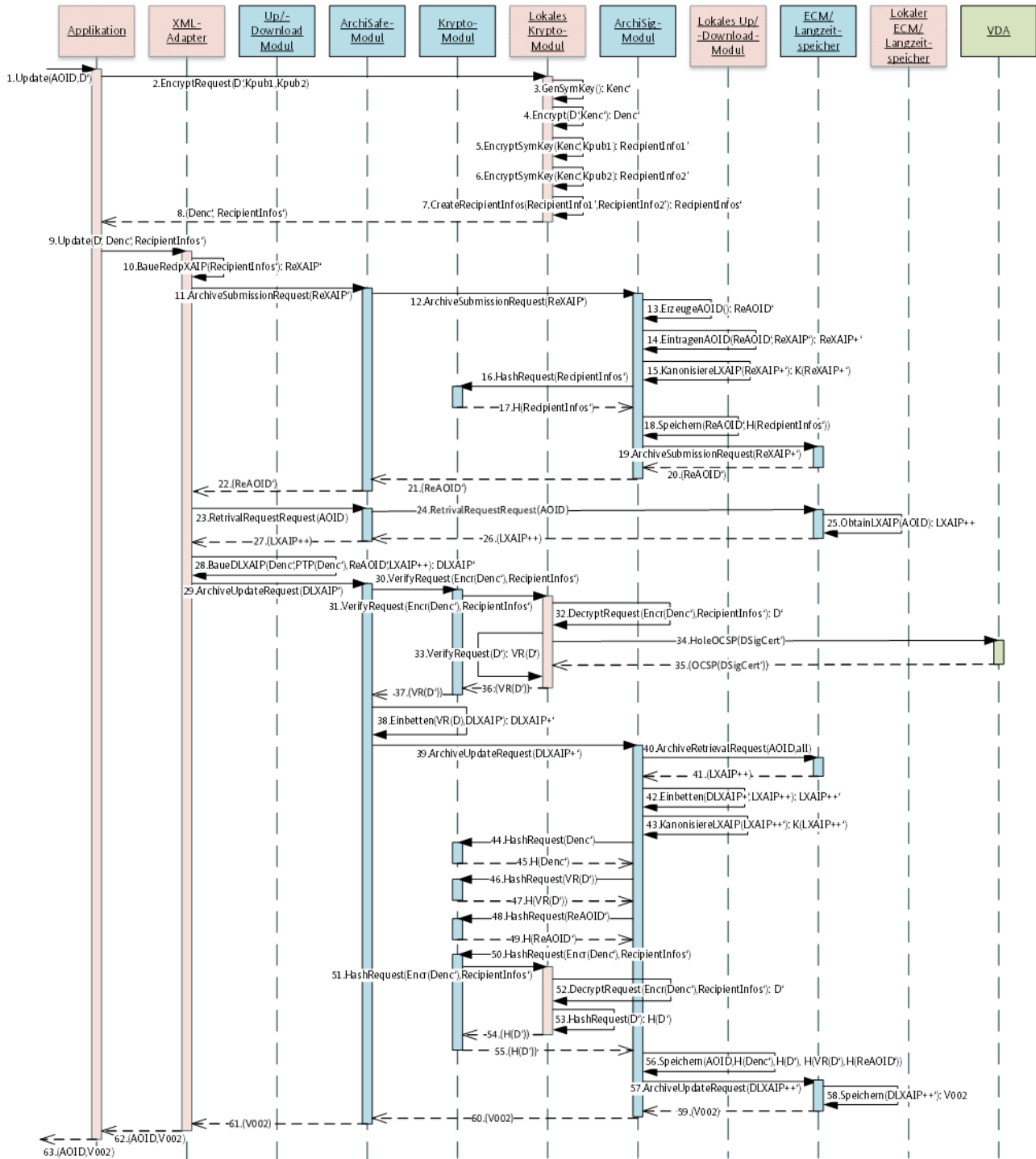


Abbildung 31: Änderung gem. [TR-ESOR-ENC] (VS-NfD reduziert).

Die nachfolgende Tabelle 13 beschreibt die Änderungen in der Abbildung 31 in Bezug auf das ursprünglich in der Abbildung 10 definierte Verhalten bei der Ablage gem. [TR-ESOR-ENC].

Ursprungsschritte	Zielschritte	Änderungsbeschreibung
23 – 27	-	Die Schritte entfallen vollumfänglich, da es keinen lokalen Speicher gibt.

Ursprungsschritte	Zielschritte	Änderungsbeschreibung
33	28	Die Referenz $DLRef'$ wird durch die Referenz $PTP(Denc')$ ²⁴ ersetzt.
35, 36	30, 31	Anstatt von $DLRef'$ wird $Encr(Denc')$ mit $RecipientInfos'$ übergeben.
56, 57	50, 51	
37, 38	32	Das Klartextdokument D' wird nicht aus dem Speicher, sondern durch die Entschlüsselung von $Denc'$ gewonnen.
58, 59	52	

Tabelle 13: Änderungen in der Abbildung 31 bezogen auf die Abbildung 10.

5.3.4 Rückgabe der technischen Beweisdaten (VS-NfD reduziert)

Der Ablauf gem. [TR-ESOR-ENC] aus dem Kap. 3.4.4 wird ohne Änderungen angewandt.

5.3.5 Prüfen der beweisrelevanten Daten und technischen Beweisdaten (VS-NfD reduziert)

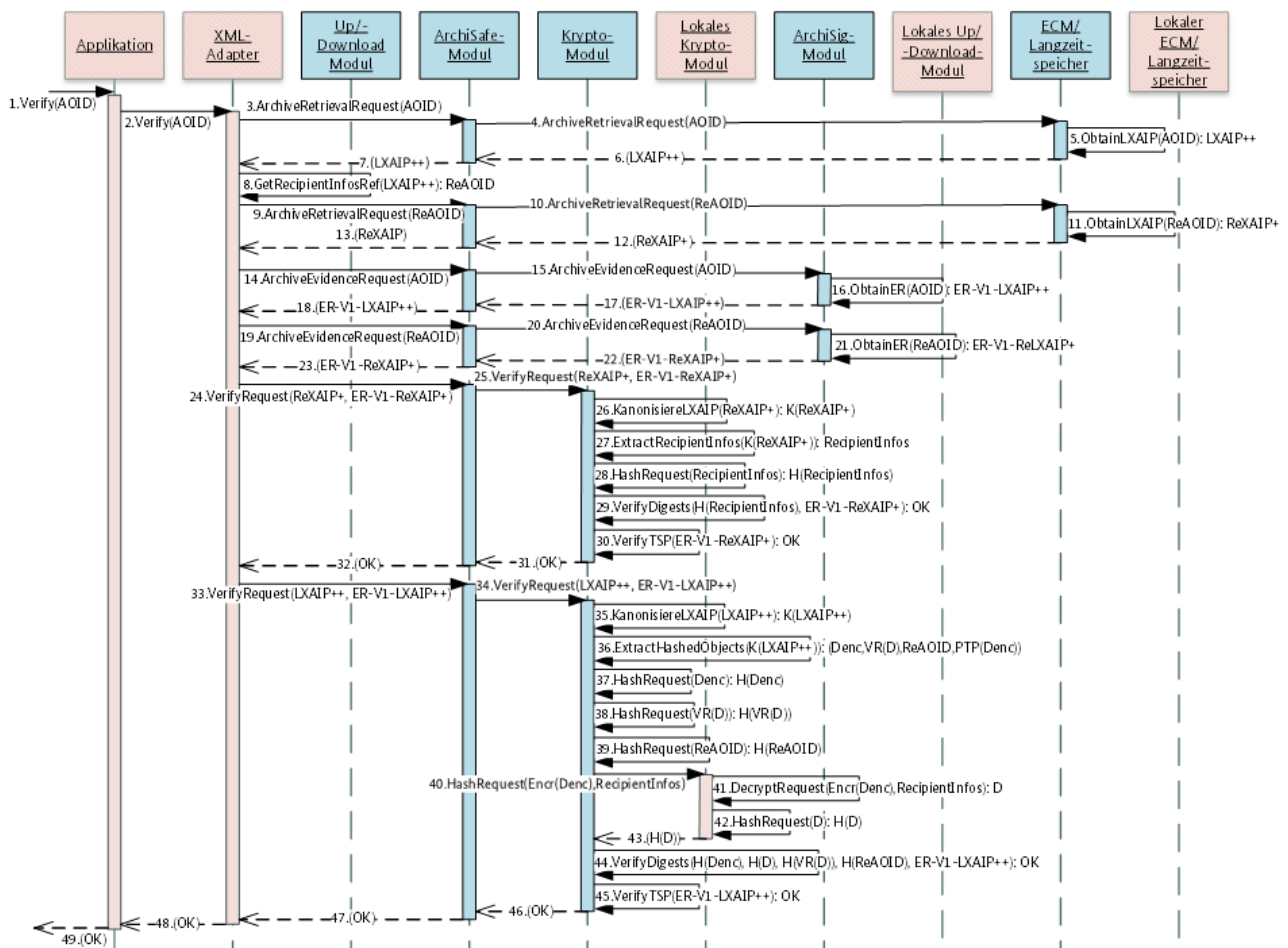


Abbildung 32: Verifikation der Beweisdaten gem. [TR-ESOR-ENC] (VS-NfD reduziert).

Die nachfolgende Tabelle 14 beschreibt die Änderungen in der Abbildung 32 in Bezug auf das ursprünglich in der Abbildung 11 definierte Verhalten bei der Verifikation gem. [TR-ESOR-ENC].

5.3.6 Vernichten von Archivinformationspaketen, inkl. Daten (VS-NfD reduziert)



Ursprungsschritte	Zielschritte	Änderungsbeschreibung
19 26 – 37	-	Die Schritte entfallen vollumfänglich, da es keinen lokalen Speicher gibt.

91

5.3.7 Erneuerung des Hashbaums (VS-NfD reduziert)

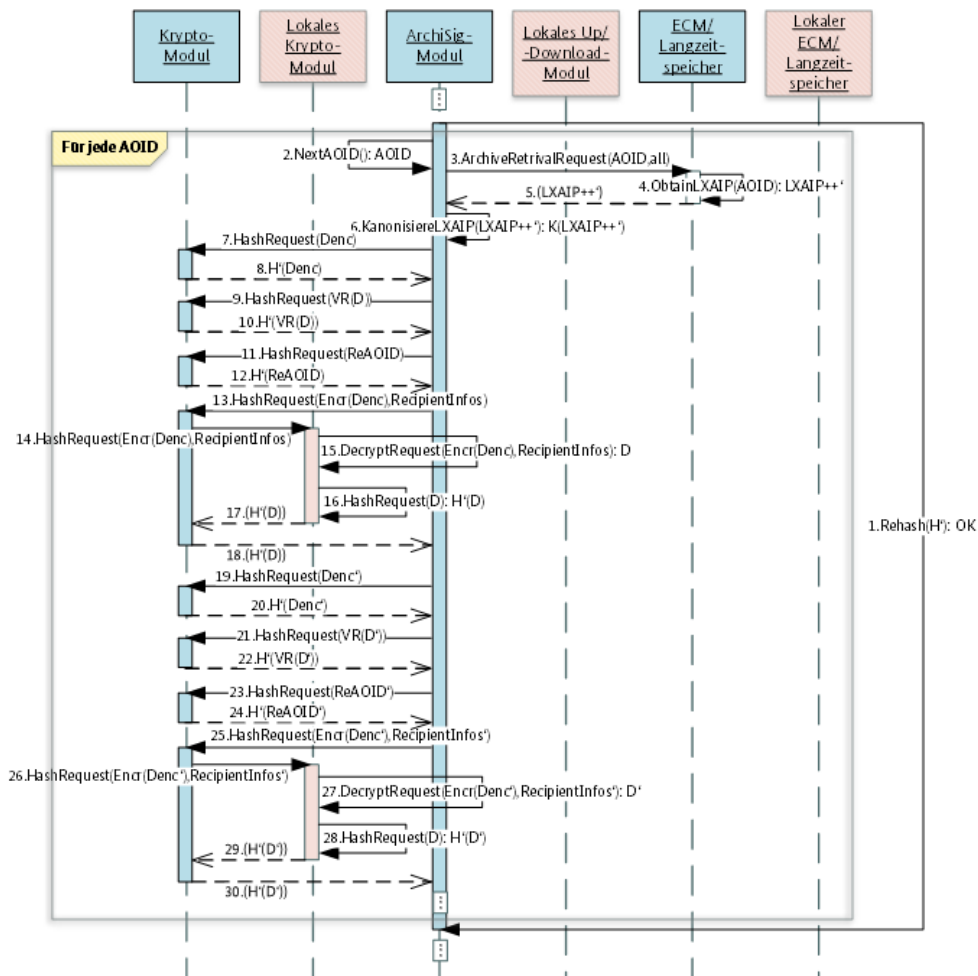


Abbildung 34: Hashwertberechnung bei Hashbaumerneuerung gem. [TR-ESOR-ENC] (VS-NfD reduziert).

Die nachfolgende Tabelle 16 beschreibt die Änderungen in der Abbildung 34 in Bezug auf das ursprünglich in der Abbildung 13 definierte Verhalten bei der Ablage gem. [TR-ESOR-ENC].

Ursprungsschritte	Zielschritte	Änderungsbeschreibung
13, 14	13, 14	Anstatt von DLRef wird $\text{Encr}(Denc)^{24}$ mit RecipientInfos übergeben.
26, 27	25, 26	Anstatt von DLRef' wird $\text{Encr}(Denc')^{24}$ mit RecipientInfos' übergeben.
15, 16	15	Das Klartextdokument D wird nicht aus dem Speicher, sondern durch die Entschlüsselung von $Denc$ gewonnen.
28, 29	27	Das Klartextdokument D' wird nicht aus dem Speicher, sondern durch die Entschlüsselung von $Denc'$ gewonnen.

Tabelle 16: Änderungen in der Abbildung 34 bezogen auf die Abbildung 13.

5.3.8 Erneuerung des Schlüsselmaterials (VS-NfD reduziert)

5.3.8.1 Erneuerung der symmetrischen Verschlüsselungsschlüssel (VS-NfD reduziert)

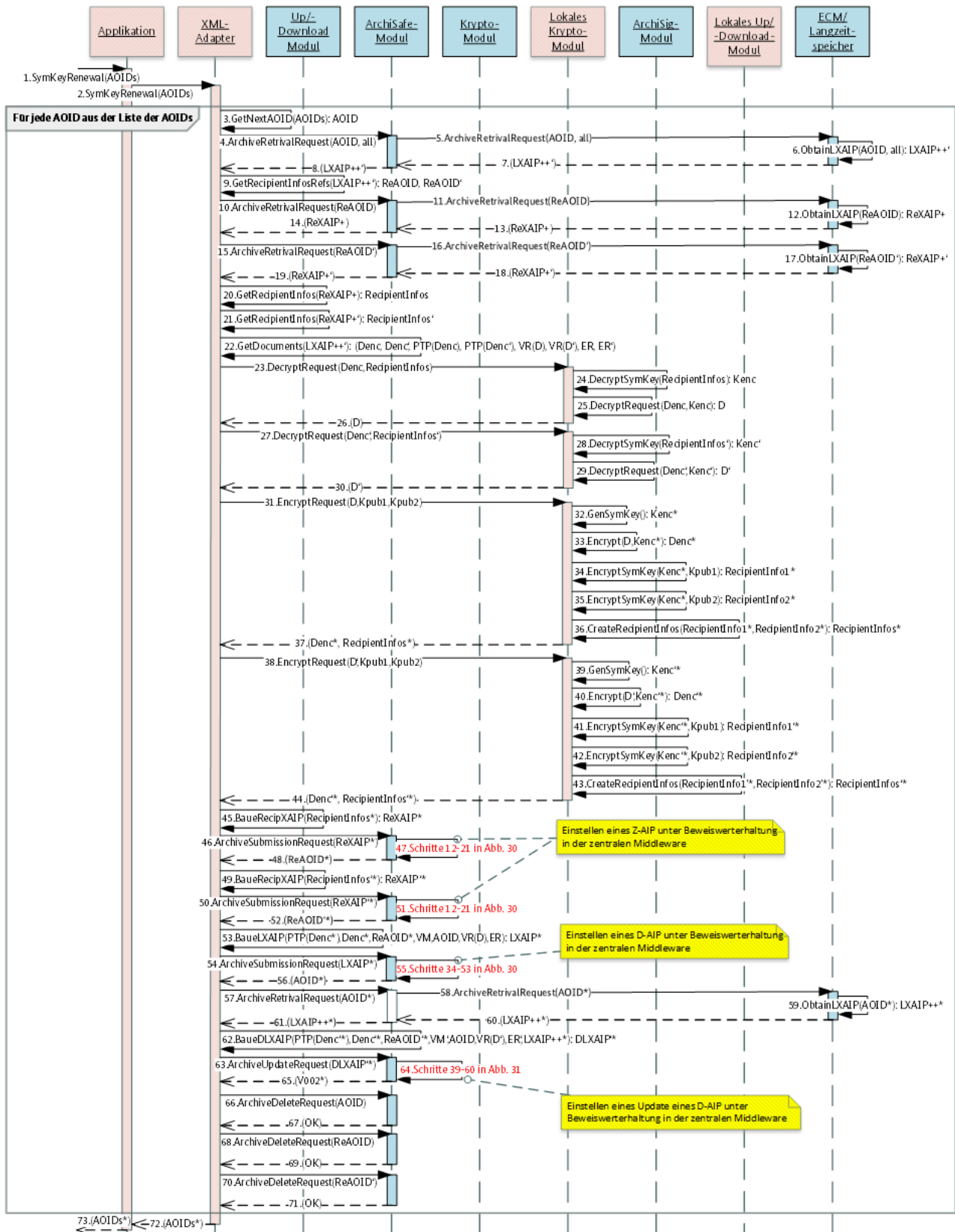


Abbildung 35: Erneuerung der symmetrischen Schlüssel gem. [TR-ESOR-ENC] (VS-NfD reduziert).

Die nachfolgende Tabelle 17 beschreibt die Änderungen in der Abbildung 35 in Bezug auf das ursprünglich in der Abbildung 15 definierte Verhalten bei der Ablage gem. [TR-ESOR-ENC].

Ursprungsschritte	Zielschritte	Änderungsbeschreibung
22	22	Anstatt von $DLRef$ und $DLRef'$ werden $PTP(Denc)^{24}$ und $PTP(Denc')^{24}$ übergeben.
53	53	Anstatt von $DLRef$ wird $PTP(Denc)^{24}$ übergeben.
62	62	Anstatt von $DLRef'$ wird $PTP(Denc')^{24}$ übergeben.
47, 51	47, 51	Von „Schritte 17-26 in Abb. 8“ in „Schritte 12-21 in Abb. 30“ geändert.
55	55	Von „Schritte 40-60 in Abb. 8“ in „Schritte 34-53 in Abb. 30“ geändert.
64	64	Von „Schritte 45-67 in Abb. 10“ in „Schritte 39-60 in Abb. 31“ geändert.

Tabelle 17: Änderungen in der Abbildung 35 bezogen auf die Abbildung 15.

5.3.8.2 Erneuerung der asymmetrischen Zugriffsberechtigungsschlüssel (VS-NfD reduziert)

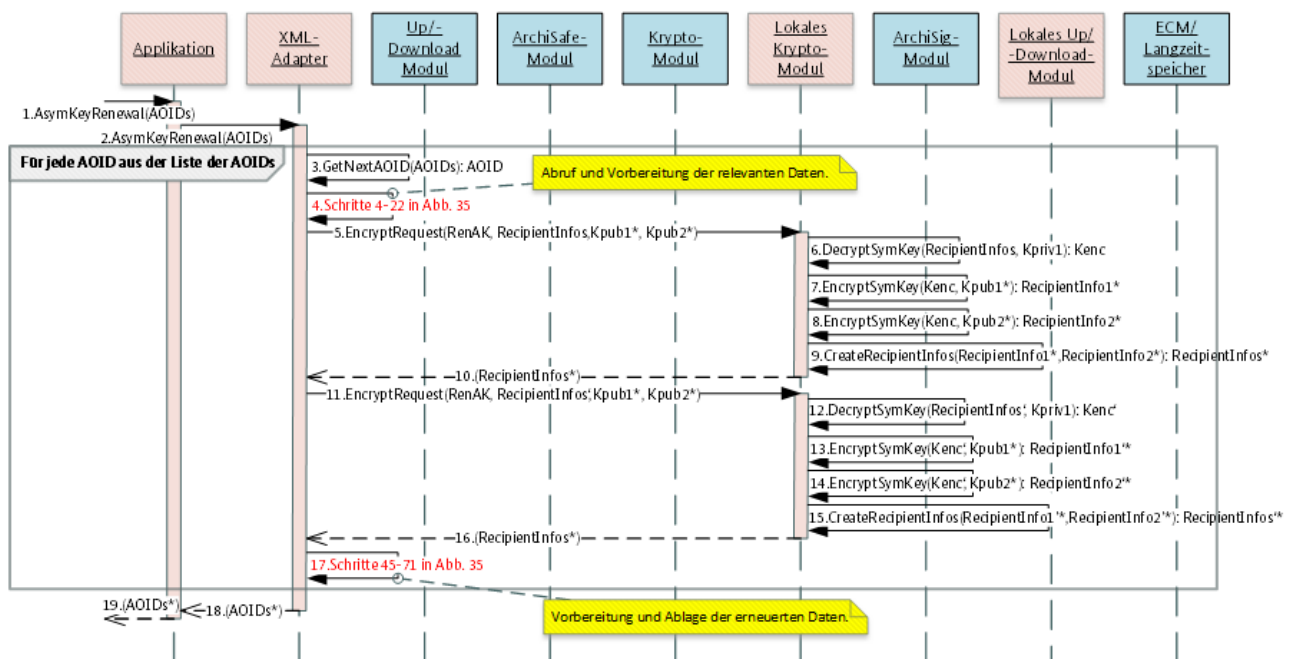


Abbildung 36: Erneuerung der asymmetrischen Schlüssel gem. [TR-ESOR-ENC] (VS-NfD reduziert).

Die nachfolgende Tabelle 18 beschreibt die Änderungen in der Abbildung 36 in Bezug auf das ursprünglich in der Abbildung 16 definierte Verhalten bei der Ablage gem. [TR-ESOR-ENC].

Ursprungsschritte	Zielschritte	Änderungsbeschreibung
4	4	Von „Schritte 4-22 in Abb. 15“ in „Schritte 4-22 in Abb. 35“ geändert.
17	17	Von „Schritte 45-71 in Abb. 15“ in „Schritte 45-71 in Abb. 35“ geändert.

Tabelle 18: Änderungen in der Abbildung 36 bezogen auf die Abbildung 16.

5.3.9 Änderung der Zugriffsberechtigung inkl. Entfernung der alten Zugriffsberechtigung (VS-NfD reduziert)

5.3.9.1 Erweiterung der Zugriffsberechtigung (VS-NfD reduziert)

Der Ablauf gem. [TR-ESOR-ENC] aus dem Kap. 3.4.9.1 wird ohne Änderungen angewandt.

5.3.9.2 Einschränkung der Zugriffsberechtigung (VS-NfD reduziert)

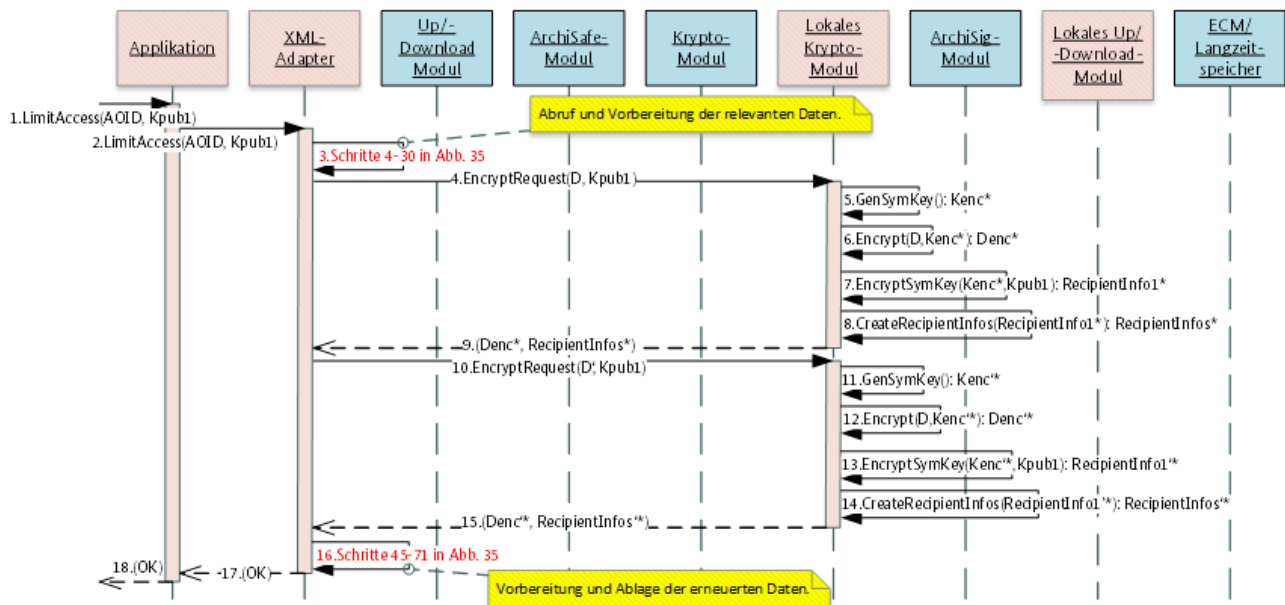


Abbildung 37: Einschränkung der Zugriffsberechtigung gem. [TR-ESOR-ENC] (VS-NfD reduziert).

Die nachfolgende Tabelle 19 beschreibt die Änderungen in der Abbildung 37 in Bezug auf das ursprünglich in der Abbildung 18 definierte Verhalten bei der Ablage gem. [TR-ESOR-ENC].

Ursprungsschritte	Zielschritte	Änderungsbeschreibung
3	3	Von „Schritte 4-30 in Abb. 15“ in „Schritte 4-30 in Abb. 35“ geändert.
16	16	Von „Schritte 45-71 in Abb. 15“ in „Schritte 45-71 in Abb. 35“ geändert.

Tabelle 19: Änderung in der Abbildung 37 bezogen auf die Abbildung 17.

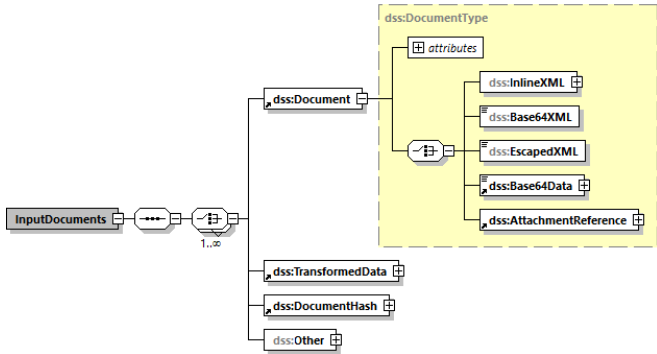
5.4 Schnittstellen (VS-NfD reduziert)

5.4.1 Die Schnittstelle TR-S.1' (VS-NfD reduziert)

5.4.1.1 Prüfung von elektronischen Signaturen, Siegel, Zeitstempel etc. (VS-NfD reduziert)

Um die Prüfung der elektronischen Signaturen, Siegel, Zeitstempel etc., die in verschlüsselten Teilen enthalten sind, zu realisieren, müssen die verschlüsselte Teile an das korrespondierende lokale Krypto-Modul übertragen und die Prüfung dort nach der zuvor stattgefundenen Entschlüsselung erfolgen. Das Ergebnis der Prüfung in der Form eines Verifikationsprotokolls (inkl. Prüfdaten) wird an das zentrale Krypto-Modul geliefert.

5.4.1.1.1 VerifyRequest (VS-NfD reduziert)

Name	<i>VerifyRequest</i>
Beschreibung	Mit Hilfe der <i>VerifyRequest</i> -Anfrage wird eine Prüfung der potentiell im Klartextdokument vorhandenen, elektronischen Signaturen, Siegel, Zeitstempeln etc. im korrespondierenden lokalen Krypto-Modul initiiert.
Details	Beschreibung Das zu prüfende verschlüsselte Dokument wird als <code>enc:EncryptedDocument</code>-Element im <code>dss:InlineXML</code>-Element des <code>dss:Document</code>-Elements des <code>dss:InputDocuments</code>-Element übergeben. 

5.4.1.1.2 VerifyResponse (VS-NfD reduziert)

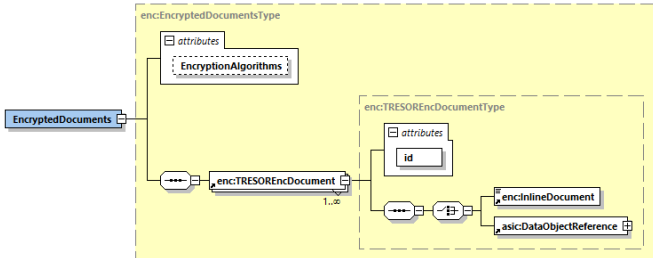
Keine Abweichungen gegenüber dem [TR-ESOR-E], Kap. 5.1.1, bezogen auf *VerifyResponse*-Antwort.

5.4.2 Die Schnittstelle TR-S.3' (VS-NfD reduziert)

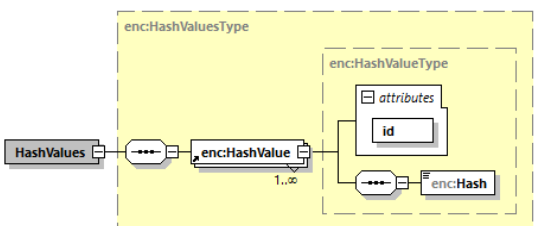
5.4.2.1 Berechnung eines Hashwerts (VS-NfD reduziert)

Für die Berechnung eines Hashwerts über die Klartextinformation D eines verschlüsselten Dokuments D_{enc} muss das verschlüsselte Dokument an das korrespondierende lokale Krypto-Modul übergeben werden, darin entschlüsselt werden, der Hashwert über die entschlüsselte Information berechnet werden und an das zentrale Krypto-Modul zurückgeliefert werden. Falls die Berechnung mehrere verschlüsselte Dokumente involviert, wäre eine Möglichkeit, diese auf einmal zu übertragen, aus Performancegründen von Vorteil. Aus dem Grund wird die im [TR-ESOR-E], Kap. 5.3.3 beschriebene Definition der TR-S.3 Schnittstelle für die TR-S.3' leicht erweitert.

5.4.2.1.1 HashRequest (VS-NfD reduziert)

Name	HashRequest
Beschreibung	Mit Hilfe der HashRequest-Anfrage werden für die Klartextdaten der übergebenen verschlüsselten Dokumente Hashwerte berechnet.
Message	Die verschlüsselten Dokumente werden als <code>enc:EncryptedDocuments</code>-Element übergeben.  Der verschlüsselte Inhalt wird im <code>enc:InlineDocument</code>-Element eines <code>enc:TRESEncDocument</code>-Elements gespeichert. Das <code>id</code>-Attribut erlaubt die Zuordnung von übergebenen Daten zu den zurückgelieferten Hashwerten.

5.4.2.1.2 HashResponse (VS-NfD reduziert)

Name	HashResponse
Beschreibung	Mit Hilfe der HashResponse-Antwort werden die für die Klartextdaten der übergebenen verschlüsselten Dokumente berechneten Hashwerte geliefert.
Message	Die berechneten Hashwerte werden als <code>enc:HashValues</code>-Element zurückgeliefert.  Jeder Hashwert wird im <code>enc:Hash</code>-Element eines zugehörigen <code>enc:HashValue</code>-Elements gespeichert. Das <code>id</code>-Attribut erlaubt die Zuordnung eines Hashwerts zu dem übergebenen, verschlüsselten Dokument.

5.5 Datenformate (VS-NfD reduziert)

5.5.1 Aufbau von D-AIP (VS-NfD reduziert)

Nachfolgende Abbildung 38 stellt ein Beispiel eines D-AIP, das zwei `enc:PlainTextProxy`-Elemente (vgl. Kap. 5.5.2) für die beiden verschlüsselten Dokumente `Denc1` und `Denc2` beinhaltet

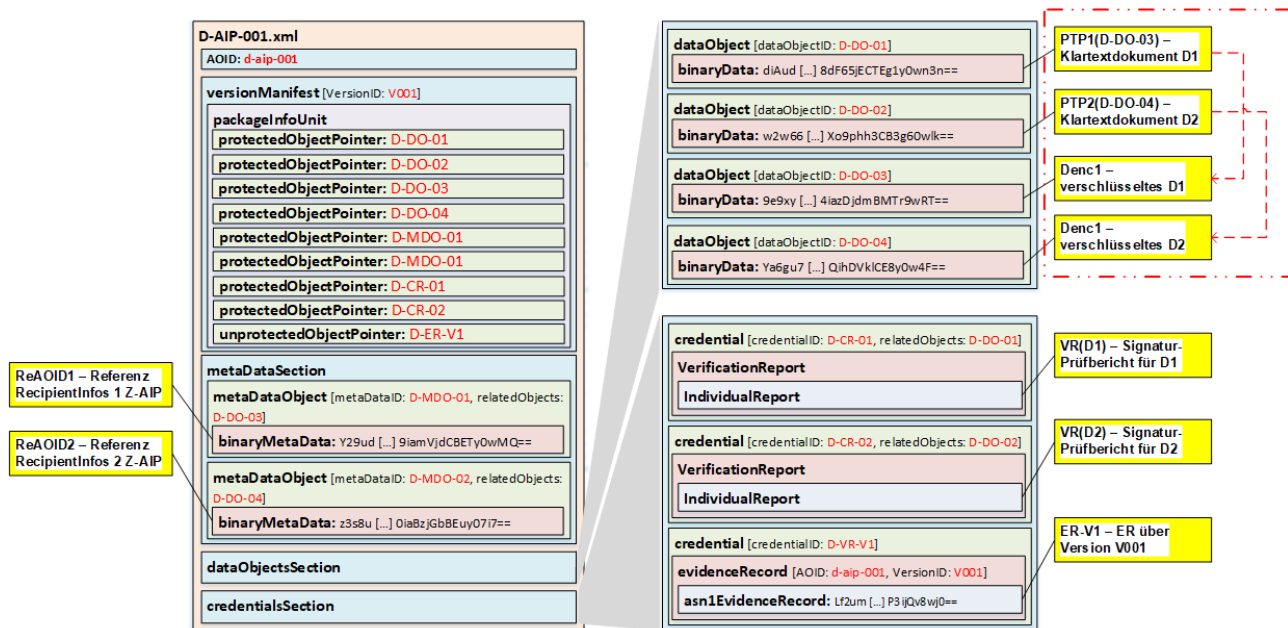


Abbildung 38: Beispiel eines D-AIP mit zwei `enc:PlainTextProxy`-Elementen (VS-NfD reduziert).

5.5.2 Referenzen auf Klartextdokumente (VS-NfD reduziert)

Um die Klartextinhalte, z. B. `D` in einem LXAIP, referenzieren zu können, muss eine Referenz auf das auch im LXAIP vorhandene, verschlüsselte Dokument, z. B. `Denc`, aufgebaut werden. Diese Referenz wird mit Hilfe von `enc:PlainTextProxy`-Element (PTP) umgesetzt, vgl. nachfolgende Abbildung 39.

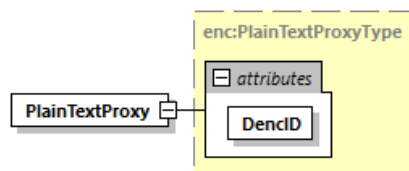


Abbildung 39: Struktur des `enc:PlainTextProxy`-Element gem. [TR-ESOR-ENC] (VS-NfD reduziert).

Das `DencID`-Attribut beinhaltet das `dataObjectID` für das korrespondierende verschlüsselte Dokument in diesem konkreten LXAIP.

6 Anhang – Beispielhafter Anwendungsfall BNotK [informativ]

6.1 Allgemeines

Das Urkundenarchiv der Notare dient der langfristigen Aufbewahrung notarieller Dokumente. Um den langfristigen Beweiswert signierter Dokumente zu sichern, kommt eine **[TR-ESOR]** konforme Lösung zum Einsatz. In Ergänzung zum Beweiswerterhalt bestehen weitere Anforderungen, die sich an die Vertraulichkeit der über einen langen Zeitraum aufzubewahrenden Dokumente richten. Ein zentrales Element ist in diesem Zusammenhang die Separation der Zugriffe zwischen den Amtstätigkeiten. Dies bedeutet, dass Dokumente nur für autorisierte Personen (Notare und deren Mitarbeiter) zugreifbar sein dürfen. Die Weiterentwicklung dieser Anforderungen führte zu dem folgenden Ansatz:

- Dokumente werden durchgängig verschlüsselt übertragen, d. h. während des Transports besteht keine Möglichkeit eines Klartextzugriffs auf die Dokumente,
- Dokumente werden ausschließlich verschlüsselt gespeichert, d. h. die aufbewahrten Dokumente sind kryptografisch gegen nicht-autorisierte Zugriffe geschützt,
- Der Zugriff auf die erforderlichen Entschlüsselungsschlüssel ist an die Rolle innerhalb der jeweiligen Amtstätigkeit gebunden. Die Schlüssel zur Dokumententschlüsselung sind nur für Personen verwendbar, die der betreffenden Amtstätigkeit zugeordnet sind. Als Sicherheitsanker dient eine Chipkarte, die zur Speicherung und Autorisierung der Schlüsselverwendung herangezogen wird.

Da in der technischen Richtlinie **[TR-ESOR]** die durchgängige Dokumentverschlüsselung im elektronischen Langzeitbewahrungssystem bis zu diesem Zeitpunkt kein Gegenstand der Betrachtungen waren, wurden erforderliche Erweiterungen definiert und umgesetzt.

6.2 Aufbau

Das System wurde hinsichtlich der **[TR-ESOR]** Referenzarchitektur um eine zusätzliche lokale Kryptografiekomponente am Arbeitsplatz erweitert. Dies bedeutet, dass das Kryptografiemodul in diesem Anwendungsfall als verteilte Anwendung verstanden wird. Im Ergebnis sind eine zentrale Middleware und eine lokale Komponente vorhanden. Das lokale Kryptografiemodul übernimmt damit die folgenden Aufgaben:

- Verifikation elektronischer Signaturen, Siegel und Zeitstempel,
- Berechnung kryptografischer Prüfsummen (Hashwerte),
- Bereitstellung hybrider Verfahren für die Ver- und Entschlüsselung von Dokumenten.

Die Client-Komponenten setzen sowohl Teile der Zugriffssteuerung auf die Klartextdokumente um und übernehmen zudem die Kommunikation mit den Archiv-Diensten, zu denen das zentrale **[TR-ESOR]** System mit den Schnittstellen und Protokollen, die von der zugehörigen technischen Richtlinie vorgegeben werden, gehört. Abbildung 40 zeigt den Aufbau des Systems in einer vereinfachten Darstellung.

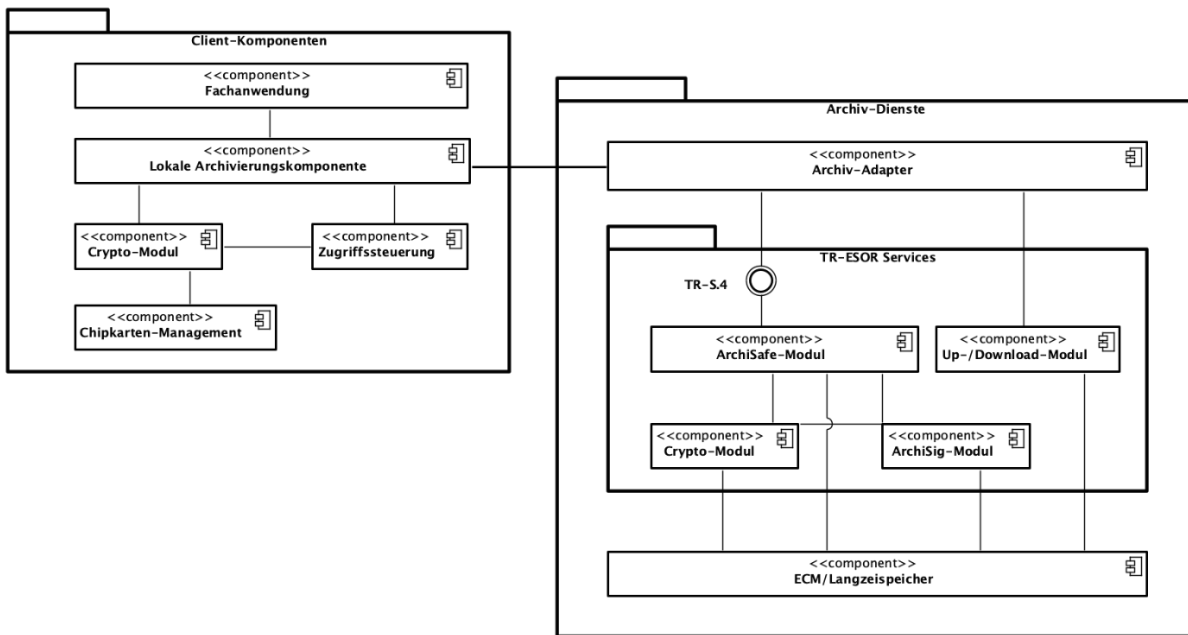


Abbildung 40: Vereinfachte Darstellung der Architektur des Urkundenarchivs der Notare

Die Client-Komponenten sind modular aufgebaut. Die lokale Fachanwendung setzt die fachlichen Anwendungsfälle des Urkundenarchivs der Notare um und nutzt eine Archivierungskomponente, die sowohl die Kommunikation mit den Archiv-Diensten als auch die Verwendung des lokalen Crypto-Moduls ermöglicht. Das Crypto-Modul regelt dabei den Zugriff auf Chipkarten, welche die in den Szenarien genutzten asymmetrischen Schlüssel verwalten.

Teil der Archivdienste ist ein Archiv-Adapter, der neben der Ansteuerung des **[TR-ESOR]** Langzeitarchivs ebenfalls Teile der Zugriffssteuerung übernimmt. Der Archiv-Adapter verwendet die Schnittstellen und Protokolle der **[TR-ESOR]** und bietet in Richtung der von der Fachanwendung verwendeten lokalen Archivierungskomponente Funktionen zur Umsetzung der für das Urkundenarchiv der Notare erforderlichen Anwendungsfälle an.

6.3 Anwendungsfälle

Im Urkundenarchiv der Notare sind neben den bereits von der **[TR-ESOR]** adressierten Anwendungsfällen, wie die Beweiswerterhaltung kryptografisch signierter Dokumente, die folgenden zusätzlichen Szenarien von Bedeutung:

- Initiale Ablage elektronischer Dokumente in verschlüsselter Form (vgl. Kap. 6.3.1),
- Abruf verschlüsselter Dokumente und Entschlüsselung durch das Notariat oder die Notarkammer (vgl. Kap. 6.3.2),
- Umschlüsselung verschlüsselter Dokumente durch das Notariat oder die Notarkammer (vgl. Kap. 6.3.3).

6.3.1 Initiale Ablage

Die initiale Ablage ist die Speicherung des Dokuments im zentralen Archiv. Das Dokument wird in einem hybriden Verfahren sowohl für die Amtstätigkeit als auch für die Notarkammer, zu welcher die Amtstätigkeit zugeordnet ist, verschlüsselt. Im Zusammenhang mit der initialen Ablage werden eventuell vorhandene eingebettete und abgesetzte Signaturen, Siegel und Zeitstempel des noch nicht verschlüsselten Dokuments geprüft und das Prüfprotokoll ebenfalls an das System zur Langzeitaufbewahrung – in Abbildung 40 als *Archiv-Dienste* bezeichnet – übergeben. Ebenso wird der Hashwert über das unverschlüsselte Dokument

berechnet und dieser in den Hashbaum aufgenommen. Mögliche vorhandene abgesetzte Signaturen werden im Klartext im Archiv gespeichert und durch den Hashbaum geschützt.

6.3.2 Abruf verschlüsselter Dokumente

Der Abruf und die Entschlüsselung aufbewahrter Dokumente bedingt, dass die Amtstätigkeit über die zugehörigen kryptografischen Schlüssel verfügt. Die kryptografischen Schlüssel sind voneinander verschieden und werden zur Authentisierung der Amtstätigkeit gegenüber der zentralen Aufbewahrungskomponente als auch für die Entschlüsselung benötigt. Das Dokument wird verschlüsselt an den Arbeitsplatz des Notars übertragen und dort unter Verwendung einer Chipkarte, welche die zugehörigen privaten Schlüssel enthält, entschlüsselt. Ein Update verschlüsselter Dokumente ist nicht vorgesehen.

6.3.3 Umschlüsselung verschlüsselter Dokumente

In manchen Fällen kann es erforderlich sein, den Besitzer des Dokuments zu ändern. Dies ist beispielsweise dann der Fall, wenn ein Notar seine Amtstätigkeit aufgibt, einen Vorgang an einen anderen Notar abgibt oder beteiligte kryptografische Algorithmen ihre Sicherheitseignung verlieren. In diesem Fall liegt die Notwendigkeit der Umschlüsselung vor, da sich der auf das Dokument Zugriffsberechtigte ändert. In diesem Falle werden die Dokumente von der Amtstätigkeit abgerufen und für den Nachfolger neu verschlüsselt. Um während der Umschlüsselung nicht das gesamte Dokument zu übertragen, erfolgt in der Archivierung eine Trennung zwischen dem asymmetrisch verschlüsselten Sitzungsschlüssel und dem symmetrisch verschlüsselten Dokument. Im Falle der Umschlüsselung wird aus Effizienzgründen lediglich der asymmetrisch verschlüsselte Sitzungsschlüssel abgerufen und mit dem neuen öffentlichen Schlüssel verschlüsselt.

6.4 Datenfluss

Soll ein elektronisches Dokument – ggf. mit Signaturen – in das Archiv übergeben werden, wird die Client-Anwendung damit beauftragt:

- Etwaige vorhandene Signaturen zu prüfen und einen OASIS-konformen Prüfbericht zu erzeugen
- Einen Hashwert über das Dokument zu berechnen
- Den zugeordneten öffentlichen Schlüssel der Amtstätigkeit aus einem Verzeichnis abzurufen
- Das Dokument mit einem hybriden Verfahren zu verschlüsseln
- Das Dokument an das Archivsystem unter Verwendung der **TR-S.4**-Schnittstelle gemäß **[TR-ESOR]** zu übergeben

Das Archivsystem nimmt die Daten entgegen und speichert diese im Langzeitspeicher. Für die langfristige Prüfbarkeit der elektronischen Signaturen ist zu beachten, dass der Hashbaum über die Hashwerte der unverschlüsselten Dokumente berechnet wird. Die Prüfung der Evidence Records erfordert daher den Abruf und die Entschlüsselung der Dokumente durch die Client-Komponente. Ebenso besteht für die Hashbaumerneuerung aufgrund drohender verlorener Sicherheitseignungen die Notwendigkeit des Zugriffs auf die unverschlüsselten Dokumente.

Der Zugriff auf die archivierten Dokumente setzt voraus, dass der Notar im Besitz des privaten Schlüssels für die Dokumententschlüsselung ist. Die asymmetrischen Schlüssel sind auf einer Chipkarte gespeichert, die Entschlüsselung des hybriden Kryptogramms erfolgt mithilfe der Chipkarte. Der Zugriff auf archivierte Dokumente zieht den folgenden Ablauf in der Client-Komponente nach sich:

- Abruf des verschlüsselten Dokuments aus dem Archiv nach vorhergehender Zugriffsberechtigung im Archiv
- Ermittlung des verwendeten Verschlüsselungsschlüssels

- Entschlüsselung des symmetrischen Sitzungsschlüssels unter Verwendung des asymmetrischen Schlüssels auf der Chipkarte

In dieser Hinsicht gleicht der Ablauf einem klassischen System für den Umgang mit verschlüsselten Dokumenten.

7 Anhang – XML-Schema-Definition

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema
  xmlns:enc="http://www.bsi.bund.de/tr-esor/enc"
  xmlns:xaip="http://www.bsi.bund.de/tr-esor/xaip"
  xmlns:tr="http://www.bsi.bund.de/tr-esor/api/1.3"
  xmlns:xs="http://www.w3.org/2001/XMLSchema"
  xmlns:dsig11="http://www.w3.org/2009/xmldsig11#"
  xmlns:asic="http://uri.etsi.org/02918/v1.2.1#"
  targetNamespace="http://www.bsi.bund.de/tr-esor/enc"
  elementFormDefault="qualified" attributeFormDefault="unqualified"
  version="1.3.0">

  <xs:import namespace="http://www.w3.org/2009/xmldsig11#"
    schemaLocation="./deps/xmldsig11-schema.xsd"/>
  <xs:import namespace="http://uri.etsi.org/02918/v1.2.1#"
    schemaLocation="./deps/en_31916201v010101.xsd"/>
  <xs:import namespace="http://www.bsi.bund.de/tr-esor/xaip"
    schemaLocation="./tr-esor-xaip-v1.3.xsd"/>
  <xs:import namespace="http://www.bsi.bund.de/tr-esor/api/1.3"
    schemaLocation="./tr-esor-interfaces-v1.3.xsd"/>

  <!--===== -->
  <!-- Version 1.3.0 vom 21.02.2025 -->
  <!--===== -->
  <!-- TR-ESOR-ENC common data types -->
  <xs:element name="X509IssuerName" type="xs:string"/>
  <xs:element name="X509SerialNumber" type="xs:string"/>
  <xs:element name="SubjectKeyIdentifier" type="xs:hexBinary"/>
  <xs:complexType name="AuthorityKeyIdentifierType">
    <xs:sequence>
      <xs:element name="keyIdentifier" type="xs:hexBinary" minOccurs="0"/>
      <xs:element name="authorityCertIssuer" type="xs:string" minOccurs="0"/>
      <xs:element name="authorityCertSerialNumber" type="xs:string" minOccurs="0"/>
    </xs:sequence>
  </xs:complexType>
  <xs:element name="AuthorityKeyIdentifier" type="enc:AuthorityKeyIdentifierType"/>
  <xs:complexType name="EncryptedKeyType">
    <xs:simpleContent>
      <xs:extension base="xs:base64Binary">
        <xs:attribute name="Algorithm" type="xs:anyURI" use="required"/>
      </xs:extension>
    </xs:simpleContent>
  </xs:complexType>
  <xs:element name="EncryptedKey" type="enc:EncryptedKeyType"/>
  <xs:complexType name="RecipientInfoType">
    <xs:sequence>
      <xs:element ref="enc:X509IssuerName"/>
      <xs:element ref="enc:X509SerialNumber"/>
      <xs:element ref="enc:SubjectKeyIdentifier" minOccurs="0"/>
      <xs:element ref="enc:AuthorityKeyIdentifier" minOccurs="0"/>
      <xs:element ref="dsig11:X509Digest" minOccurs="0"/>
      <xs:element ref="enc:EncryptedKey" minOccurs="0"/>
    </xs:sequence>
    <xs:attribute name="RecipientInfoID" type="xs:ID" use="required"/>
    <xs:attribute name="KeyEncryptionAlgorithm" type="xs:anyURI" use="optional"/>
  </xs:complexType>
  <xs:element name="RecipientInfo" type="enc:RecipientInfoType"/>
  <xs:complexType name="RecipientInfosType">
    <xs:sequence>
      <xs:element ref="enc:RecipientInfo" maxOccurs="unbounded"/>
    </xs:sequence>
  </xs:complexType>
  <xs:element name="RecipientInfos" type="enc:RecipientInfosType"/>
  <xs:complexType name="AccessControlAIPReferenceType">
    <xs:sequence>
```

```

        <xs:element ref="xaip:AOID"/>
        <xs:element name="Description" type="xs:string" minOccurs="0"/>
    </xs:sequence>
</xs:complexType>
<xs:element name="AccessControlAIPReference" type="enc:AccessControlAIPReferenceType"/>

<!-- ===== -->
<!-- Delete -->
<!-- ===== -->
<xs:element name="DeleteRequest">
    <xs:complexType>
        <xs:complexContent>
            <xs:extension base="tr:RequestType">
                <xs:sequence>
                    <xs:element ref="asic:DataObjectReference" maxOccurs="unbounded"/>
                </xs:sequence>
            </xs:extension>
        </xs:complexContent>
    </xs:complexType>
</xs:element>
<xs:element name="DeleteResponse">
    <xs:complexType>
        <xs:complexContent>
            <xs:extension base="tr:ResponseType"/>
        </xs:complexContent>
    </xs:complexType>
</xs:element>

<!-- ===== -->
<!-- Encryption/Decryption -->
<!-- ===== -->
<xs:element name="TRESOREncMode" type="enc:TRESOREncModeType"
    default="http://www.bsi.bund.de/DE/tr-esor/enc-mode/encdoc"/>
<xs:simpleType name="TRESOREncModeType">
    <xs:restriction base="xs:anyURI">
        <xs:enumeration value="http://www.bsi.bund.de/DE/tr-esor/enc-mode/encdoc"/>
        <xs:enumeration value="http://www.bsi.bund.de/DE/tr-esor/enc-mode/renasym"/>
        <xs:enumeration value="http://www.bsi.bund.de/DE/tr-esor/enc-mode/extacces"/>
    </xs:restriction>
</xs:simpleType>
<xs:element name="InlineDocument" type="xs:base64Binary"/>
<xs:complexType name="TRESOREncDocumentType">
    <xs:sequence>
        <xs:choice>
            <xs:element ref="enc:InlineDocument"/>
            <xs:element ref="asic:DataObjectReference"/>
        </xs:choice>
    </xs:sequence>
    <xs:attribute name="id" type="xs:ID" use="required"/>
</xs:complexType>
<xs:element name="TRESOREncDocument" type="enc:TRESOREncDocumentType"/>
<xs:complexType name="DocumentsToBeEncryptedType">
    <xs:sequence>
        <xs:element ref="enc:TRESOREncDocument" minOccurs="1" maxOccurs="unbounded"/>
    </xs:sequence>
</xs:complexType>
<xs:element name="DocumentsToBeEncrypted" type="enc:DocumentsToBeEncryptedType"/>
<xs:complexType name="EncryptedDocumentsType">
    <xs:sequence>
        <xs:element ref="enc:TRESOREncDocument" minOccurs="1" maxOccurs="unbounded"/>
    </xs:sequence>
    <xs:attribute name="EncryptionAlgorithms" type="xs:anyURI"/>
</xs:complexType>
<xs:element name="EncryptedDocuments" type="enc:EncryptedDocumentsType"/>
<xs:complexType name="DecryptedDocumentsType">
    <xs:sequence>

```



```

    <xs:element ref="enc:TRESOREncDocument" minOccurs="1" maxOccurs="unbounded"/>
  </xs:sequence>
</xs:complexType>
<xs:element name="DecryptedDocuments" type="enc:DecryptedDocumentsType"/>

<!-- ===== -->
<!-- Hash -->
<!-- ===== -->
<xs:complexType name="HashValueType">
  <xs:sequence>
    <xs:element name="Hash" type="xs:hexBinary"/>
  </xs:sequence>
  <xs:attribute name="id" type="xs:ID" use="required"/>
</xs:complexType>
<xs:element name="HashValue" type="enc:HashValueType"/>
<xs:complexType name="HashValuesType">
  <xs:sequence>
    <xs:element ref="enc:HashValue" minOccurs="1" maxOccurs="unbounded"/>
  </xs:sequence>
</xs:complexType>
<xs:element name="HashValues" type="enc:HashValuesType"/>

<!-- ===== -->
<!-- TR-ESOR-ENC VS-NfD reduziert -->
<!-- ===== -->
<xs:complexType name="PlainTextProxyType">
  <xs:attribute name="DencID" type="xs:IDREF" use="required"/>
</xs:complexType>
<xs:element name="PlainTextProxy" type="enc:PlainTextProxyType"/>
</xs:schema>

```